

Seminario de Práctica

Licenciatura en Informática

SISTEMA OMNES

**Diseño de un sistema de gestión administrativa y de
historias clínicas para la fundación OMNES**

Profesor Titular disciplinar: Pablo Alejandro Virgolini

Profesor Tutora Experto: Ana Carolina Ferreyra

David Emanuel Gigena

28 de junio del 2026

Índice

Introducción	5
Justificación.....	5
Definiciones del proyecto y del sistema	5
Elicitación.....	7
Cuestionario utilizado para la entrevista:	8
Resultados de la Elicitación.....	9
Análisis de la competencia.....	13
Matriz comparativa	14
Análisis FODA.....	14
Conocimiento del negocio	15
Diagrama de dominio.....	16
Diagnóstico de procesos relevados.....	17
Propuesta de solución funcional.....	18
Módulos y alcance.....	18
Reglas de negocio clave	25
Propuesta técnica	25
Diagrama de arquitectura.....	26
Requerimientos	26
Requerimientos candidatos	29
Tabla de casos de uso	30
Descripción de casos de uso.....	31
Etapas de análisis.....	37
Diagramas de secuencia.....	37
Etapas de diseño	41
Etapas de implementación.....	47
Diagrama de despliegue.....	47
Etapas de pruebas	48
Plan de pruebas.....	49
Interfaz gráfica	52
Diseño e implementación de la base de datos.....	54

Normalización	59
Creación de tablas MySQL.....	60
Inserción de registros	61
Consultas SQL	63
Eliminación de registros	65
Definiciones de comunicación	65
Requerimientos de comunicación	65
Tecnologías y protocolos utilizados	65
Repositorio del proyecto.....	66
Desarrollo del sistema en Java	66
Introducción.....	66
Diseño orientado a objetos	67
Diagrama de clases de implementación	67
Aplicación de Programación Orientada a Objetos	68
Implementación.....	68
Estructura del proyecto	68
Clases desarrolladas	69
Conceptos de Programación Orientada a Objetos aplicados	70
Conceptos de Programación Orientada a Objetos y Java aplicados.....	76
Manejo de excepciones	79
Menú principal	81
Ejecución y pruebas	81
Registro de pacientes	81
Asignación de turnos	82
Consulta de turnos	83
Manejo de excepciones	83
Repositorio del proyecto.....	84
Implementación del prototipo en Java	84
Selección del patrón de diseño y justificación.....	84
Selección del patrón de diseño y justificación.....	85
Arquitectura MVC	86

Desarrollo de las clases	88
Interfaz gráfica del sistema	89
Persistencia de datos.....	93
Manejo de excepciones	96
Uso de ArrayList.....	97
Verificación y Validación (V&V)	97
Conclusión.....	98
Repositorio del proyecto final	98

Introducción

La Fundación OMNES es una organización ubicada en la ciudad de Córdoba, orientada al desarrollo de servicios educativos y terapéuticos destinados a niños, jóvenes y adultos con o sin discapacidad.

Actualmente, gran parte de sus procesos administrativos y clínicos se realizan de forma manual mediante registros en papel y planillas independientes, dificultando la organización, trazabilidad y acceso a la información.

El proyecto que se presenta a continuación propone el diseño, desarrollo de un sistema informático de gestión administrativa que centralice la gestión de tareas, facturación, registro de historias clínicas y generación de reportes.

Justificación

La necesidad de desarrollar un sistema de gestión administrativa y de historias clínicas surge a partir de las dificultades actuales que presenta la Fundación OMNES en la administración y control de su información clínica y operativa.

El uso de registros manuales y documentación en papel genera problemas de organización, duplicidad de información y dificultades en la trazabilidad de los procesos administrativos y clínicos de la institución. Asimismo, estas limitaciones dificultan la realización de auditorías, el seguimiento de tareas y el acceso rápido a la información por parte de los profesionales y responsables administrativos.

La implementación de un sistema centralizado permitirá optimizar la gestión de la información, mejorar el control de los procesos internos y facilitar el registro y consulta de historias clínicas, contribuyendo tanto a la eficiencia administrativa como al cumplimiento de las normativas vigentes.

Desde el punto de vista social, el proyecto impactará positivamente al asegurar que los profesionales registren correctamente la atención de los pacientes, contribuyendo a una atención de mayor calidad y al cumplimiento de las normativas vigentes.

Definiciones del proyecto y del sistema

Objetivo general:

Desarrollar un sistema de gestión integral que permita digitalizar y centralizar los procesos administrativos y clínicos de la Fundación OMNES, mejorando la trazabilidad de la información, reduciendo errores operativos y asegurando el cumplimiento de las normativas vigentes.

Objetivos específicos

- Gestión de tareas: Esencial para el seguimiento, rastreabilidad y optimización de los procesos en la organización
- Facturación: Pilar fundamental para la sostenibilidad y crecimiento, permitiendo el manejo flexible de porcentajes y control de pagos.
- Gestión de turnos: Asegura la trazabilidad clínica y administrativa, vinculando la atención de pacientes con historias clínicas y facturación.
- Registro de historias clínicas: Primordial para respaldar la transparencia institucional frente a las familias, organismos de control y facilitar la coordinación entre los profesionales.

Planificación global de los objetivos a cumplir:

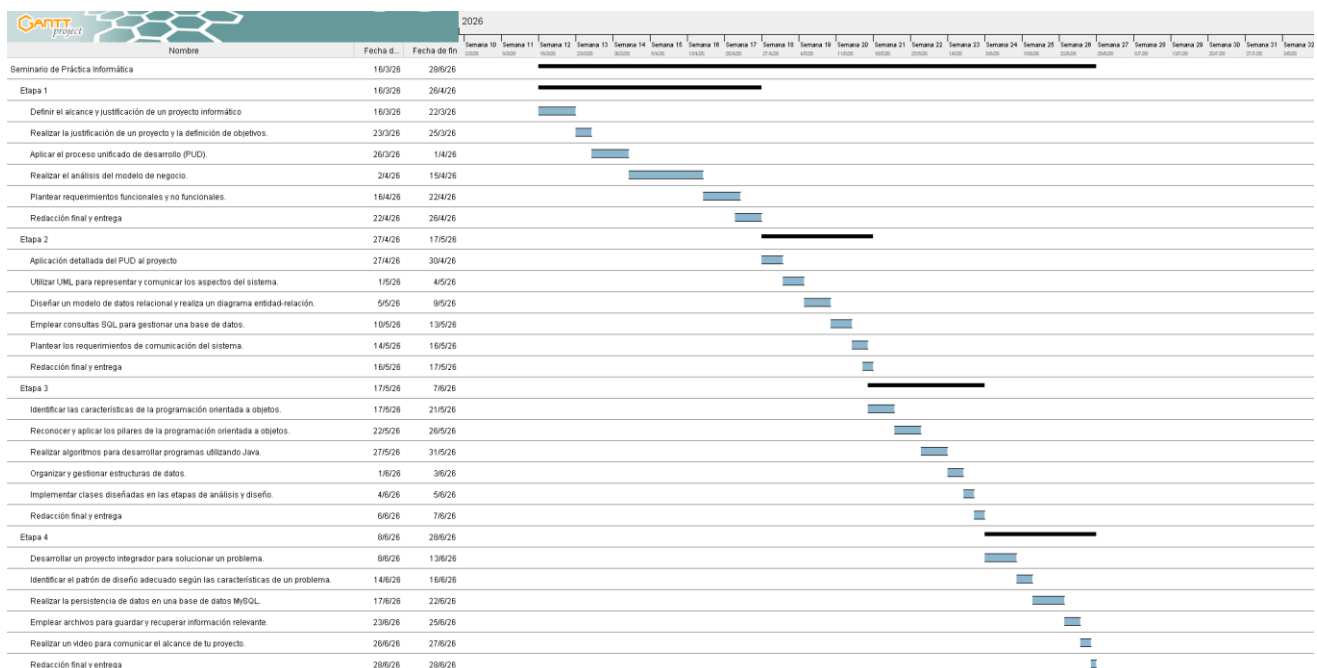


Figura 1 – Diagrama de Gantt con la planificación del proyecto

Objetivos del sistema:

El sistema permitirá digitalizar y centralizar la información crítica de la Fundación OMNES mediante la gestión de tareas, facturación e historias clínicas, con accesos diferenciados por roles de usuario y la posibilidad de generar reportes y auditorías, garantizando trazabilidad, cumplimiento normativo y eficiencia administrativa.

Límites del sistema:

- El sistema no reemplaza la atención profesional: solo registra y audita lo realizado.
- El sistema no gestiona pagos electrónicos automáticos (solo registra y concilia los montos declarados y comprobantes).
- El sistema no realiza diagnósticos clínicos: registra información cargada por los profesionales.
- El acceso está limitado a los usuarios autorizados de la fundación (no es público).

Alcances del sistema:

- Gestión de usuarios y roles: alta, baja y modificación con permisos diferenciados.
- Gestión de tareas: creación, asignación, seguimiento, cierre y reportes.
- Facturación: registro de facturas emitidas por profesionales o por la fundación, cálculo de aportes y conciliación de pagos.
- Gestión de turnos: Organización, registro y asignación de horarios disponibles de los profesionales a pacientes, controlando estados de turnos y evitando solapamientos.
- Historias clínicas: registro inmutable por profesional, cierre diario y cruce con facturación.
- Reportes: generación de balances, reportes clínicos y de rendimiento del personal.
- Auditoría: registro de accesos y acciones críticas en bitácora.

Restricciones del sistema:

- Debe implementarse en Java con interfaz gráfica JavaFX o Swing.
- Debe persistir los datos en MySQL, utilizando JDBC para la conexión.
- Funcionará en red local (LAN/VPN) con acceso a una base de datos centralizada.
- La carga de historias clínicas debe hacerse dentro del día de atención; fuera de ese plazo no se permite registrar y el turno queda no facturable.
- El sistema debe cumplir con normas de auditoría de obras sociales, replicando el formato oficial del libro clínico en sus reportes.

Elicitación

La elicitación es el proceso de adquisición de conocimiento relevante necesario para identificar y documentar los requerimientos del sistema. Para este proyecto se aplicó la técnica de entrevista semiestructurada, con el fin de comprender el funcionamiento actual de la Fundación OMNES, sus procesos y las necesidades de digitalización.

Enfoque: entrevista semiestructurada.

Fases del proceso:

- 1- Planificación
 - a. Identificación de stakeholders.
 - b. Objetivos y alcance de la entrevista.
 - c. Preparación de guía y consentimientos.
- 2- Ejecución
 - a. Entrevistas 1:1 y observación de artefactos (Libro de historias clínicas, cuadernos de tareas, planillas de facturación, PDFs Arca, comprobantes bancarios)
- 3- Análisis:
 - a. Limpieza y consolidación de notas.
 - b. Agrupación por temas (tareas, facturación, turnos, historias clínicas, reportes)
 - c. Análisis de la competencia.
- 4- Validación
 - a. Revisión con stakeholders prioritarios.
 - b. Ajustes y cierre del set de requerimientos.

Stakeholders entrevistados:

- Presidenta de la fundación: rol gerencial, administradora de usuarios y responsable de la gestión general.
- Contadora: encargada de la facturación y relación con obras sociales.
- Administrativos: responsables de tareas operativas varias y asignación de turnos.
- Profesionales de la salud: psicología, psicopedagogía, fonoaudiología, psicomotricidad, entre otros.

Técnica aplicada:

Se utilizó un cuestionario de preguntas abiertas y observación directa de los registros actuales en cuadernos y planillas. Las entrevistas permitieron identificar las principales problemáticas y expectativas respecto al nuevo sistema.

Cuestionario utilizado para la entrevista:

Las siguientes preguntas fueron diseñadas como guía orientativa en el proceso de elicitación de requerimientos. Al tratarse de una entrevista semiestructurada, las preguntas sirvieron como punto de partida y la conversación fluyó de manera natural, permitiendo indagar en mayor profundidad según las respuestas de los entrevistados.

- 1- Información general:
 - a. ¿Cuáles son actualmente los principales procesos administrativos de la fundación?
 - b. ¿Qué herramientas utilizan hoy para registrar información (ej.: cuadernos, planillas, software)?
 - c. ¿Cuáles considera que son los mayores problemas en la gestión actual?
- 2- Gestión de usuarios y roles
 - a. ¿Quiénes utilizan (o utilizarán) el sistema dentro de la fundación?
 - b. ¿Qué niveles de acceso deberían tener?
 - c. ¿Quién debería estar autorizado para dar de alta o baja a otros usuarios?
- 3- Gestión de tareas
 - a. ¿Cómo se asignan y controlan hoy las tareas del personal?
 - b. ¿Qué problemas aparecen con el método actual (ej.: olvidos, falta de seguimiento)?
 - c. ¿Qué funcionalidades les gustaría que tenga un módulo de gestión de tareas?
- 4- Facturación e ingresos
 - a. ¿Cómo es hoy el proceso de facturación con los profesionales y obras sociales?
 - b. ¿Cómo registran los porcentajes que cada profesional aporta a la fundación?
 - c. ¿Qué dificultades enfrentan con los pagos diferidos de las obras sociales?
 - d. ¿Les resultaría útil que el sistema lea automáticamente facturas en PDF o comprobantes bancarios?
- 5- Gestión de turnos:
 - a. ¿Cómo se gestionan los turnos actualmente? ¿Quién es el encargado de asignarlos?
 - b. ¿Hay un registro de los estados de los turnos?

- c. ¿La facturación toma solo turnos Atendidos?
- d. ¿Qué reportes necesitan: asistencia, no-show, uso de consultorios, cancelaciones?
- 6- Historias clínicas
 - a. ¿Cómo se registran actualmente las atenciones de los pacientes?
 - b. ¿Qué problemas se generan con el libro físico de historias clínicas?
 - c. ¿Qué aspectos deberían ser obligatorios en el registro digital (ej.: fecha, profesional, disciplina, firma digital)?
 - d. ¿Necesitan que los profesionales puedan consultar la historia clínica completa de un paciente? ¿Con qué restricciones?
- 7- Reportes y auditoría
 - a. ¿Qué informes necesitan generar regularmente (ej.: balances, auditorías, rendimiento del personal, estadísticas de pacientes)?
 - b. ¿Con qué frecuencia deberían generarse estos reportes (mensual, anual, por consulta)?
 - c. ¿Consideran importante registrar en el sistema quién accedió a qué información y cuándo?

Resultados de la Elicitación

A partir de las entrevistas realizadas a la presidenta de la fundación, la contadora, los administrativos y los profesionales de la salud, se identificaron las principales problemáticas a través de un mapeo AS-IS:

1- Gestión de usuarios y roles

- Problema Actual (AS-IS): No existe control de accesos, todos comparten cuadernos y registros físicos con riesgo de errores y falta de seguridad.

Diagrama general de casos de uso:

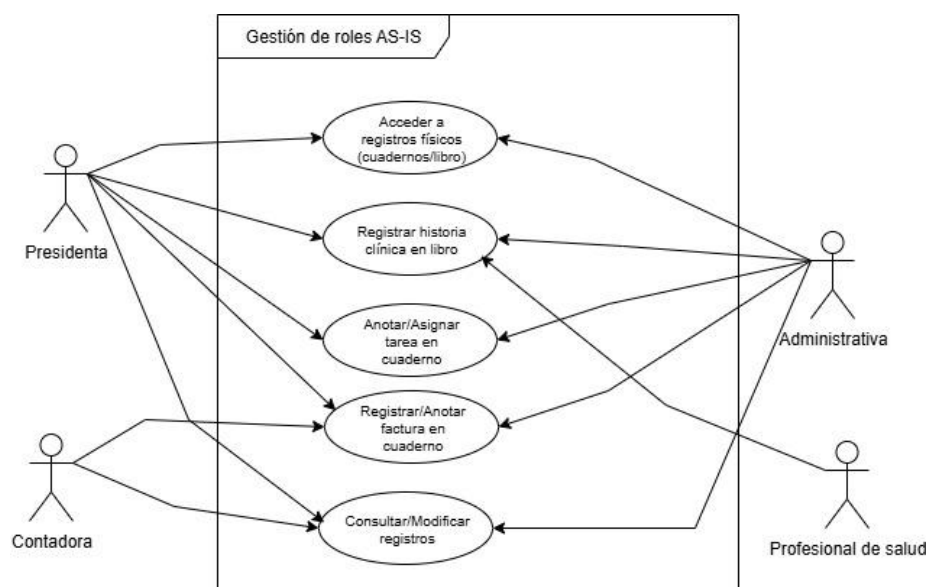


Figura 2 – Diagrama general de casos de uso gestión de usuarios y roles

Todos los roles (presidenta, contadora y administrativas) acceden de manera indistinta a los registros físicos. Los profesionales registran manualmente en el libro de historias clínicas, pero también pueden modificar o cargar en fechas posteriores no respetando la normativa. No existe un esquema de permisos ni trazabilidad de acciones. Cualquier actor puede consultar y modificar contenidos, registrar historias clínicas, tareas o facturas. Este modelo explica los problemas de seguridad, errores, pérdida de información y dificultad de auditoría detectados en la elicitación.

2- Gestión de tareas

- Problema Actual (AS-IS): Se anotan en cuadernos o se delegan tareas en forma verbal con riesgo de tareas olvidadas, sin seguimiento ni trazabilidad.

Diagrama general de actividades:

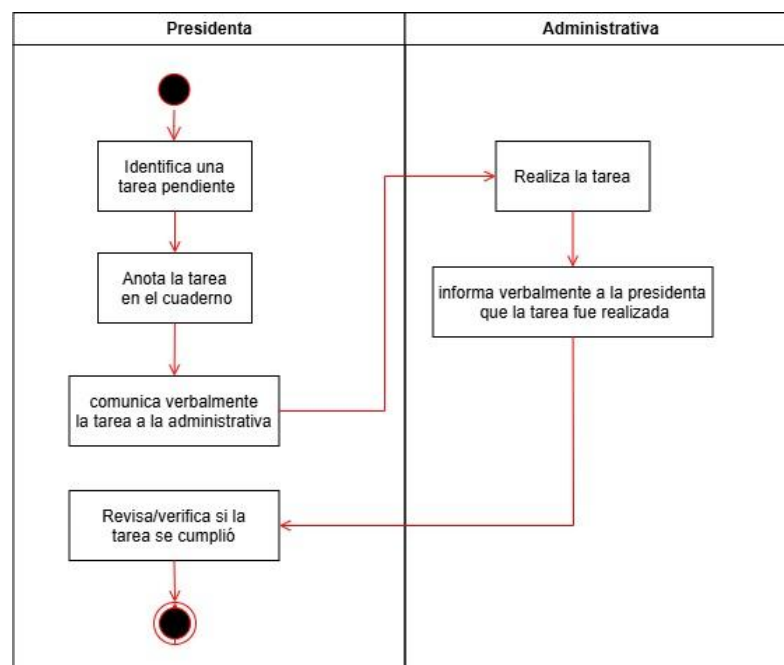


Figura 3 – Diagrama general de actividades Gestión de tareas

Las tareas se registran en cuadernos y se comunican de forma informal. No hay estados, vencimientos ni notificaciones, por lo que se producen olvidos y falta de trazabilidad.

3- Facturación

- Problema actual (AS-IS): Registro manual en cuadernos; porcentajes variables; demoras en pagos de obras sociales; conciliación bancaria manual.

Diagrama general de actividades:

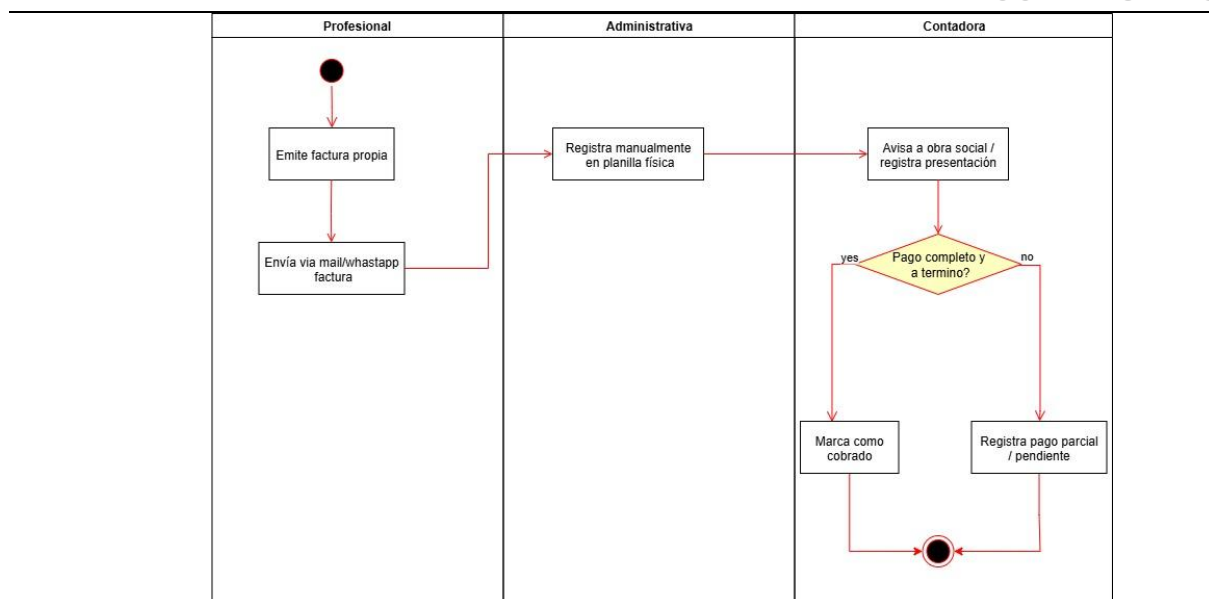


Figura 4 – Diagrama general de actividades Facturación

Proceso manual, porcentajes a la fundación calculados aparte y conciliación bancaria manual; alta posibilidad de errores y demoras, no hay back-up de información del libro físico por lo que también hay riesgo alto de pérdida de información.

4- Gestión de turnos

- Problema actual (AS-IS): Registro manual de turnos según disponibilidad de profesionales, se intenta completar la jornada.

Diagrama general de actividades:

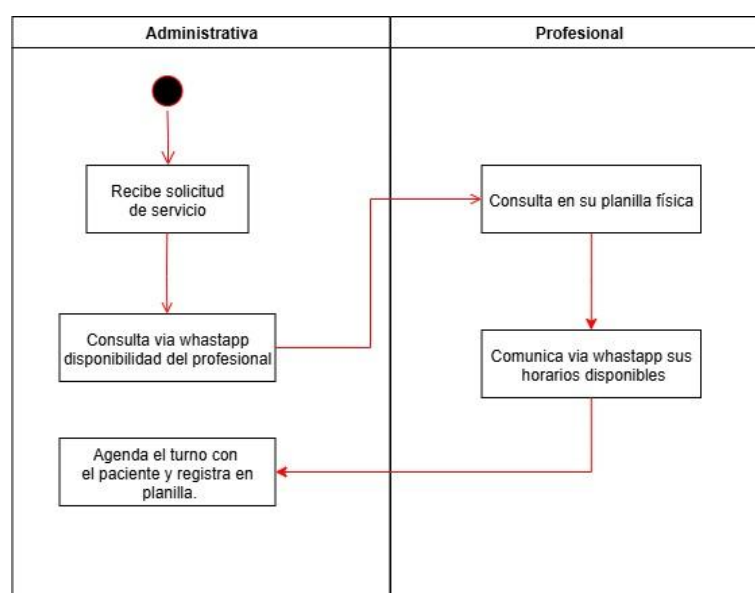


Figura 5 – Diagrama general de actividades Gestión de turnos

Los turnos se coordinan de forma informal (cuaderno/mensajes), sin control de solapamientos ni registros de reprogramaciones o cancelaciones, lo que dificulta la trazabilidad clínica y administrativa.

5- Historias clínicas

- Problema actual (AS-IS): Libro físico; registros tardíos; reemplazos indebidos de profesionales; falta de trazabilidad.

Diagrama general de actividades:

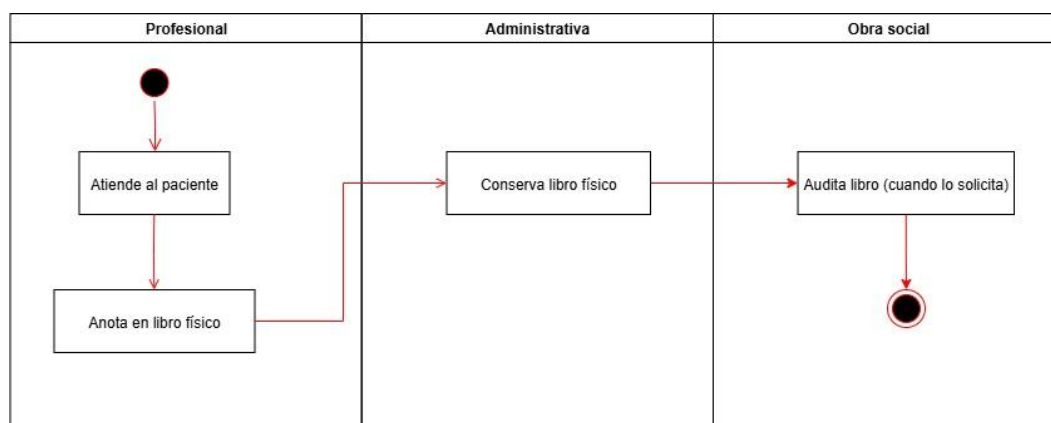


Figura 6 – Diagrama general de actividades Historias clínicas

Los registros se realizan en un libro físico, con problemas de registros tardíos, reemplazos indebidos de profesionales y falta de trazabilidad. Las auditorías de las obras sociales se dificultan por errores de correlación y demoras en la carga.

6- Reportes

- Problema actual (AS-IS): No existen reportes integrados, el análisis es de forma manual.

Diagrama general de actividades:

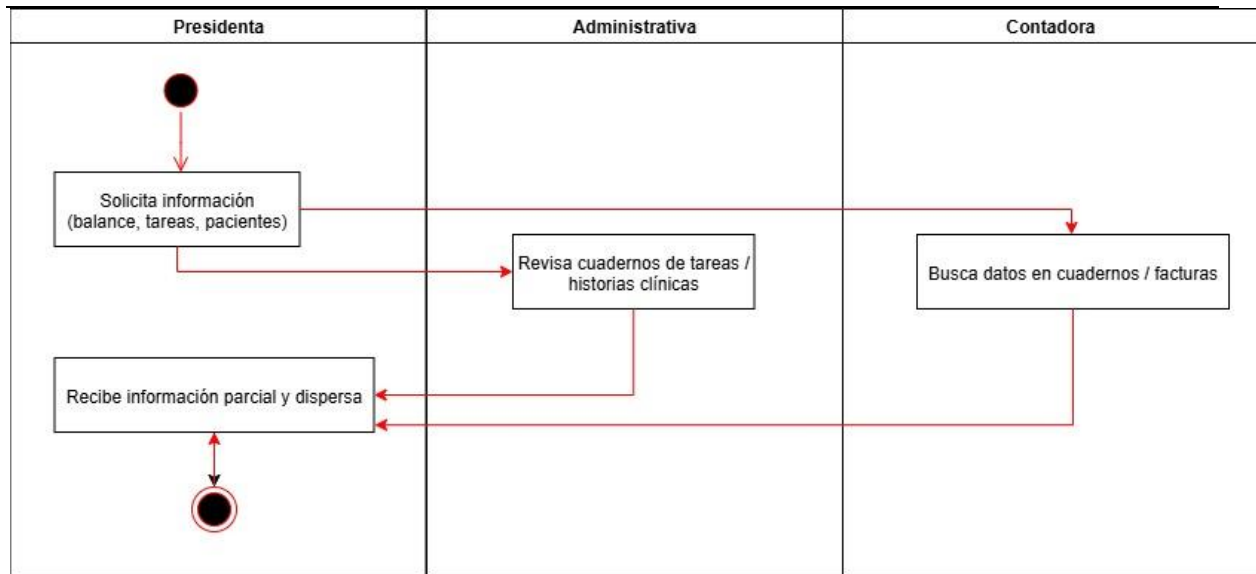


Figura 7 – Diagrama general de actividades Reportes

No existe un sistema centralizado de reportes. La información se encuentra dispersa en cuadernos, facturas físicas y registros manuales, lo que dificulta obtener balances, estadísticas o auditorías en tiempo y forma.

Análisis de la competencia

Actualmente existen en el mercado diversos sistemas de gestión clínica y administrativa, como Galeno, TurnosWeb u Omnia salud, que ofrecen soluciones integrales para instituciones de salud. Estos sistemas suelen incluir módulos de gestión de pacientes, turnos en línea, facturación electrónica y generación de reportes.

Sin embargo presentan algunas limitaciones para el caso de la Fundación OMNES:

- Costos elevados de licencia: al ser una institución joven, no pueden afrontar los costos recurrentes de estos softwares.
- Rigidez funcional: las soluciones comerciales están diseñadas para clínicas o sanatorios ya consolidados y en algunos casos no contempla la situación y necesidad particular de esta fundación.
- Escasa personalización: adaptar estas herramientas a la realidad local implicaría costos adicionales de consultoría e implementación.
- Dependencia tecnológica: en muchos casos se requiere infraestructura en la nube y soporte especializado que encarece aún más la adopción.

En contraste, el sistema propuesto:

- Será desarrollado a medida, tomando en cuenta las necesidades específicas de la Fundación (gestión de tareas, turnos, facturación mixta, historias clínicas y reportes).

- Utilizará Java y MySQL, tecnologías robustas y de libre acceso, evitando costos de licencias.
- Permitirá una evolución gradual: inicialmente cubrirá la modalidad actual (profesionales monotributistas), y en el futuro podrá adaptarse a un esquema centralizado cuando la Fundación obtenga la habilitación completa.
- Ofrecerá trazabilidad y transparencia, mejorando la organización y facilitando auditorías, algo crítico para la sustentabilidad de la institución.

Matriz comparativa

Funcionalidad / Criterio	Software Comercial (Galeno, TurnosWeb, Omnia salud)	Propuesta Fundación OMNES
Gestión de tareas internas	Limitada o inexistente	Completa y adaptada
Gestión de turnos	Incluida, pero poco flexible	Personalizada al flujo de OMNES
Registro de historias clínicas	Incluida (digital y estandarizada)	Adaptada a requerimientos normativos locales
Facturación	Facturación electrónica estándar	Mixta: profesionales independientes + futuro centralizado
Reportes	Avanzados, pero genéricos	Específicos: balance, pacientes, auditorías, rendimiento
Costo de licencias	Alto, mensual o anual	Nulo (desarrollo interno, sin licencias)
Personalización	Limitada, con costo adicional	Total, desde el diseño inicial
Escalabilidad	Requiere módulos pagos adicionales	Escalable en base a la realidad de OMNES

Análisis FODA

El análisis FODA permite evaluar tanto los factores internos como externos que influyen en el desarrollo del sistema propuesto. Este enfoque proporciona una visión estratégica que complementa el relevamiento técnico, ayudando a justificar por qué el desarrollo de un software propio resulta la mejor opción para la Fundación OMNES frente a las alternativas comerciales disponibles.



Figura 8 – Análisis FODA sistema propuesto para la fundación OMNES

A partir de la entrevista realizada con los actores principales de la Fundación OMNES y del análisis de los procesos actuales (AS-IS), se identificaron problemáticas críticas relacionadas con la gestión manual y dispersa de la información: tareas anotadas en cuadernos, turnos asignados por WhatsApp, historias clínicas sin trazabilidad, y facturación dependiente de registros físicos.

Asimismo, el análisis de la competencia y el FODA confirmaron que un desarrollo propio, modular y basado en tecnologías libres, no solo es la alternativa más viable desde el punto de vista económico, sino también la que mejor se adapta a la realidad y proyección de la Fundación.

En conclusión, la elicitación permitió definir con claridad los alcances y lineamientos del sistema, constituyendo la base para avanzar hacia la especificación de requerimientos.

Conocimiento del negocio

La Fundación OMNES es una institución dedicada a la creación de servicios educativos, inclusivos y de apoyo al desarrollo de las capacidades individuales. Su actividad principal se centra en la prestación de servicios clínicos y pedagógicos a través de profesionales de distintas disciplinas (fonoaudiología, psicología, psicopedagogía, psicomotricidad, acompañamiento terapéutico, entre otros).

Actualmente, la gestión administrativa de la Fundación se realiza de manera manual: las tareas se anotan en cuadernos, los turnos se coordinan por mensajería instantánea, las historias clínicas se registran en formato papel y la facturación depende de cada profesional, que emite comprobantes

como monotributista. Esta modalidad genera problemas de trazabilidad, duplicación de información, falta de control y dificultades al momento de realizar auditorías de obras sociales.

El sistema propuesto se integra directamente en este contexto como una herramienta que permitirá digitalizar y centralizar los procesos de gestión. De esta forma, se busca optimizar la administración de turnos, tareas, historias clínicas y facturación, brindando mayor confiabilidad de la información y asegurando el cumplimiento de requisitos normativos.

La solución no solo responde a la problemática actual, sino que también se diseña para acompañar el crecimiento de la Fundación, contemplando escenarios futuros donde la institución facture directamente y requiera reportes avanzados para organismos de control.

Diagrama de dominio

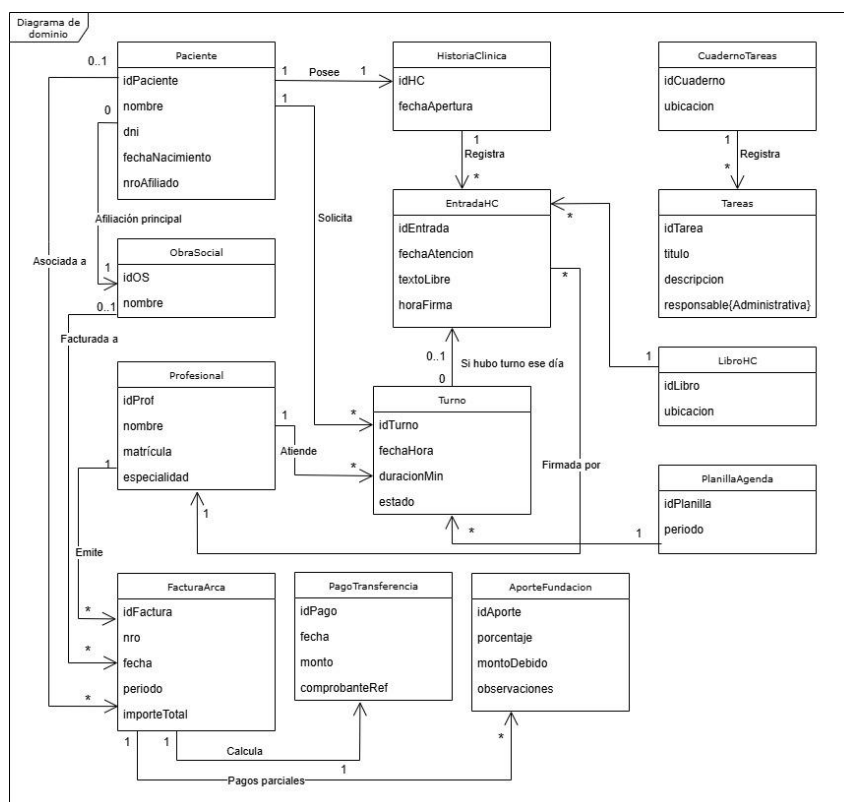


Figura 9 – Diagrama de dominio estado actual organizativo OMNES

El diagrama de dominio presentado describe el estado actual de los procesos de la Fundación OMNES a partir de las entidades clave que intervienen en la operatoria diaria. En él se representan los principales elementos del negocio —usuarios, pacientes, historias clínicas, facturación, turnos, tareas y reportes— junto con sus atributos más relevantes y las relaciones que los vinculan.

Este modelo permite visualizar de forma clara cómo se gestionan actualmente los registros, destacando la dependencia de cuadernos físicos, planillas manuales y comunicación informal. Asimismo, funciona como base para identificar las ineficiencias y riesgos que fueron diagnosticados en

cada proceso (duplicación de información, falta de trazabilidad, riesgo de pérdida de datos, ausencia de control de accesos).

De esta manera, el diagrama de dominio no solo sintetiza el conocimiento del negocio relevado, sino que también constituye un insumo fundamental para la siguiente etapa del análisis: el diagnóstico de procesos y la propuesta de solución a través del sistema informático a desarrollar.

Diagnóstico de procesos relevados

1- Proceso: Gestión de usuario y roles

Problemas

- Altas/bajas/permisos informales: no hay trazabilidad ni control de accesos.
- Riesgo de compartir credenciales de accesos indebidos a información sensible.

Causas

- Ausencia de sistema de autenticación y perfiles.
- Falta de política formal de seguridad y registro de auditoría.

2- Proceso: Gestión de tareas

Problemas

- Tareas olvidadas/duplicadas, estados ambiguos, poca responsabilidad asignada.
- Imposible medir cumplimiento ni tiempos; no hay evidencias.

Causas

- Registro manual en cuadernos y comunicación verbal/WhatsApp.
- No existe bandeja única, notificaciones ni reportes.

3- Proceso: Gestión de turnos

Problemas

- Solapamientos de horarios y disponibilidad desactualizada.
- Reprogramaciones/cancelaciones sin historial; difícil justificar ausencias/no-show.

Causas

- Agenda distribuida en planillas físicas y mensajería.
- No hay calendario central ni reglas automáticas de validación.

4- Proceso: Historias clínicas

Problemas

- Registros tardíos o fuera de secuencia; riesgo de inconsistencias.

- Posibilidad de “reemplazos” indebidos de profesionales.
- Dificultad para auditorías por falta de trazabilidad e inmutabilidad.

Causas

- Único soporte en libro físico; sin bloqueo ni firma digital.
- Cultura de carga diferida; ausencia de controles y reglas de negocio en el registro.

5- Proceso: Facturación

Problemas

- Conciliación manual entre facturas ARCA, porcentajes y pagos; errores y demoras.
- Pagos parciales sin control de saldo; difícil saber “qué falta cobrar”.
- Dependencia de que la administrativa cargue datos que debería aportar el profesional.

Causas

- Falta de lectura automática de PDF ARCA y vinculación a pacientes/turnos.
- Porcentaje de aporte calculado “a mano” y sin historial.
- Comprobantes de transferencia enviados por fuera (correo/WhatsApp) y sin asociación a la factura.

6- Proceso: Reportes

Problemas

- Información parcial y dispersa; tiempos altos para responder a la Presidencia.
- Errores/omisiones por consolidación manual; baja confiabilidad para auditorías.

Causas

- Datos distribuidos en cuadernos/libros/planillas y facturas físicas.
- No existe base de datos central ni consultas/indicadores estandarizados.

Propuesta de solución funcional

A partir de las problemáticas detectadas en el análisis AS-IS y de las necesidades relevadas durante el proceso de elicitación, se realizó un mapeo TO-BE con el objetivo de modelar el funcionamiento futuro de los procesos administrativos y clínicos de la Fundación OMNES mediante una solución informática con el propósito de digitalizar y centralizar la operación de la Fundación OMNES para asegurar trazabilidad clínica/administrativa, reducir errores y disponer de reportes confiables.

Módulos y alcance

1. **Gestión de usuarios y roles:** El sistema incorporará un modelo de autenticación y control de accesos basado en roles diferenciados, permitiendo administrar permisos y restricciones según las responsabilidades de cada usuario dentro de la fundación.

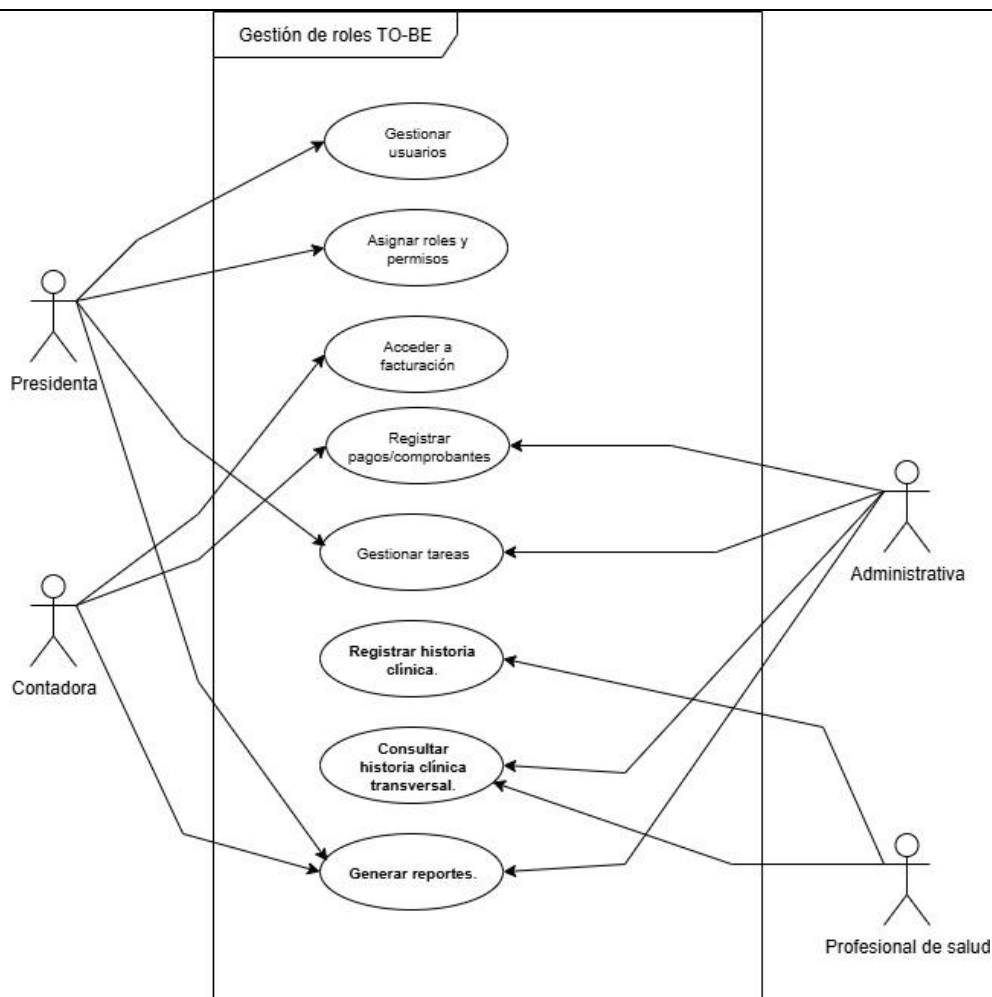


Figura 10 – Diagrama de Casos de Uso TO-BE (Gestión de Roles)

- La presidenta actuará como administradora general del sistema, gestionando usuarios, asignando permisos y supervisando las operaciones globales.
- La Contadora tendrá acceso a los módulos de facturación y reportes contables.
- Las Administrativas se enfocarán en la gestión operativa de tareas, turnos y soporte administrativo.
- Los Profesionales de salud podrán registrar historias clínicas y consultar información clínica relacionada con los pacientes asignados.

El modelo propuesto permitirá mantener trazabilidad completa de las acciones realizadas, garantizando seguridad, control de accesos y cumplimiento normativo.

- Gestión de tareas:** La propuesta contempla un sistema centralizado de gestión de tareas que permitirá crear, aprobar, asignar y realizar seguimiento de actividades internas de la fundación.



Figura 11 – Diagrama de Casos de Uso TO-BE (Gestión de Tareas)

Cada tarea contará con:

- a) Estados de avance, vencimientos, comentarios y adjuntos, permitiendo mantener un registro auditado de las acciones realizadas por los distintos usuarios.
- b) Notificaciones internas y recordatorios automáticos para mejorar el seguimiento operativo y reducir incumplimientos.

La Presidencia podrá acceder a reportes por responsable, estado y período, facilitando el control de la operación y la supervisión institucional.

3. **Facturación:** El módulo de facturación se diseñó contemplando tanto el funcionamiento actual de la fundación como posibles escenarios futuros de crecimiento y centralización administrativa.

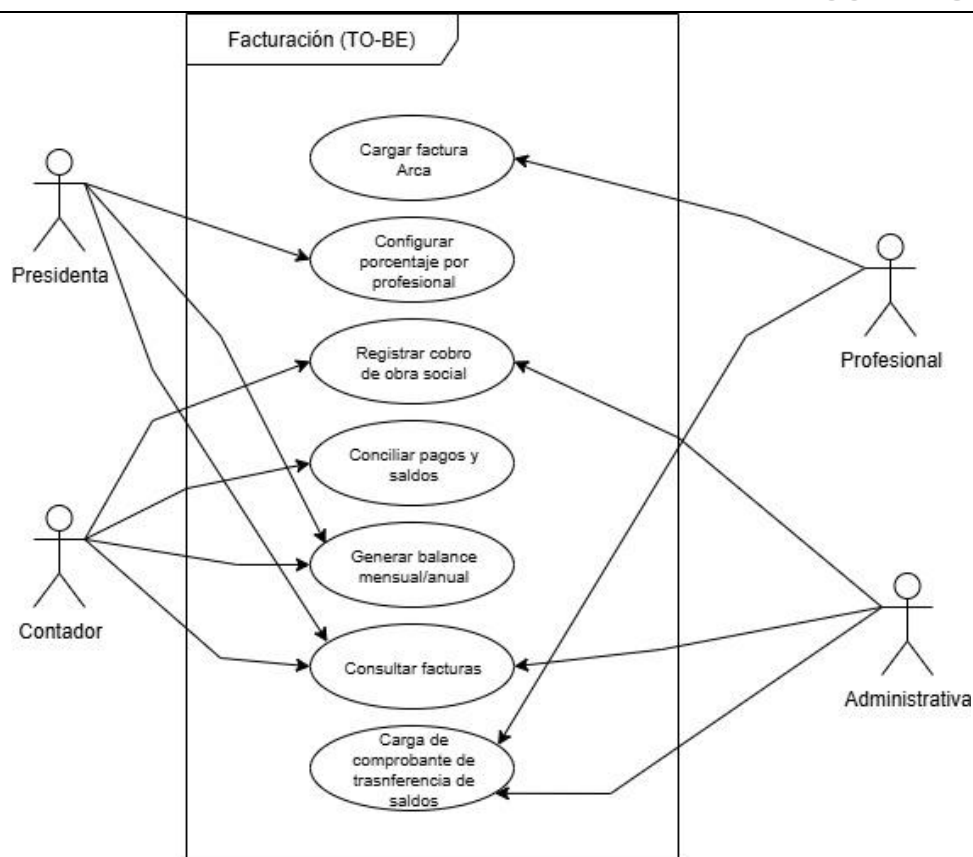


Figura 12 – Diagrama de Casos de Uso TO-BE (Facturación)

- Los Profesionales podrán cargar comprobantes PDF emitidos mediante ARCA, permitiendo al sistema extraer automáticamente los datos relevantes para registrar la facturación correspondiente.
- La Presidenta podrá configurar porcentajes variables asociados a cada profesional, permitiendo adaptar los cálculos según las condiciones administrativas definidas por la fundación.
- La Contadora tendrá acceso al registro y conciliación de pagos, controlando saldos pendientes, cobros parciales o totales y verificando la consistencia contable de la información registrada.
- Las Administrativas podrán consultar facturas, cargar comprobantes de transferencias y colaborar en el seguimiento operativo de la documentación administrativa.
- El sistema permitirá registrar cobros provenientes de obras sociales y generar balances mensuales y anuales, centralizando la información contable de la institución.

El modelo propuesto permitirá mejorar la trazabilidad administrativa y financiera de la fundación, reduciendo errores operativos y facilitando los procesos de conciliación y control contable.

4. **Gestión de turnos:** El sistema incorporará una agenda digital centralizada por profesional, permitiendo administrar turnos mediante asignación, reprogramación, cancelación y control automático de solapamientos.

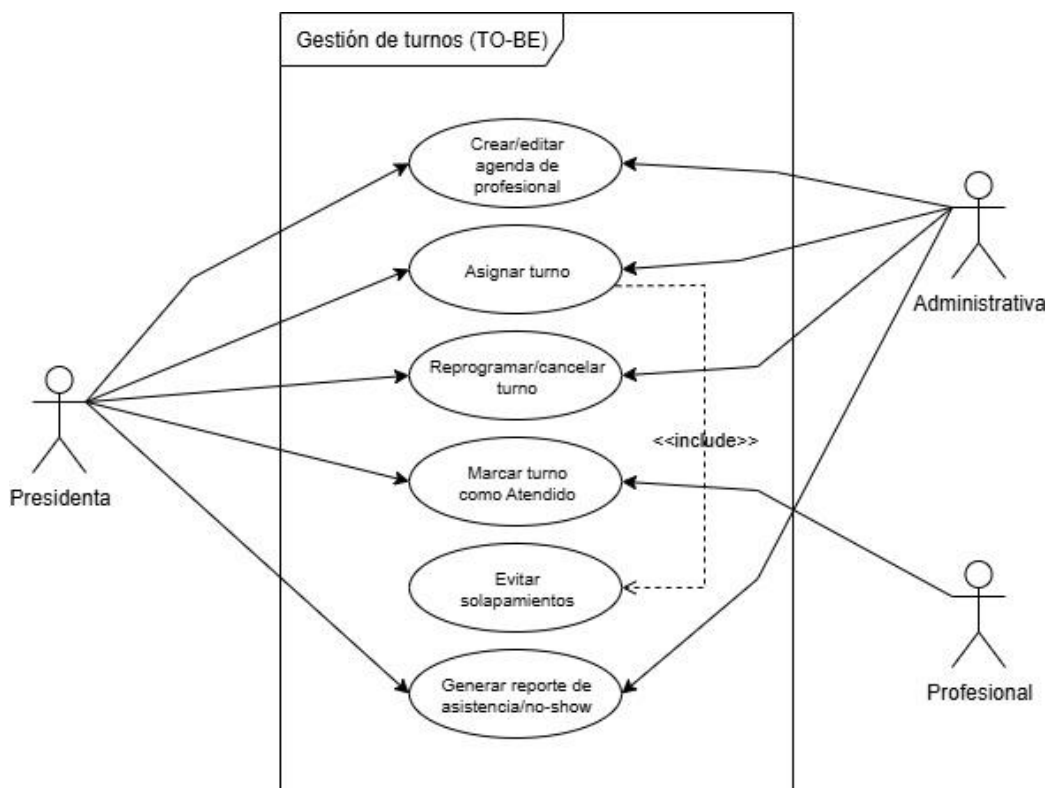


Figura 13 – Diagrama de Casos de Uso TO-BE (Gestión de turnos)

- Las Administrativas podrán registrar, asignar, reprogramar y cancelar turnos, definiendo fecha, horario, paciente y profesional correspondiente.
- Los Profesionales tendrán acceso a la visualización de su agenda diaria y podrán consultar el estado de los turnos asignados.
- El sistema controlará automáticamente posibles solapamientos de horarios, evitando conflictos en la asignación de turnos y mejorando la organización operativa.
- Los turnos manejarán distintos estados operativos, tales como Programado, Reprogramado, Cancelado, Atendido y Ausente, permitiendo mantener trazabilidad administrativa y clínica de las atenciones realizadas.
- Cuando un turno sea marcado como “Atendido”, el sistema habilitará automáticamente el registro de la historia clínica correspondiente a dicha atención.
- El módulo de facturación considerará únicamente los turnos efectivamente atendidos para los procesos administrativos y contables definidos por la fundación.
- El sistema permitirá generar reportes de asistencia y ausentismo (no-show), facilitando el seguimiento operativo y el análisis administrativo de las prestaciones realizadas.

El modelo propuesto permitirá mejorar la organización de agendas profesionales, reducir errores operativos en la asignación de turnos y fortalecer la integración entre la gestión clínica y administrativa de la institución.

5. **Historias clínicas:** La propuesta funcional contempla la digitalización completa del registro de historias clínicas mediante un modelo de carga diaria realizado por los profesionales responsables de cada atención.

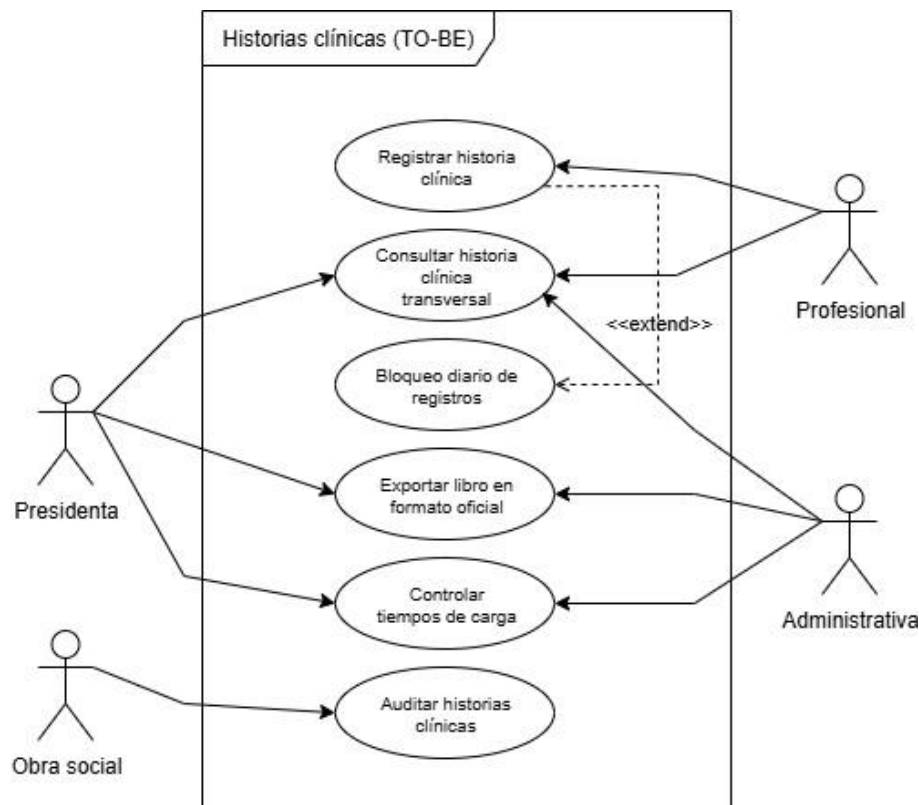


Figura 14 – Diagrama de Casos de Uso TO-BE (Historias Clínicas)

- a) Los Profesionales podrán registrar digitalmente las historias clínicas correspondientes a las atenciones realizadas durante la jornada, asociando cada registro al paciente y turno correspondiente.
- b) El sistema implementará mecanismos de control temporal y bloqueo automático de registros luego del cierre diario, evitando modificaciones posteriores y garantizando inmutabilidad y trazabilidad de la información clínica.
- c) Los Profesionales asignados a un mismo paciente podrán consultar historias clínicas de manera transversal en modo lectura, manteniendo permisos diferenciados según el rol y las responsabilidades de cada usuario.
- d) Las Administrativas y la presidenta podrán exportar libros clínicos en formatos compatibles con auditorías institucionales y controles de obras sociales.

- e) El sistema emitirá alertas y controles relacionados con el cumplimiento de carga diaria de historias clínicas, facilitando el seguimiento administrativo y reduciendo inconsistencias documentales.

El modelo propuesto permitirá mejorar la trazabilidad de la información clínica, fortalecer los mecanismos de control institucional y facilitar el cumplimiento de los requerimientos administrativos y normativos aplicables a la organización.

6. **Reportes:** El sistema contará con un módulo centralizado de generación de reportes administrativos, clínicos y operativos, permitiendo consolidar información crítica de la institución para la toma de decisiones.

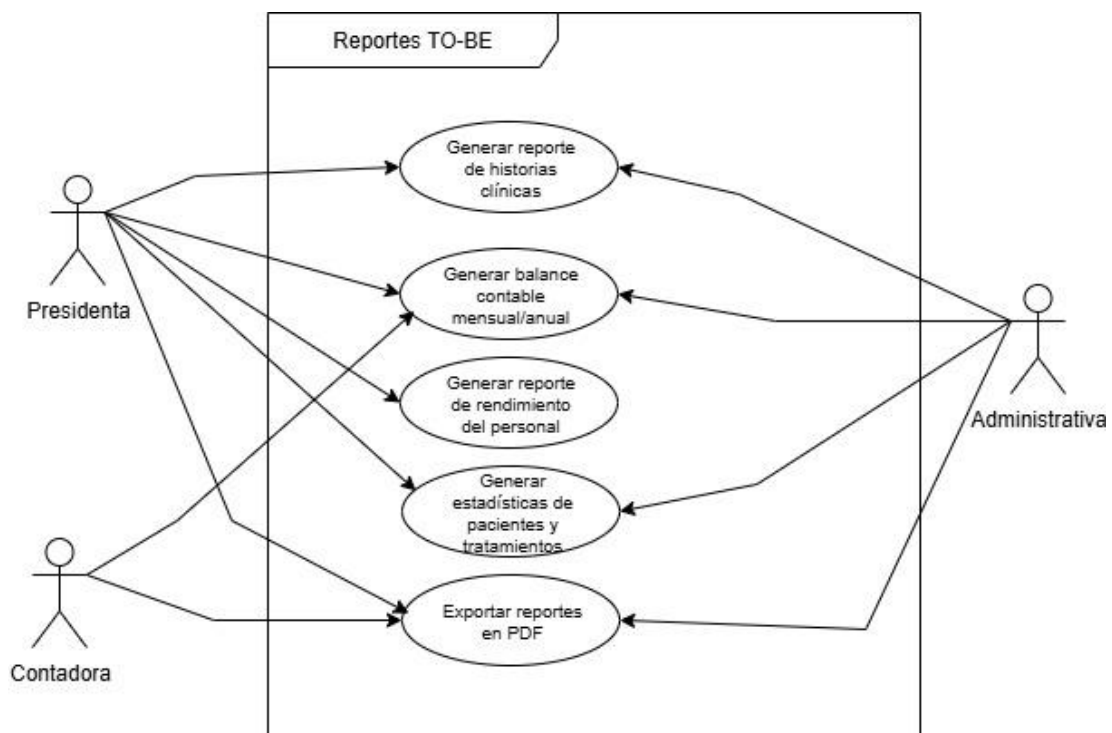


Figura 15 – Diagrama de Casos de Uso TO-BE (Reportes)

- La Presidenta podrá acceder a reportes generales de funcionamiento institucional, incluyendo indicadores administrativos, operativos y clínicos.
- La Contadora tendrá acceso a balances mensuales y anuales, conciliaciones de pagos y reportes financieros asociados a la operatoria de la fundación.
- Las Administrativas podrán consultar reportes relacionados con turnos, asistencia, ausentismo y seguimiento operativo de tareas.
- Los Profesionales podrán acceder a estadísticas vinculadas a pacientes, tratamientos y atenciones realizadas, según los permisos definidos por el sistema.
- El sistema permitirá generar reportes de historias clínicas en formatos compatibles con auditorías institucionales y requerimientos administrativos de obras sociales.

- f) Todos los reportes podrán exportarse en formato digital, facilitando procesos de auditoría, controles internos y análisis operativo de la organización.

El modelo propuesto permitirá centralizar información estratégica de la fundación, facilitando la toma de decisiones y mejorando la disponibilidad y confiabilidad de los datos institucionales.

Reglas de negocio clave

1. Solo el Profesional asignado puede marcar un turno como Atendido y registrar HC del día.
2. HC inmutable después del cierre diario; correcciones por anexo con trazabilidad.
3. Facturas ARCA requieren PDF fuente más porcentaje vigente para calcular aporte/ saldo; pagos pueden ser parciales.
4. Todo cambio relevante queda auditado (quién/cuándo/qué).

Propuesta técnica

- Lenguaje/UI: Java 17 + JavaFX (o Swing) para aplicación escritorio multi-usuario.
- Persistencia: MySQL 8; acceso con JDBC.
- Arquitectura lógica:
 - Presentación (UI): vistas JavaFX/Swing.
 - Aplicación/Servicios: orquesta casos de uso, valida reglas.
 - Dominio: entidades y lógica de negocio (Turno, Paciente, EntradaHC, Factura, etc.).
 - Persistencia (DAO/Repository): clases DAO con SQL preparado, transacciones y mapeos manuales.
- Bibliotecas:
 - Apache PDFBox u OpenPDF para lectura de PDF ARCA y generación de reportes/constancias en PDF.
 - JasperReports opcional para reportes.
- Seguridad: contraseñas con hash (PBKDF2/BCrypt), control de roles en UI y Servicios, bitácora de auditoría.
- Backups: mysqldump diario + copia de adjuntos (facturas/comprobantes) a un NAS/Disco externo.
- Escalabilidad futura: mantener “emisor” en Factura y “ReglaPorcentaje” por profesional; así se conmuta de profesional/fundación sin cambios estructurales.

Arquitectura de despliegue:

Opción recomendada (local centralizada):

- Servidor local (mini-PC/PC) con MySQL 8 y carpeta compartida “/adjuntos”.
- Clientes (notebooks/escritorios) con la app JavaFX conectando por LAN/Wi-Fi vía JDBC.
- Almacenamiento de archivos (PDF facturas, comprobantes, exportes) en recurso compartido del servidor.
- Backups automáticos a NAS/Disco externo (y opcional copia en la nube).

Alternativas:

- Mono-equipo (para piloto): MySQL + app en una sola notebook y carpeta local de adjuntos.
- Acceso remoto: VPN al servidor de la Fundación para uso externo de profesionales (si se habilita).

Diagrama de arquitectura

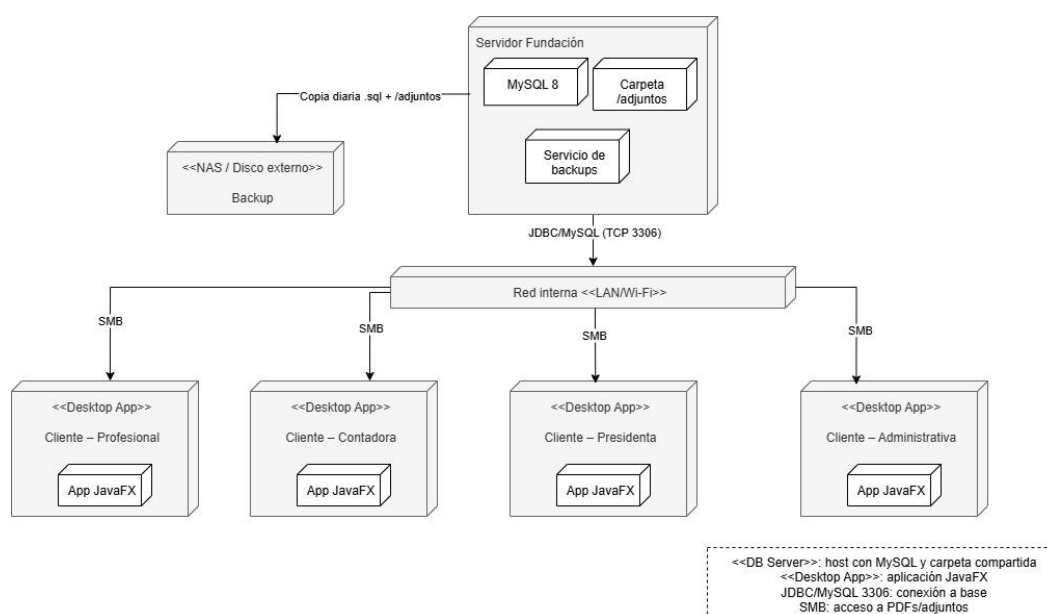


Figura 16 - Diagrama de despliegue de la arquitectura física del sistema propuesto.

Requerimientos

Los requerimientos del sistema representan las funcionalidades y restricciones que debe cumplir el software para responder a las necesidades detectadas en la Fundación OMNES. Fueron obtenidos a través de un proceso de elicitación basado en entrevistas semiestructuradas con los principales stakeholders (presidenta, profesionales, contadora y administrativa), análisis de procesos actuales y diagnóstico de las problemáticas relevadas.

Estos requerimientos constituyen la base del desarrollo, ya que permiten:

- Establecer con precisión qué hará el sistema (requerimientos funcionales) y cómo deberá comportarse (requerimientos no funcionales).
- Asegurar la trazabilidad entre las necesidades del negocio y las soluciones propuestas.
- Servir como contrato entre el cliente y el equipo de desarrollo, delimitando el alcance del proyecto y sus prioridades.

A continuación, se detallan los requerimientos identificados para el sistema, organizados en funcionales y no funcionales.

Catálogo de Requerimientos Funcionales (RF)

(IDs por módulo para orden y trazabilidad)

1. Gestión de usuarios y roles (USU)

- RF-USU-01: Iniciar sesión con usuario y contraseña.
- RF-USU-02: Alta, baja y edición de usuarios (solo presidenta).
- RF-USU-03: Asignar roles y permisos (solo presidenta).
- RF-USU-04: Registrar auditoría (quién/cuándo/qué) de acciones sensibles.
- RF-USU-05: Bloqueo por intentos fallidos y restablecimiento de contraseña.

2. Gestión de tareas (TAR)

- RF-TAR-01: Crear/asignar tareas (solo Presidenta).
- RF-TAR-02: Actualizar estado: Nueva/Asignada/En curso/Bloqueada/Completada/Cerrada
- RF-TAR-03: Comentarios y adjuntos en tareas.
- RF-TAR-04: Bandeja de tareas por usuario, filtros y búsqueda.
- RF-TAR-05: Recordatorio interno por vencimiento.

3. Facturación (FAC) – escenario actual (profesional emite)

- RF-FAC-01: Subir PDF ARCA de factura (profesional).
- RF-FAC-02: Leer/extraer datos del PDF (nº, fecha, obra social, importe, período).
- RF-FAC-03: Calcular aporte a la fundación según porcentaje por profesional (vigente).
- RF-FAC-04: Registrar comprobante de transferencia (parcial/total) por factura.
- RF-FAC-05: Conciliar: estado de factura (Pendiente/Parcial/Conciliada) y saldo.
- RF-FAC-06: Carga manual por Administrativa si el profesional no puede (con adjunto PDF).
- RF-FAC-07: Configurar porcentajes por profesional y vigencias.

- RF-FAC-08: Soportar emisor = Fundación (futuro) sin cambios estructurales.

4. Gestión de turnos (TUR)

- RF-TUR-01: Crear/editar agenda por profesional.
- RF-TUR-02: Asignar turno (paciente, profesional, fecha/hora, duración).
- RF-TUR-03: Evitar solapamientos por profesional.
- RF-TUR-04: Cambiar estado: Programado/Reprogramado/Cancelado/Atendido/Ausente.
- RF-TUR-05: Marcar Atendido (solo profesional asignado).
- RF-TUR-06: Reportes de asistencia/no-show por período y profesional.
- RF-TUR-07: Recordatorio interno previo al turno.
- RF-TUR-08: Reprogramar/cancelar.

5. Historias clínicas (HC)

- RF-HC-01: Registrar entrada de HC únicamente si existe turno Atendido el mismo día.
- RF-HC-02: Inmutabilidad de la entrada tras cierre diario.
- RF-HC-03: Consulta transversal por profesionales asignados al paciente.
- RF-HC-04: Generar informe en formato libro para auditorías.
- RF-HC-05: Adjuntar archivos a la entrada (evidencias).

6. Reportes (REP)

- RF-REP-01: Balance mensual/anual (deudores, cobranzas, saldos).
- RF-REP-02: Reporte de rendimiento operativo (tareas, cumplimiento, ausencias).
- RF-REP-03: Estadísticas de pacientes y prestaciones por período y obra social.
- RF-REP-04: Exportar reportes a PDF.

Requerimientos No Funcionales (RNF)

- RNF-SEG-01: Contraseñas con hash seguro (PBKDF2/BCrypt).
- RNF-SEG-02: Control de acceso por roles en UI y servicios.
- RNF-SEG-03: Auditoría de acciones: altas/bajas, cambios de estados, registros de HC y facturas.
- RNF-PERF-01: Operaciones comunes < 2 s en red local para consultas típicas.
- RNF-DB-01: Integridad referencial en MySQL (FK, transacciones en operaciones críticas).
- RNF-RES-01: Backups automáticos diarios de BD (.sql) y semanales de adjuntos.
- RNF-CONF-01: Parametrización de duraciones de turno y listas de estados.

- RNF-USAB-01: UI consistente (atajos, validaciones en formulario, mensajes claros).
- RNF-CUMP-01: Soporte a auditorías (exportes de libro HC y trazabilidad).

Requerimientos candidatos

Los siguientes requerimientos se consideran candidatos, ya que no resultan indispensables en la versión inicial del sistema, pero representan funcionalidades que podrían agregarse a futuro para ampliar el alcance y mejorar la experiencia de uso:

Funcionales (RFC):

- RFC01: Integración con portales de obras sociales para registrar automáticamente la facturación y estado de pagos.
- RFC02: Generación de recordatorios automáticos por correo electrónico o SMS a profesionales y pacientes sobre turnos asignados.
- RFC03: Implementación de firma digital para validar y autenticar historias clínicas.
- RFC04: Exportación de reportes en formatos adicionales (Excel, gráficos interactivos, dashboards).
- RFC05: Acceso remoto seguro para profesionales externos mediante VPN o aplicación móvil complementaria.

No funcionales (RNC):

- RNC01: Migración futura del sistema a una arquitectura cliente-servidor en la nube para mayor escalabilidad.
- RNC02: Implementación de un sistema de auditoría detallada (log completo de actividades por usuario).
- RNC03: Aplicación de cifrado de datos sensibles en la base de datos (ejemplo: datos de pacientes e historiales clínicos).

Diagrama de casos de uso Global

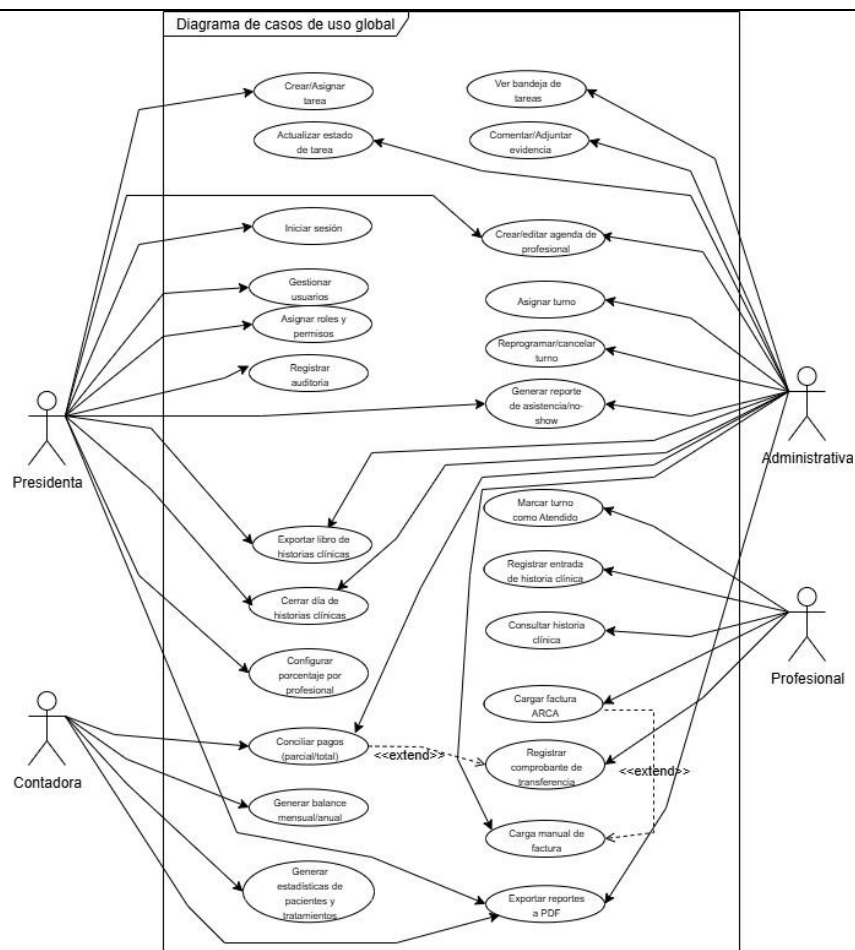


Figura 17 - Diagrama de casos de uso global correspondiente al sistema integral de gestión para la Fundación OMNES.

El diagrama de casos de uso global muestra la interacción de los distintos actores con el sistema propuesto, organizando las funcionalidades principales en seis módulos: gestión de usuarios y roles, gestión de tareas, facturación, gestión de turnos, registro de historias clínicas y reportes. Este modelo permite visualizar de manera integral cómo cada actor participa en el sistema y cuáles son las acciones que podrá realizar, garantizando trazabilidad entre los requerimientos levantados en la etapa de elicitación y las funcionalidades del sistema.

Tabla de casos de uso

Requerimiento	Caso de uso	Actor principal	Paquete de análisis	de	Comentarios
RF-USU-01	CU-USU-01	Presidenta	Usuarios y seguridad		Iniciar sesión
RF-USU-02	CU-USU-02	Presidenta	Usuarios y seguridad	y	Gestionar usuarios

RF-USU-03	CU-USU-03	Presidenta	Usuarios seguridad y	Asignar roles y permisos
RF-TAR-01	CU-TAR-01	Presidenta	Tareas	Crear/Asignar tarea
RF-TAR-02	CU-TAR-02	Administrativa	Tareas	Actualizar estado de tarea
RF-TAR-03	CU-TAR-03	Administrativa	Tareas	Comentar/Adjuntar evidencia
RF-TAR-04	CU-TAR-04	Administrativa	Tareas	Ver bandeja de tareas
RF-FAC-01	CU-FAC-01	Profesional	Facturación	Cargar factura ARCA
RF-FAC-04	CU-FAC-02	Profesional	Facturación	Registrar comprobante de transferencia
RF-FAC-05	CU-FAC-03	Contadora	Facturación	Conciliar pagos
RF-FAC-07	CU-FAC-04	Presidenta	Facturación	Configurar porcentaje por profesional
RF-FAC-06	CU-FAC-05	Administrativa	Facturación	Carga manual de factura (contingencia)
RF-TUR-02	CU-TUR-01	Presidenta	Turnos	Crear/editar agenda de profesional
RF-TUR-02	CU-TUR-02	Administrativa	Turnos	Asignar turno
RF-TUR-08	CU-TUR-03	Administrativa	Turnos	Reprogramar/cancelar turno
RF-TUR-04	CU-TUR-04	Profesional	Turnos	Marcar turno como Atendido
RF-TUR-06	CU-TUR-05	Administrativa	Turnos	Reporte asistencia/no-show
RF-TUR-03	CU-TUR-06	Administrativa	Turnos	Validar solapamientos
RF-HC-01	CU-HC-01	Profesional	Historias clínicas	Registrar entrada de historia clínica
RF-HC-03	CU-HC-02	Profesional	Historias clínicas	Consultar historia clínica
RF-HC-04	CU-HC-03	Administrativa	Historias clínicas	Exportar libro de historias clínicas
RF-HC-02	CU-HC-04	Presidenta	Historias clínicas	Cerrar día de historias clínicas
RF-REP-01	CU-REP-01	Contadora	Reportes	Generar balance mensual/anual
RF-REP-02 RF-REP-03	CU-REP-02	Presidenta	Reportes	Estadísticas de pacientes y tratamientos
RF-REP-04	CU-REP-03	Administrativa	Reportes	Exportar reportes a PDF

Descripción de casos de uso

Caso de uso 1: Gestionar usuarios (CU-USU-02)

Actor principal	Presidenta
Referencias	RF-USU-02

Descripción	Permitir que la Presidenta administre los usuarios del sistema de la Fundación OMNES, realizando altas, modificaciones, activaciones, desactivaciones y asignación de roles, con el fin de controlar el acceso y los permisos de utilización de la plataforma.
Precondición	• Debe existir un usuario administrador habilitado en el sistema. • La presidenta debe haber iniciado sesión con permisos administrativos.
Flujo principal	1- La presidenta accede al módulo de gestión de usuarios. 2- El sistema muestra el listado de usuarios registrados y las opciones disponibles. 3- La presidenta selecciona la opción "Crear usuario". 4- La presidenta ingresa los datos correspondientes del nuevo usuario. 5- La presidenta selecciona el rol correspondiente para el usuario. 6- La presidenta confirma la operación. 7- El sistema valida la información ingresada. 8- El sistema registra el nuevo usuario en la base de datos. 9- El sistema asigna el rol seleccionado al usuario. 10- El sistema registra fecha, hora y usuario responsable de la operación realizada. 11- El sistema informa que el usuario fue registrado correctamente.
Postcondición	• El usuario queda registrado en el sistema con el rol correspondiente asignado. • El usuario queda habilitado para acceder a las funcionalidades permitidas según su rol. • El sistema registra trazabilidad de la operación realizada.
Flujo alternativo	• A1: Si la presidenta omite completar datos obligatorios del usuario el sistema informa los campos faltantes y solicita completar la información requerida. • A2: Si la presidenta intenta registrar un usuario ya existente el sistema informa que el usuario ya se encuentra registrado e impide continuar con la operación.

Caso de uso 2: Asignar turno (CU-TUR-02)

Actor principal	Administrativa
Referencias	RF-TUR-02
Descripción	Permitir que la administrativa registre y gestione turnos para pacientes y profesionales de la Fundación OMNES, asegurando disponibilidad horaria, evitando solapamientos y manteniendo trazabilidad administrativa y clínica de las atenciones programadas.
Precondición	- La administrativa debe haber iniciado sesión en el sistema. - El paciente y el profesional están registrados en el sistema. - La agenda del profesional está habilitada.
Flujo principal	- La administrativa accede al módulo de turnos. - Selecciona al paciente y profesional. - Indica fecha, hora y duración del turno. - El sistema valida que no exista solapamiento con otro turno del mismo profesional. - El sistema registra el turno en estado Programado. - Se muestra confirmación en pantalla.
Postcondición	El turno queda registrado en estado "Programado", visible en la agenda del profesional y disponible para los procesos administrativos y clínicos correspondientes.
Flujo alternativo	- A1: Si hay solapamiento, el sistema rechaza la asignación y muestra el conflicto. - A2: Si faltan datos obligatorios, solicita completarlos.

Caso de uso 3: Registrar historia clínica (CU-HC-01)

Actor principal	Profesional
Referencias	RF-HC-01
Descripción	Permitir que el profesional registre la atención de un paciente en su historia clínica, asegurando trazabilidad e inmutabilidad.
Precondición	El profesional debe haber iniciado sesión en el sistema.

	- El profesional tiene un turno marcado como atendido para ese paciente en la fecha actual. - El paciente está registrado en el sistema.
Flujo principal	- El profesional inicia sesión y accede al paciente correspondiente. - Selecciona "Registrar entrada en historia clínica". - Escribe el detalle de la sesión: fecha, actividades realizadas, observaciones. - El sistema registra la entrada asociada al paciente y profesional.
Postcondición	La entrada de historia clínica queda registrada y disponible para consulta por los profesionales autorizados, quedando sujeta al cierre diario definido por el sistema.
Flujo alternativo	- A1: Si el turno no existe o no está marcado como Atendido, el sistema bloquea la operación. - A2: Si el profesional no está asignado al paciente, no puede ver su historia.

Caso de uso 4: Cargar factura ARCA (CU-FAC-01)

Actor principal	Profesional
Referencias	RF-FAC-01
Descripción	Permitir que el profesional registre su facturación mediante la carga de un PDF ARCA, posibilitando la extracción automática de datos y el cálculo del aporte administrativo a la Fundación.
Precondición	- El profesional debe haber iniciado sesión en el sistema. - El profesional tiene un porcentaje de aporte configurado. - El archivo PDF cumple la estructura ARCA.
Flujo principal	- El profesional accede al módulo de facturación. - Selecciona "Cargar factura ARCA (PDF)". - Adjunta el archivo PDF. - El sistema extrae automáticamente número, fecha, obra social, período e importe. - El sistema calcula el porcentaje de aporte que corresponde a la Fundación. - El sistema registra la factura, asociando el archivo PDF y calculando

	automáticamente el saldo y aporte correspondiente. 7- Muestra un resumen con importe total, aporte debido y saldo.
Postcondición	La factura queda registrada en el sistema con su aporte calculado y en espera de comprobante de pago.
Flujo alternativo	• A1: Si el PDF no es legible, el sistema solicita carga manual (Administrativa). • A2: Si no hay porcentaje configurado, bloquea la operación.

Caso de uso 5: Crear/Asignar tarea (CU-TAR-01)

Actor principal	Presidenta
Referencias	RF-TAR-01
Descripción	Permitir que la presidenta registre, asigne y supervise tareas administrativas u operativas dentro de la Fundación OMNES, definiendo responsables, vencimientos y estados de seguimiento para mejorar la organización y trazabilidad de las actividades institucionales.
Precondición	• La presidenta debe haber iniciado sesión en el sistema. • Deben existir usuarios habilitados para la asignación de tareas
Flujo principal	1- La presidenta accede al módulo de gestión de tareas. 2- El sistema muestra la bandeja de tareas y las opciones disponibles. 3- La presidenta selecciona la opción "Crear tarea". 4- La presidenta ingresa título, descripción, responsable y fecha de vencimiento. 5- La presidenta confirma la creación de la tarea. 6- El sistema registra la tarea en estado "Pendiente". 7- El sistema asigna la tarea al usuario seleccionado. 8- El sistema registra fecha, hora y usuario responsable de la creación. 9- El sistema envía una notificación interna al responsable asignado.
Postcondición	• La tarea queda registrada en el sistema. • La tarea queda asociada al usuario responsable.

	- El sistema registra trazabilidad de la operación realizada. - El responsable asignado recibe una notificación interna.
Flujo alternativo	A1: Si el usuario administrador omite datos obligatorios al crear la tarea el sistema solicita completar los campos obligatorios.

Caso de uso 6: Marcar turno como Atendido (CU-TUR-04)

Actor principal	Profesional
Referencias	RF-TUR-04
Descripción	Permitir que el profesional registre la atención efectiva de un turno previamente asignado, habilitando automáticamente la carga de la historia clínica correspondiente y permitiendo considerar la prestación dentro de los procesos administrativos y de facturación.
Precondición	- El profesional debe haber iniciado sesión en el sistema. - El turno debe existir en la agenda del profesional. - El turno debe encontrarse en estado "Programado".
Flujo principal	- El profesional accede a la agenda diaria - El sistema muestra los turnos asignados al profesional. - El profesional selecciona un turno. - El profesional marca el turno como "Atendido". - El sistema actualiza el estado del turno. - El sistema registra fecha y hora de actualización. - El sistema habilita el registro de la historia clínica correspondiente. - El sistema marca la prestación como válida para procesos administrativos y de facturación.
Postcondición	- El turno queda registrado en estado "Atendido". - La historia clínica correspondiente queda habilitada para su carga. - La atención queda disponible para procesos administrativos y de facturación. - El sistema registra trazabilidad de la operación realizada.

Flujo alternativo	• A1: Si el profesional intenta marcar como atendido un turno cancelado el sistema informa que el turno no puede modificarse debido a su estado actual • A2: Si el profesional selecciona un turno previamente atendido el sistema informa que ya fue registrado como atendido.
-------------------	--

Etapa de análisis

Diagramas de secuencia

Los diagramas de secuencia permiten representar de forma dinámica la interacción entre los distintos actores, interfaces, controladores y componentes internos del sistema durante la ejecución de un caso de uso determinado.

En esta etapa del análisis se utilizaron diagramas de secuencia UML con el objetivo de modelar el comportamiento funcional del sistema propuesto para la Fundación OMNES, identificando el flujo de mensajes, validaciones, reglas de negocio y operaciones realizadas entre los distintos elementos involucrados.

Para el desarrollo de las siguientes etapas del proyecto (análisis detallado, diseño, implementación y pruebas), se seleccionaron los casos de uso considerados centrales para el funcionamiento del sistema, priorizando aquellos con mayor impacto sobre la lógica de negocio, persistencia de datos, trazabilidad e integración entre módulos.

Los casos de uso seleccionados fueron:

- Gestionar usuarios (CU-USU-02)
- Asignar turno (CU-TUR-02)
- Registrar historia clínica (CU-HC-01)
- Cargar factura ARCA (CU-FAC-01)

Asimismo, otros casos de uso definidos en la etapa de relevamiento y análisis funcional, como “Marcar turno como Atendido” y “Crear/Asignar tarea”, fueron mantenidos como parte del modelo analítico general del sistema, aunque no se desarrollarán en profundidad en las siguientes etapas, debido a criterios de alcance y priorización del prototipo.

A continuación, se presentan los diagramas de secuencia correspondientes a los casos de uso seleccionados para el desarrollo del sistema.

Caso de uso 1: Gestionar usuarios (CU-USU-02)

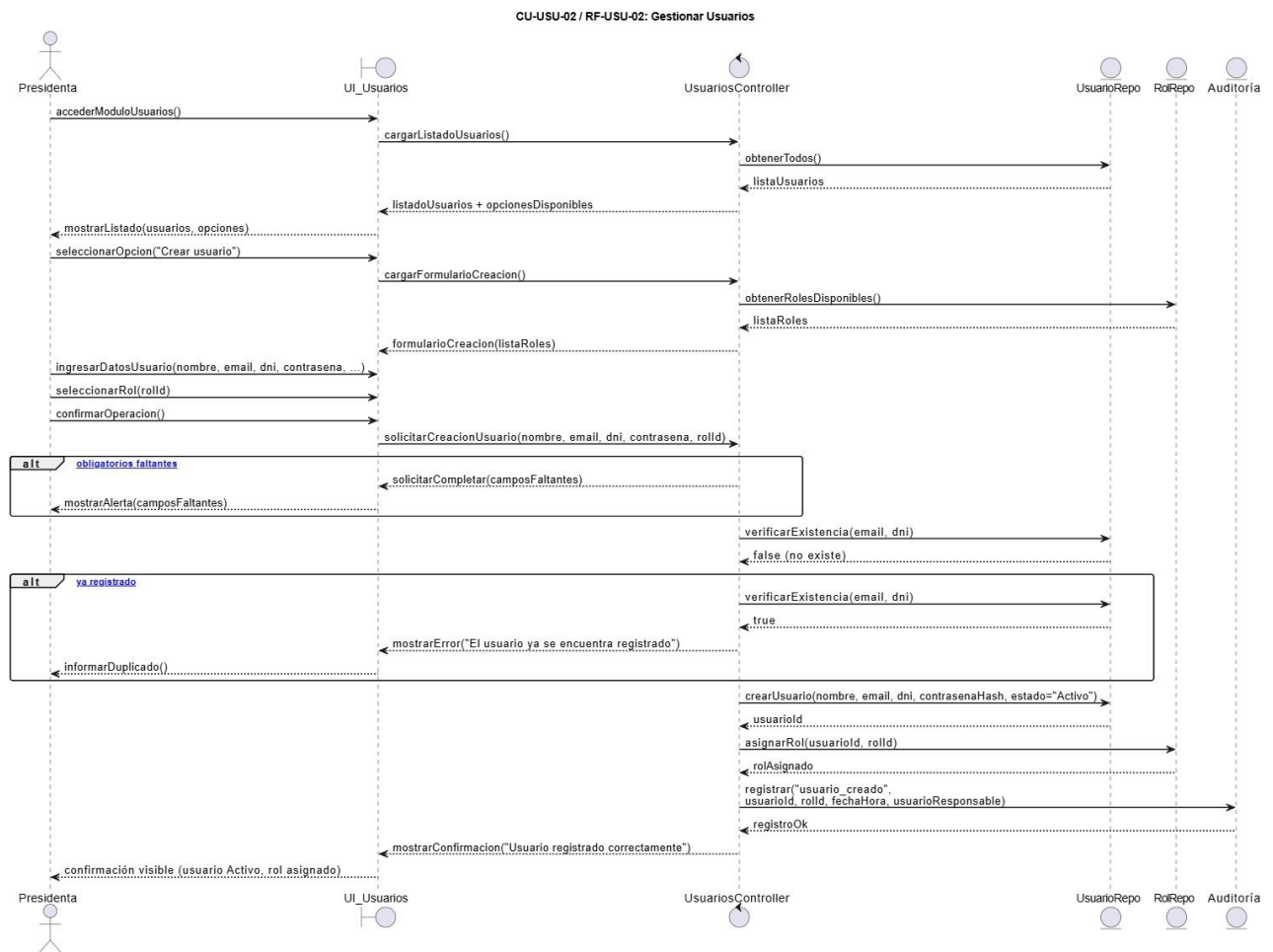


Figura 18 - Diagrama de secuencia del caso de uso CU-USU-02: Gestionar Usuarios.

El sistema carga el listado de usuarios existentes y los roles disponibles. La Presidenta completa el formulario con los datos del nuevo usuario y selecciona su rol. Antes de persistir, el sistema valida que no falten campos obligatorios y que el usuario no se encuentre previamente registrado. Si ambas validaciones son superadas, se crea el usuario en estado *Activo*, se le asigna el rol correspondiente y se registra la operación en auditoría.

Caso de uso 2: Asignar turno (CU-TUR-02)

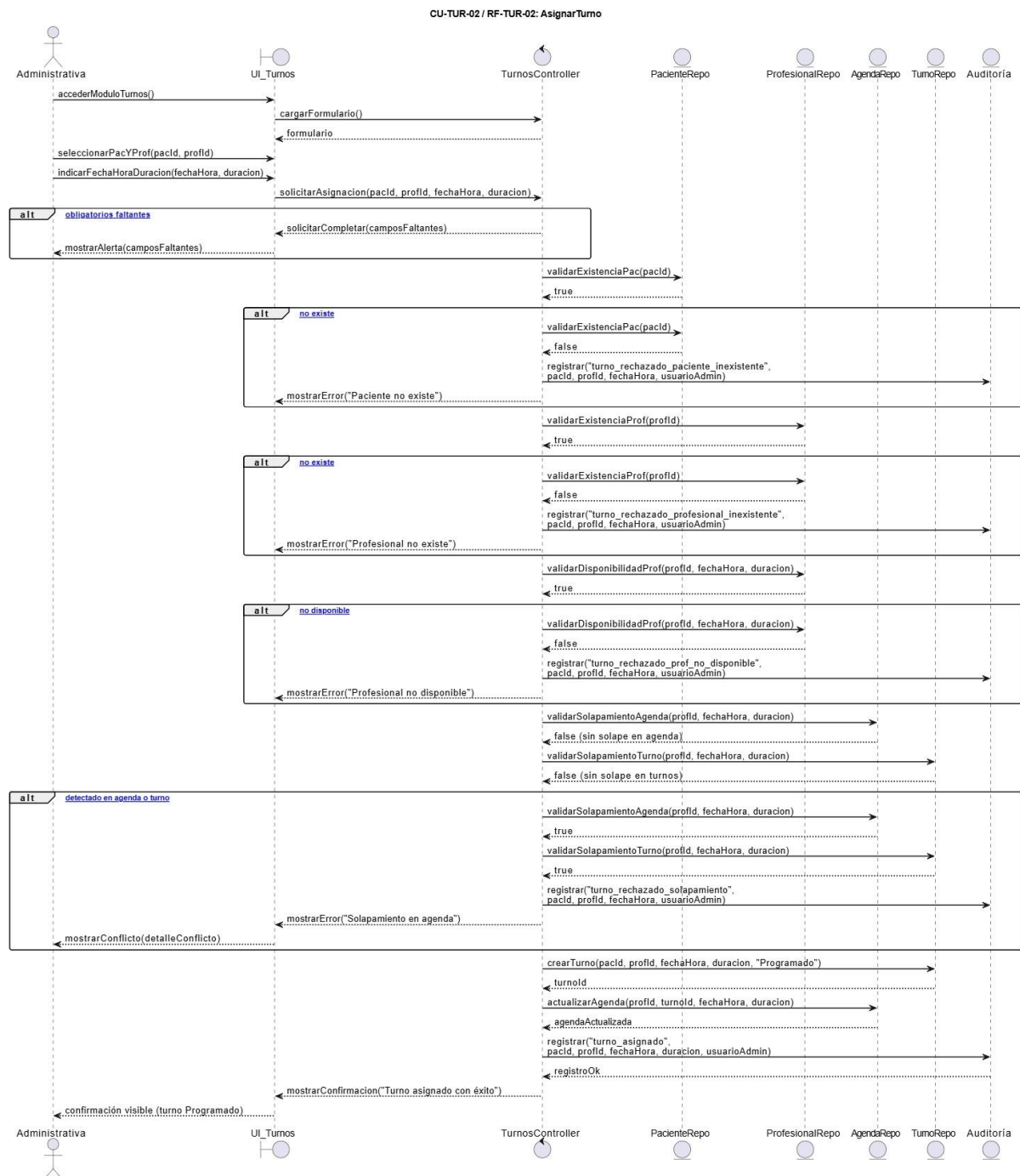


Figura 19 - Diagrama de secuencia del caso de uso CU-TUR-02: Asignar turno.

El flujo inicia con el acceso al módulo y la carga del formulario, donde se ingresan paciente, profesional, fecha, hora y duración. Antes de registrar el turno, el sistema ejecuta una serie de validaciones en cadena: existencia del paciente, existencia del profesional, disponibilidad horaria y ausencia de solapamientos. Cada validación fallida genera un rechazo con su correspondiente registro de auditoría.

Si todas las validaciones son exitosas, el turno se crea en estado programado, se actualiza la agenda del profesional y se confirma la operación a la administrativa.

Caso de uso 3: Registrar historia clínica (CU-HC-01)

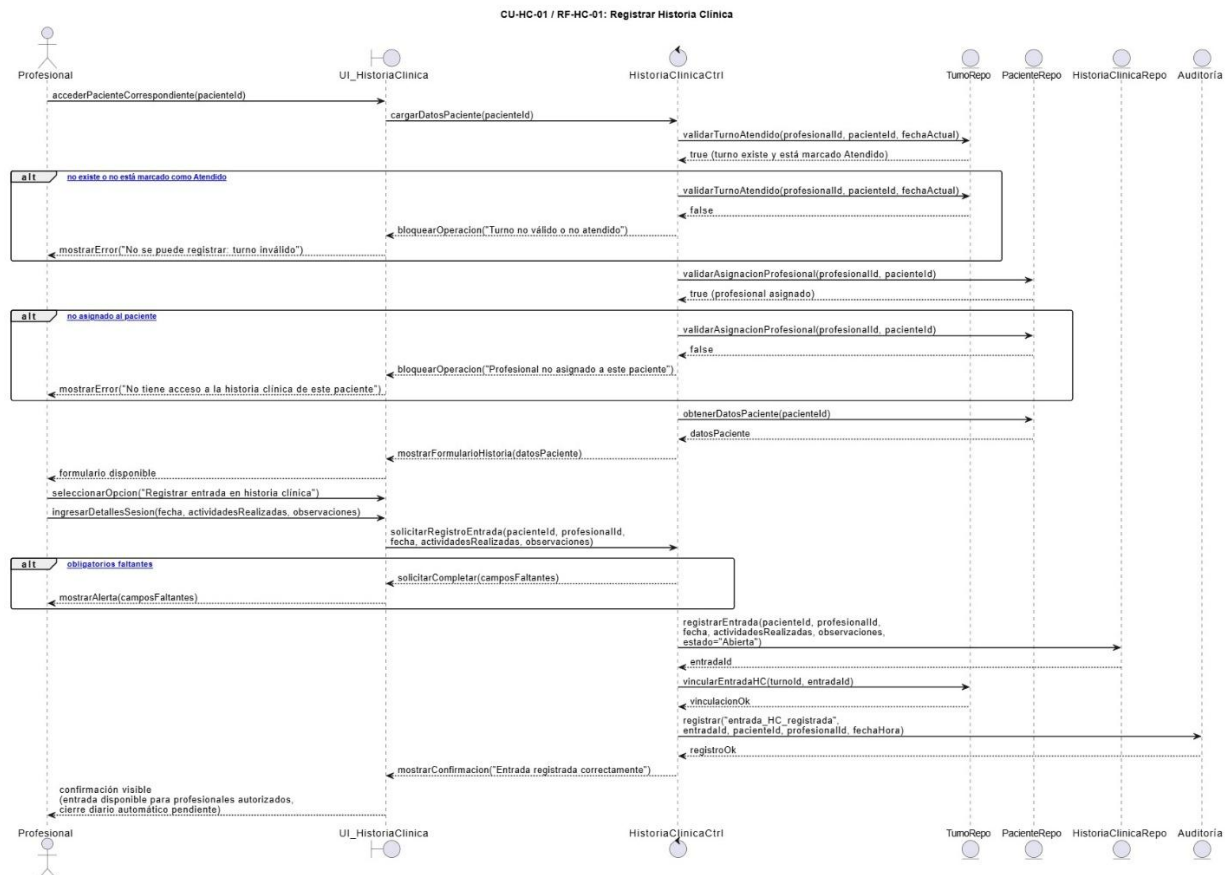


Figura 20 - Diagrama de secuencia del caso de uso CU-HC-01: Registrar historia clínica.

El sistema realiza dos validaciones previas al acceso al formulario: que el turno del día esté marcado como *Atendido* y que el profesional esté efectivamente asignado al paciente. Si alguna de estas condiciones no se cumple, la operación es bloqueada. Una vez habilitado el formulario, el profesional ingresa fecha, actividades realizadas y observaciones. El sistema persiste la entrada en estado *Abierta*, la vincula al turno correspondiente y registra la operación en auditoría, quedando la entrada disponible para los profesionales autorizados hasta el cierre diario automático.

Caso de uso 4: Cargar factura ARCA (CU-FAC-01)

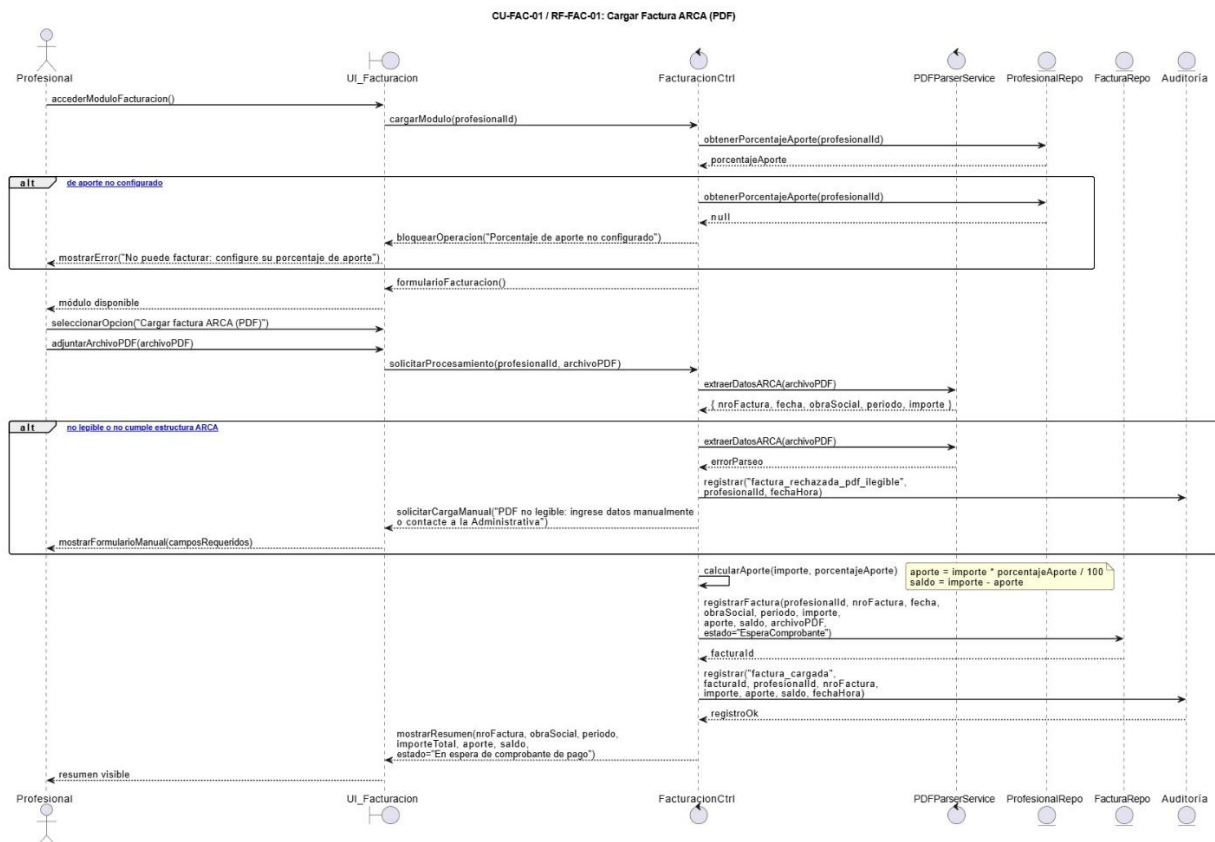


Figura 21 - Diagrama de secuencia del caso de uso CU-FAC-01: Cargar factura ARCA.

El sistema verifica que el profesional tenga un porcentaje de aporte configurado; de lo contrario, bloquea la operación. Una vez adjuntado el PDF, el servicio de parsing extrae automáticamente los datos estructurados: número de factura, fecha, obra social, período e importe. Si el PDF no es legible o no responde a la estructura ARCA, el sistema deriva la carga al ingreso manual. Con los datos disponibles, el controlador calcula el aporte a la Fundación y el saldo resultante, registra la factura en estado *En espera de comprobante de pago* y presenta al profesional un resumen con los valores calculados.

Etapas de diseño

La etapa de diseño tiene como objetivo definir la estructura interna del sistema propuesto para la Fundación OMNES, estableciendo la organización de sus componentes, responsabilidades y relaciones principales.

En esta instancia se modelan los elementos necesarios para representar cómo será implementada la solución informática a partir de los requerimientos obtenidos durante la etapa de análisis. Para ello,

se utilizan diagramas UML orientados a describir la arquitectura lógica del sistema, las clases involucradas, los mecanismos de control y la interacción con la persistencia de datos.

El diseño propuesto adopta una organización modular basada en separación de responsabilidades, distinguiendo componentes de interfaz, control, acceso a datos y entidades principales del negocio. Esta estructura busca facilitar la mantenibilidad, escalabilidad y trazabilidad del sistema durante las etapas posteriores de implementación y pruebas.

Asimismo, se priorizaron los casos de uso considerados centrales para el funcionamiento operativo de la fundación, modelando aquellos elementos con mayor impacto sobre la gestión administrativa, clínica y contable del sistema.

A continuación, se presentan los diagramas de clases correspondientes a los casos de uso seleccionados para el desarrollo del proyecto.

Caso de uso 1: Gestionar usuarios (CU-USU-02)

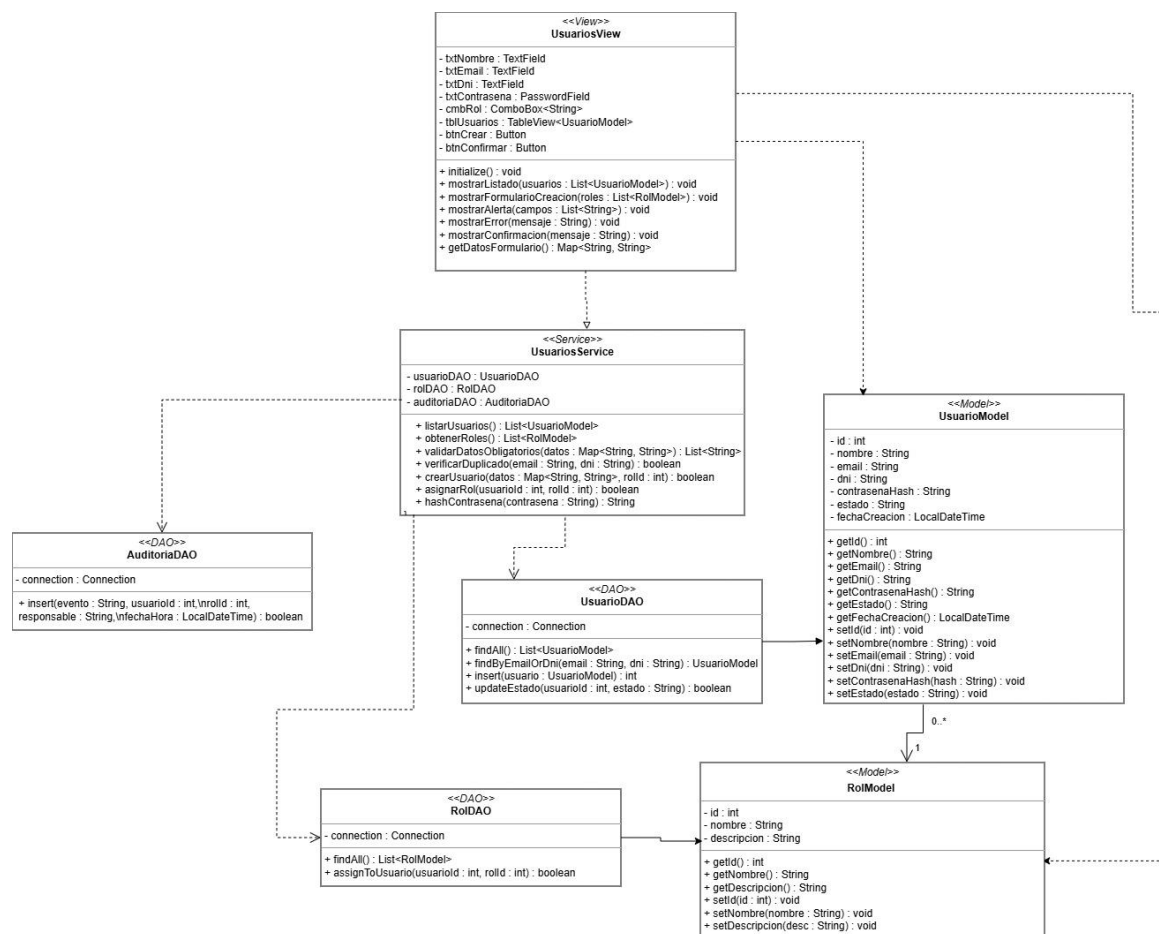


Figura 22 - Diagrama de clases del caso de uso CU-USU-02: Gestionar usuarios.

El diagrama de clases de diseño correspondiente al caso de uso “Gestionar usuarios” representa la estructura interna del módulo de administración de usuarios del sistema. La arquitectura propuesta se organiza en capas diferenciadas de vista, servicio, acceso a datos y modelo, siguiendo un enfoque

modular orientado a separación de responsabilidades. La clase UsuariosView concentra la interacción gráfica mediante JavaFX, mientras que UsuariosService implementa las validaciones y reglas de negocio relacionadas con la creación de usuarios, asignación de roles y seguridad de credenciales mediante hash de contraseñas. Las clases DAO encapsulan el acceso a MySQL mediante JDBC y las entidades UsuarioModel y RolModel representan la persistencia de los datos del dominio. El diseño garantiza trazabilidad de operaciones mediante el componente de auditoría y facilita la mantenibilidad y escalabilidad del sistema.

Caso de uso 2: Asignar turno (CU-TUR-02)

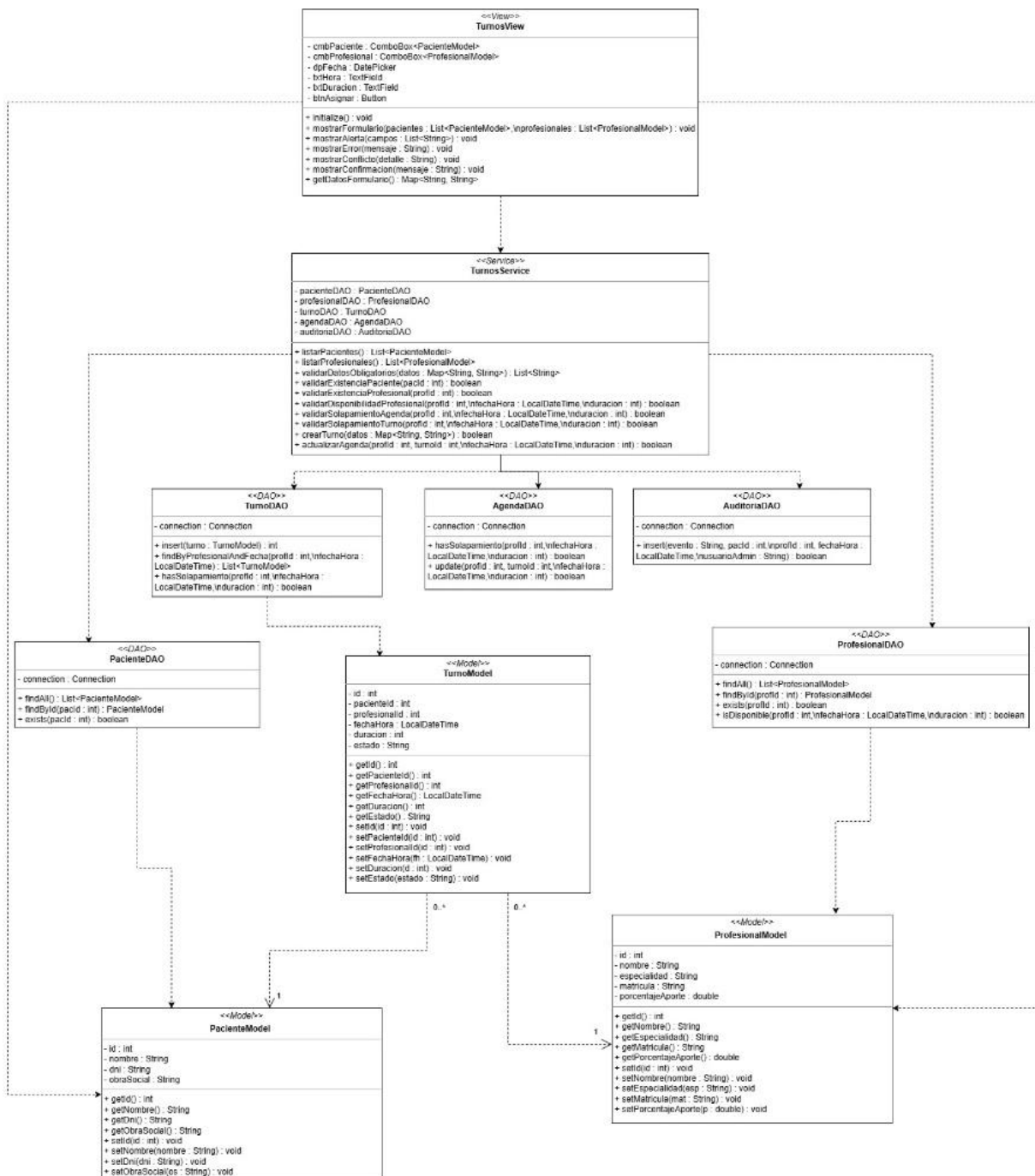


Figura 23 - Diagrama de clases del caso de uso CU-TUR-02: Asignar turno.

El diagrama de clases de diseño correspondiente al caso de uso “Asignar turno” representa la estructura interna del módulo de gestión de turnos del sistema. La solución se organiza en capas de vista, servicio, acceso a datos y modelo, manteniendo separación de responsabilidades entre la interfaz gráfica, la lógica de negocio y la persistencia de datos. La clase TurnosView implementa la interacción con la administrativa mediante JavaFX, mientras que TurnosService centraliza las validaciones relacionadas con existencia de pacientes y profesionales, disponibilidad horaria y control de solapamientos. Los DAO encapsulan el acceso a la base de datos MySQL mediante JDBC y los modelos TurnoModel, PacienteModel y ProfesionalModel representan las entidades persistidas del dominio. El diseño incorpora además un mecanismo de auditoría para registrar tanto asignaciones exitosas como rechazos operativos, garantizando trazabilidad sobre las operaciones realizadas.

Caso de uso 3: Registrar historia clínica (CU-HC-01)

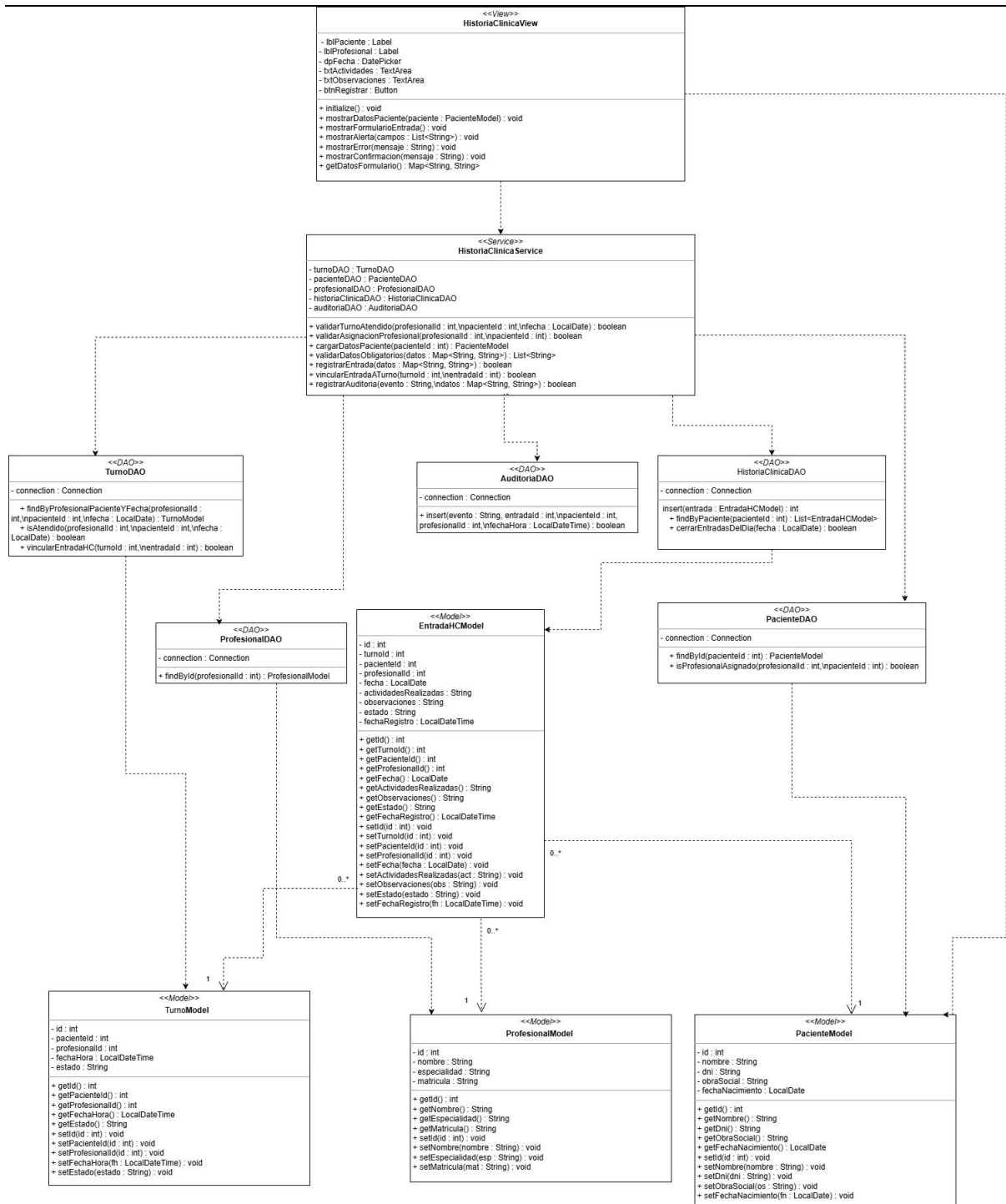


Figura 24 - Diagrama de clases del caso de uso CU-HC-01: Registrar Historia Clínica.

El diagrama de clases de diseño correspondiente al caso de uso “Registrar historia clínica” representa la estructura interna del módulo encargado de gestionar las entradas clínicas de los pacientes. La arquitectura se organiza en capas de vista, servicio, acceso a datos y modelo. La clase HistoriaClinicaView implementa la interfaz gráfica utilizada por el profesional para registrar la atención realizada, mientras que HistoriaClinicaService centraliza las validaciones funcionales y coordina la

lógica de negocio del caso de uso. Las clases DAO encapsulan el acceso a la base de datos MySQL mediante JDBC y las entidades EntradaHCMModel, TurnoModel, PacienteModel y ProfesionalModel representan los datos persistidos del dominio. El diseño contempla validaciones relacionadas con el estado del turno, la asignación profesional-paciente y el registro de auditoría de las operaciones realizadas, garantizando integridad y trazabilidad sobre la información clínica almacenada.

Caso de uso 4: Cargar factura ARCA (CU-FAC-01)

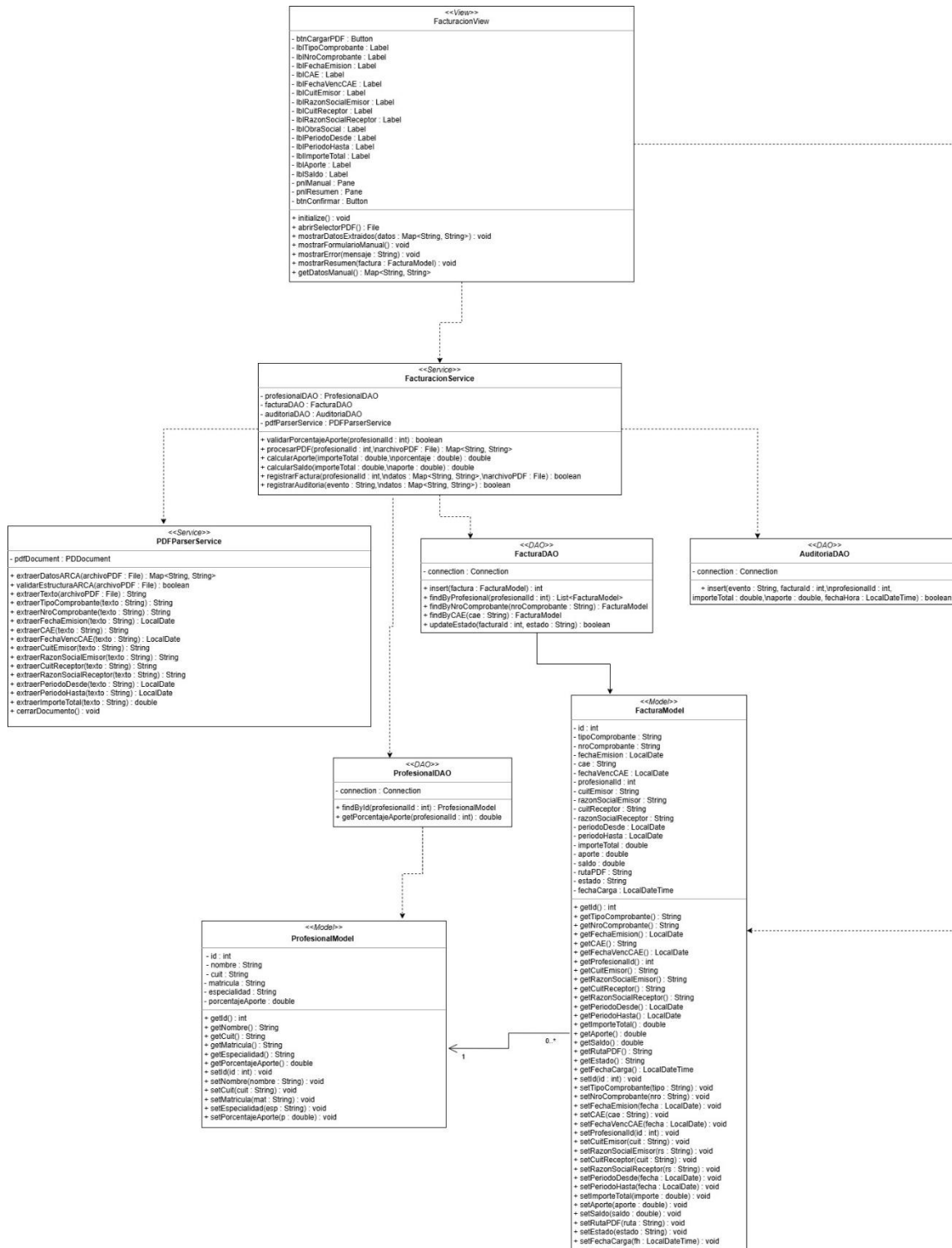


Figura 25 - Diagrama de clases del caso de uso CU-FAC-01: Cargar factura ARCA.

El diagrama de clases de diseño correspondiente al caso de uso “Cargar factura ARCA” representa la estructura interna del módulo encargado de procesar y registrar comprobantes electrónicos emitidos por los profesionales de la Fundación OMNES. El diseño se basa en el modelo real de Factura C utilizado por monotributistas, incorporando campos propios del sistema ARCA/AFIP como número de comprobante, CAE, fecha de vencimiento, CUIT, razón social, período facturado e importe total. La arquitectura se organiza en capas de vista, servicio, acceso a datos y modelo. FacturacionView administra la interacción con el usuario, FacturacionService centraliza la lógica de negocio y los cálculos de aporte institucional, PDFParserService encapsula la extracción automática de datos desde el PDF utilizando Apache PDFBox, y las clases DAO gestionan la persistencia sobre MySQL 8 mediante JDBC.

La entidad FacturaModel representa la estructura completa del comprobante electrónico, mientras que FacturaDAO implementa las validaciones necesarias para evitar registros duplicados mediante controles sobre número de factura y CAE. El sistema almacena la ruta física del archivo PDF en el servidor, manteniendo el documento original disponible para auditoría y consulta posterior. El diseño también contempla el registro de auditoría de cada operación realizada, asegurando trazabilidad sobre las cargas efectuadas por los profesionales.

Etapas de implementación

Diagrama de despliegue

El diagrama de despliegue representa la arquitectura física propuesta para la implementación del sistema integral de gestión de la Fundación OMNES. El sistema fue diseñado bajo una arquitectura cliente-servidor local centralizada, utilizando aplicaciones de escritorio desarrolladas en JavaFX conectadas a una base de datos MySQL 8 mediante JDBC sobre red interna LAN/Wi-Fi.

La infraestructura contempla distintos clientes de escritorio para los perfiles de Profesional, Contadora, Presidenta y Administrativa, cada uno ejecutando la aplicación JavaFX con sus correspondientes capas de vista, servicios y acceso a datos. Todas las estaciones se conectan al servidor central utilizando el puerto TCP 3306 para las operaciones de persistencia sobre MySQL.

El servidor de la Fundación OMNES concentra la base de datos principal del sistema, el almacenamiento de auditorías y una carpeta compartida destinada a adjuntos y comprobantes PDF, particularmente utilizada por el módulo de facturación ARCA. Asimismo, se incorpora un servicio de backup automático basado en mysqldump, encargado de generar copias diarias de la base de datos y de los archivos adjuntos almacenados.

Las copias de seguridad son transferidas automáticamente hacia un NAS o disco externo de almacenamiento, garantizando redundancia de la información y recuperación ante fallos. Esta arquitectura permite centralizar la información clínica, administrativa y contable de la organización, facilitando la trazabilidad, la integridad de los datos y el acceso concurrente de múltiples usuarios dentro de la red interna de la institución.

El diseño propuesto prioriza simplicidad de implementación, bajo costo operativo y facilidad de mantenimiento, características adecuadas para la escala y necesidades actuales de la Fundación OMNES.

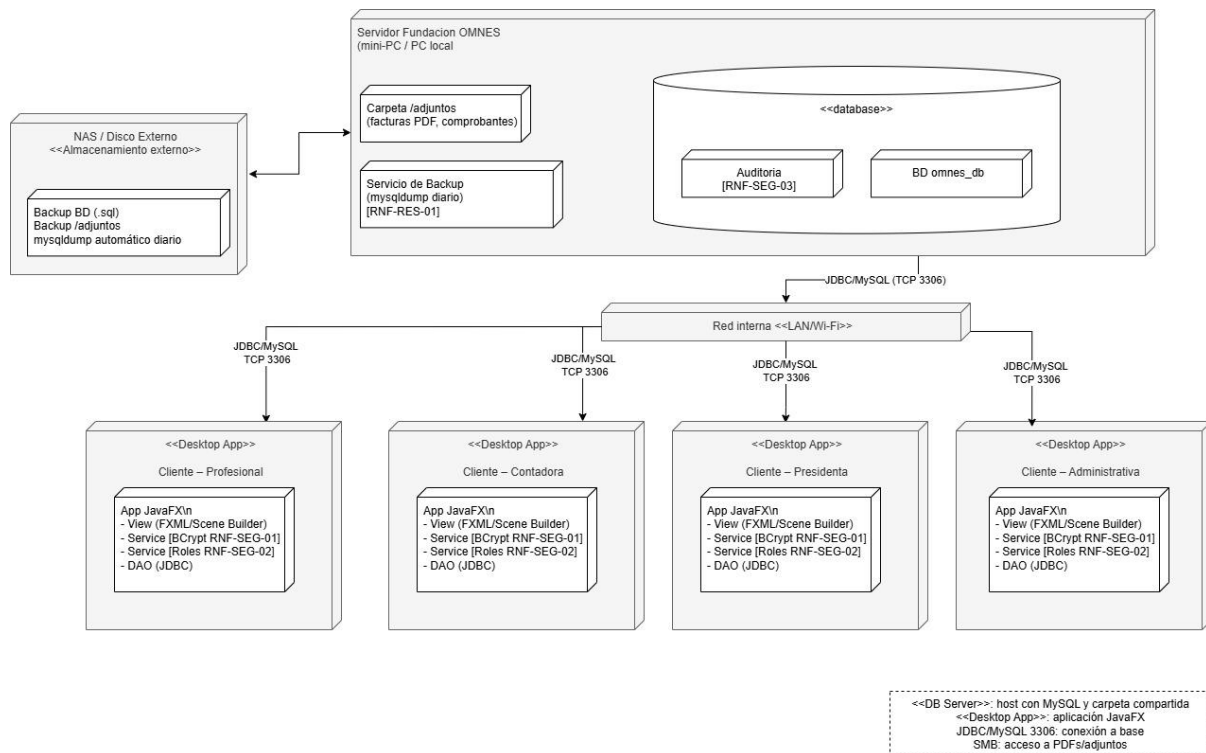


Figura 26 - Diagrama de despliegue arquitectura física.

Etapa de pruebas

La etapa de pruebas tiene como objetivo verificar que las funcionalidades implementadas en el sistema cumplan correctamente con los requerimientos funcionales y no funcionales definidos durante las etapas de análisis y diseño.

En esta instancia se planifican las validaciones necesarias para comprobar el correcto comportamiento del sistema ante distintos escenarios de uso, incluyendo situaciones válidas, inválidas y casos límite.

Para el sistema de gestión integral de la Fundación OMNES se definieron pruebas funcionales orientadas principalmente a los casos de uso considerados críticos para la operación del sistema, priorizando aquellos que involucran validaciones de negocio, control de acceso, persistencia de datos y trazabilidad de operaciones.

Las pruebas propuestas contemplan técnicas de caja negra, partición de equivalencias y análisis de valores de frontera, aplicadas sobre los módulos principales desarrollados durante el presente trabajo.

Plan de pruebas

El plan de pruebas define las funcionalidades que serán evaluadas, el tipo de prueba asociado y la técnica utilizada para validar el comportamiento esperado del sistema.

Caso de uso	Código prueba	Tipo de prueba	Técnica propuesta	Observaciones
CU-TUR-02 – Asignar turno	CPT01	Funcional	Caja negra	Verificación de asignación correcta de turnos
CU-TUR-02 – Asignar turno	CPT02	Funcional	Partición de equivalencias	Validación de profesional inexistente
CU-TUR-02 – Asignar turno	CPT03	Funcional	Valores de frontera	Validación de solapamientos horarios
CU-HC-01 – Registrar historia clínica	CPH01	Funcional	Caja negra	Verificación de acceso permitido únicamente a profesionales asignados
CU-FAC-01 – Cargar factura ARCA	CPF01	Funcional	Caja negra	Validación de carga correcta de factura PDF
CU-FAC-01 – Cargar factura ARCA	CPF02	Funcional	Partición de equivalencias	Validación de PDF ilegible o inválido
CU-USU-02 – Gestionar usuarios	CPU01	Funcional	Caja negra	Verificación de alta correcta de usuario
CU-USU-02 – Gestionar usuarios	CPU02	Funcional	Partición de equivalencias	Validación de usuario duplicado

Caso de prueba – Asignar turno (CU-TUR-02)

El presente caso de prueba tiene como objetivo validar el correcto funcionamiento del módulo de asignación de turnos, verificando que el sistema controle adecuadamente la disponibilidad del profesional, la existencia de pacientes y los posibles conflictos de agenda.

Requerimientos asociados

Requerimiento	Descripción
---------------	-------------

RF-TUR-02	El sistema debe permitir asignar turnos a pacientes
RF-TUR-03	El sistema debe validar disponibilidad horaria
RF-TUR-04	El sistema debe impedir solapamientos
RF-SEG-03	El sistema debe registrar auditoría de operaciones

Clases de equivalencia válidas

Código	Condición
CEV1	Paciente existente
CEV2	Profesional existente
CEV3	Horario disponible
CEV4	Turno sin solapamiento

Clases de equivalencia inválidas

Código	Condición
CEI1	Paciente inexistente
CEI2	Profesional inexistente
CEI3	Profesional no disponible
CEI4	Solapamiento detectado

Casos de comportamiento esperado

Entrada	Resultado esperado	Mensaje del sistema
Paciente válido + horario libre	Turno asignado	"Turno asignado con éxito"
Paciente inexistente	Operación rechazada	"Paciente no existe"
Profesional inexistente	Operación rechazada	"Profesional no existe"
Profesional ocupado	Operación rechazada	"Profesional no disponible"
Horario solapado	Operación rechazada	"Solapamiento en agenda/consultorio"

Caso de prueba – Cargar factura ARCA (CU-FAC-01)

El objetivo de este caso de prueba es verificar que el sistema pueda procesar correctamente comprobantes PDF emitidos por ARCA/AFIP, extrayendo automáticamente la información relevante y calculando el aporte correspondiente a la Fundación OMNES.

Requerimiento	Descripción
RF-FAC-01	El sistema debe permitir cargar facturas PDF
RF-FAC-02	El sistema debe extraer datos automáticamente
RF-FAC-03	El sistema debe calcular el aporte institucional
RF-SEG-03	El sistema debe registrar auditoría

Clases de equivalencia válidas

Código	Condición
CEV1	PDF legible
CEV2	Factura con estructura válida ARCA
CEV3	CAE válido
CEV4	Profesional con porcentaje configurado

Clases de equivalencia inválidas

Código	Condición
CEI1	PDF ilegible
CEI2	PDF corrupto
CEI3	Factura duplicada
CEI4	Profesional sin porcentaje de aporte

Casos de comportamiento esperado

Entrada	Resultado esperado	Mensaje del sistema
PDF válido	Factura registrada	"Factura cargada correctamente"
PDF ilegible	Operación rechazada	"PDF no legible"
CAE duplicado	Operación rechazada	"Factura ya registrada"

Profesional sin configuración	Operación bloqueada	“Configure porcentaje de aporte”
-------------------------------	---------------------	----------------------------------

Las pruebas definidas permiten validar los principales flujos funcionales del sistema propuesto, verificando tanto escenarios exitosos como situaciones de error y validaciones de negocio críticas.

La estrategia planteada prioriza pruebas funcionales sobre los módulos más sensibles del sistema, especialmente aquellos vinculados con la asignación de turnos, registro clínico y facturación institucional, debido a su impacto directo sobre la operación diaria de la fundación.

Asimismo, el diseño de pruebas mantiene trazabilidad con los requerimientos funcionales definidos durante la etapa de análisis, permitiendo asegurar coherencia entre especificación, diseño e implementación futura del sistema.

Interfaz gráfica

Los prototipos de interfaz gráfica permiten representar visualmente la interacción de los distintos usuarios con el sistema propuesto para la Fundación OMNES. En esta etapa se presentan las principales pantallas correspondientes a los casos de uso priorizados durante el análisis y diseño del sistema.

Los prototipos fueron diseñados siguiendo criterios de usabilidad, simplicidad operativa y separación por roles de usuario, contemplando las necesidades específicas de administrativas, profesionales y presidencia de la fundación.

Las interfaces fueron modeladas como aplicaciones de escritorio utilizando JavaFX y Scene Builder, manteniendo coherencia con la arquitectura definida durante la etapa de diseño.

A continuación, se presentan los principales prototipos desarrollados para el sistema.

Prototipo – Asignación de turnos

El siguiente prototipo representa la interfaz propuesta para la asignación de turnos dentro del sistema de gestión integral de la Fundación OMNES. La pantalla permite a la administrativa seleccionar paciente, profesional, fecha, horario y duración del turno, además de visualizar los turnos registrados en el sistema.

La interfaz fue diseñada utilizando JavaFX y Scene Builder, manteniendo coherencia con la arquitectura definida durante la etapa de diseño.

Sistema Integral Fundación OMNES

Gestión de Turnos

Dashboard

Usuarios

Turnos

Historias Clínicas

Facturación

Reportes

Asignación de Turnos

Paciente:

Seleccionar paciente

Profesional:

Seleccionar profesional

Fecha:

Hora:

Hora

Duración:

Minutos

Observaciones:

Observaciones administrativas

Asignar Turno

Cancelar

Turnos registrados

Paciente	Profesional	Fecha	Hora	Estado
Tabla sin contenido				

Figura 27 – Prototipo 1 Asignación de turnos.

Prototipo – Registro de Historia Clínica

El diseño refleja la distribución funcional definida durante las etapas de análisis y diseño, permitiendo al profesional visualizar los datos del paciente y registrar la información clínica asociada a una sesión previamente atendida.

La interfaz incluye una sección informativa con los datos principales del paciente y del profesional asignado, además de un formulario central para la carga de actividades realizadas y observaciones clínicas. También se incorpora una tabla destinada a visualizar entradas anteriores de la historia clínica, facilitando la trazabilidad y consulta de registros previos. El menú lateral mantiene la integración con los distintos módulos del sistema, respetando la estructura general de navegación propuesta para la aplicación de escritorio.

El objetivo del prototipo es representar de manera visual la futura experiencia de uso del sistema, validando la organización de componentes, distribución de información y flujo operativo antes de la etapa de implementación definitiva.

Figura 28 – Prototipo 2 Registrar historia clínica.

Diseño e implementación de la base de datos

La persistencia de la información del Sistema Integral Fundación OMNES se diseñó utilizando una base de datos relacional implementada en MySQL 8. El modelo de datos fue construido a partir de los requerimientos funcionales identificados durante las etapas de análisis y diseño, garantizando la trazabilidad entre los casos de uso, los diagramas de clases de diseño y la estructura física de almacenamiento.

Para facilitar la comprensión del modelo y mantener la consistencia con la arquitectura modular propuesta, el diseño de la base de datos se presenta inicialmente mediante diagramas entidad-relación (DER) organizados por módulo funcional. Posteriormente, estos módulos serán integrados en un modelo de datos unificado que representará la estructura completa del sistema.

Cada entidad incluye sus atributos principales, claves primarias (PK), claves foráneas (FK) y relaciones de cardinalidad, permitiendo representar adecuadamente la gestión de usuarios, asignación de turnos, historias clínicas, facturación y auditoría de operaciones realizadas dentro de la plataforma.

DER – Módulo Gestión de Usuarios

El siguiente diagrama entidad-relación representa la estructura de datos necesaria para la gestión de usuarios del sistema. El modelo contempla las entidades Rol, Usuario y Auditoría, permitiendo administrar los perfiles de acceso y registrar la trazabilidad de las operaciones realizadas por los distintos actores de la aplicación.

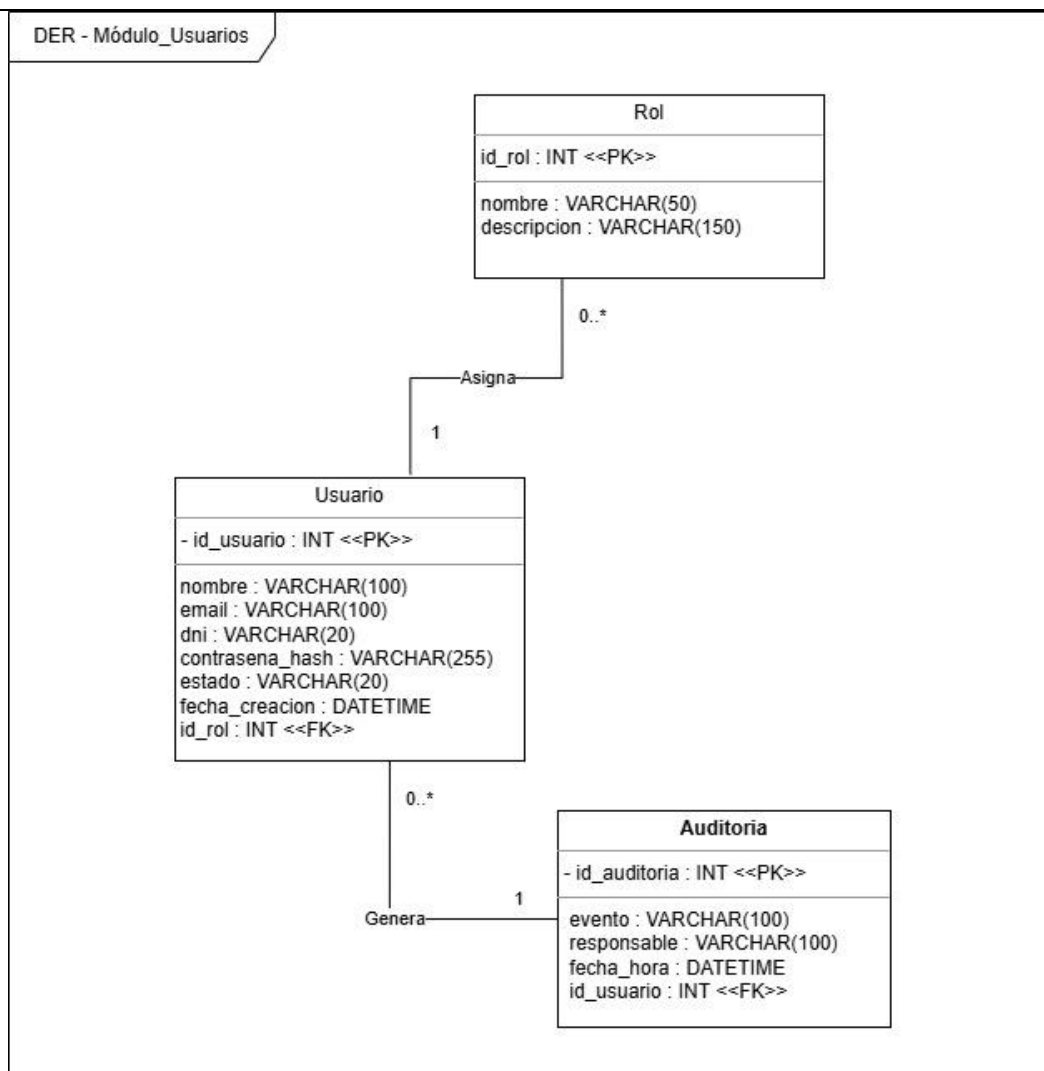


Figura 29 – Diagrama entidad-relación (módulo usuarios)

DER – Módulo Asignación de Turnos

El siguiente diagrama entidad-relación representa la estructura de datos correspondiente al módulo de asignación de turnos. El modelo incorpora las entidades Paciente, Profesional y Turno, permitiendo gestionar la programación de atenciones y la relación entre pacientes y profesionales dentro de la Fundación OMNES.

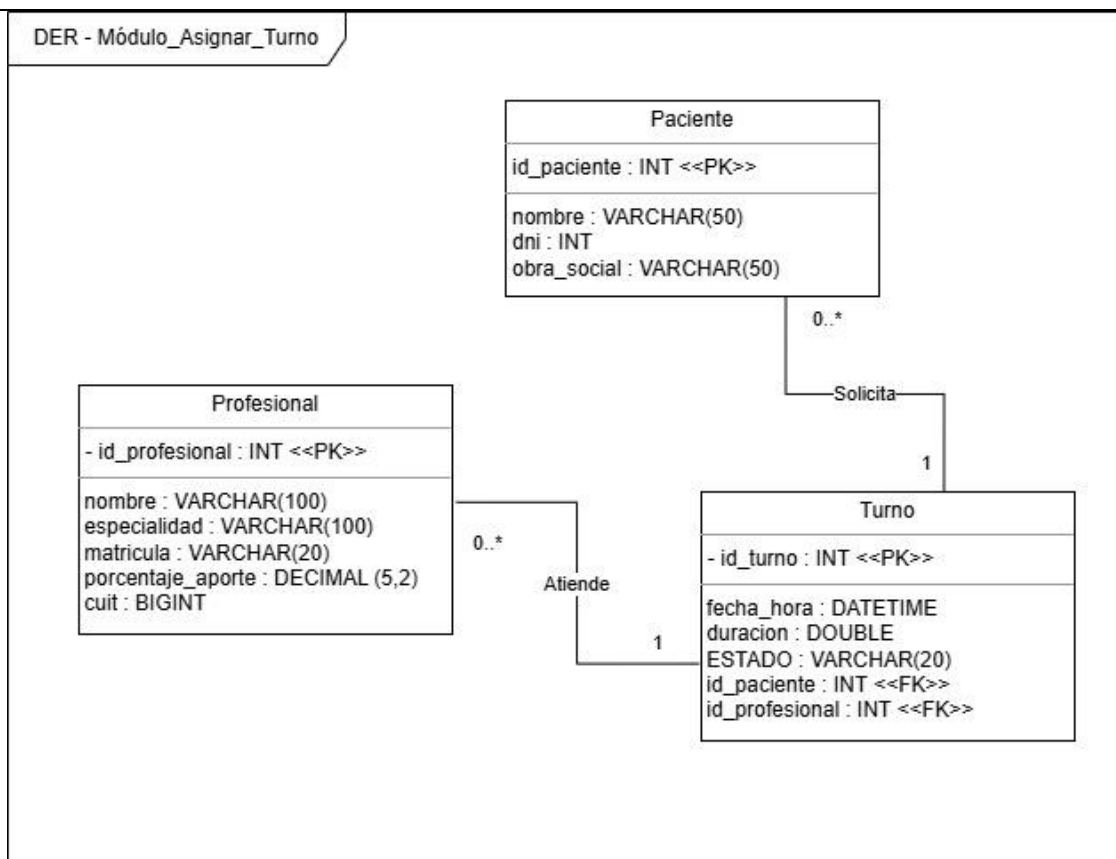


Figura 30 – Diagrama entidad-relación (módulo Asignar Turnos)

DER – Módulo Registrar historia clínica

El siguiente diagrama entidad-relación representa la estructura de datos correspondiente al registro de historias clínicas. La entidad EntradaHC almacena las actividades realizadas y observaciones registradas durante cada sesión, manteniendo la vinculación con el paciente, el profesional actuante y el turno que dio origen a la atención.

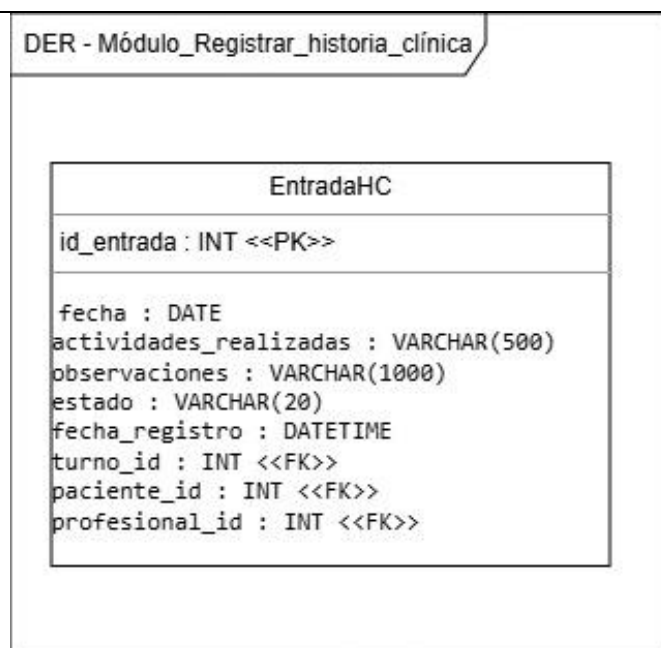


Figura 31 – Diagrama entidad-relación (módulo registrar Historia Clínica)

DER – Módulo Facturación

El siguiente diagrama entidad-relación representa la estructura de datos correspondiente al proceso de carga y gestión de facturas emitidas por los profesionales de la Fundación OMNES. El modelo permite almacenar la información proveniente de los comprobantes electrónicos de ARCA, así como la vinculación con el profesional responsable de la prestación.

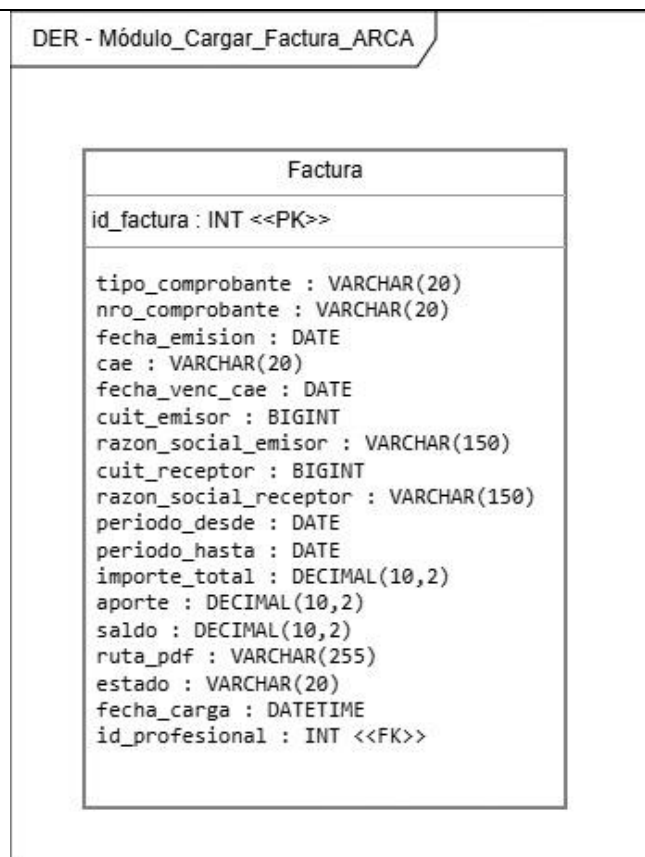


Figura 32 – Diagrama entidad-relación (módulo Cargar Factura ARCA)

DER – Integrado del Sistema

Una vez definidos los módulos individuales, se presenta el modelo entidad-relación integrado del Sistema Integral Fundación OMNES. Este diagrama consolida todas las entidades y relaciones necesarias para soportar el funcionamiento completo de la solución propuesta.

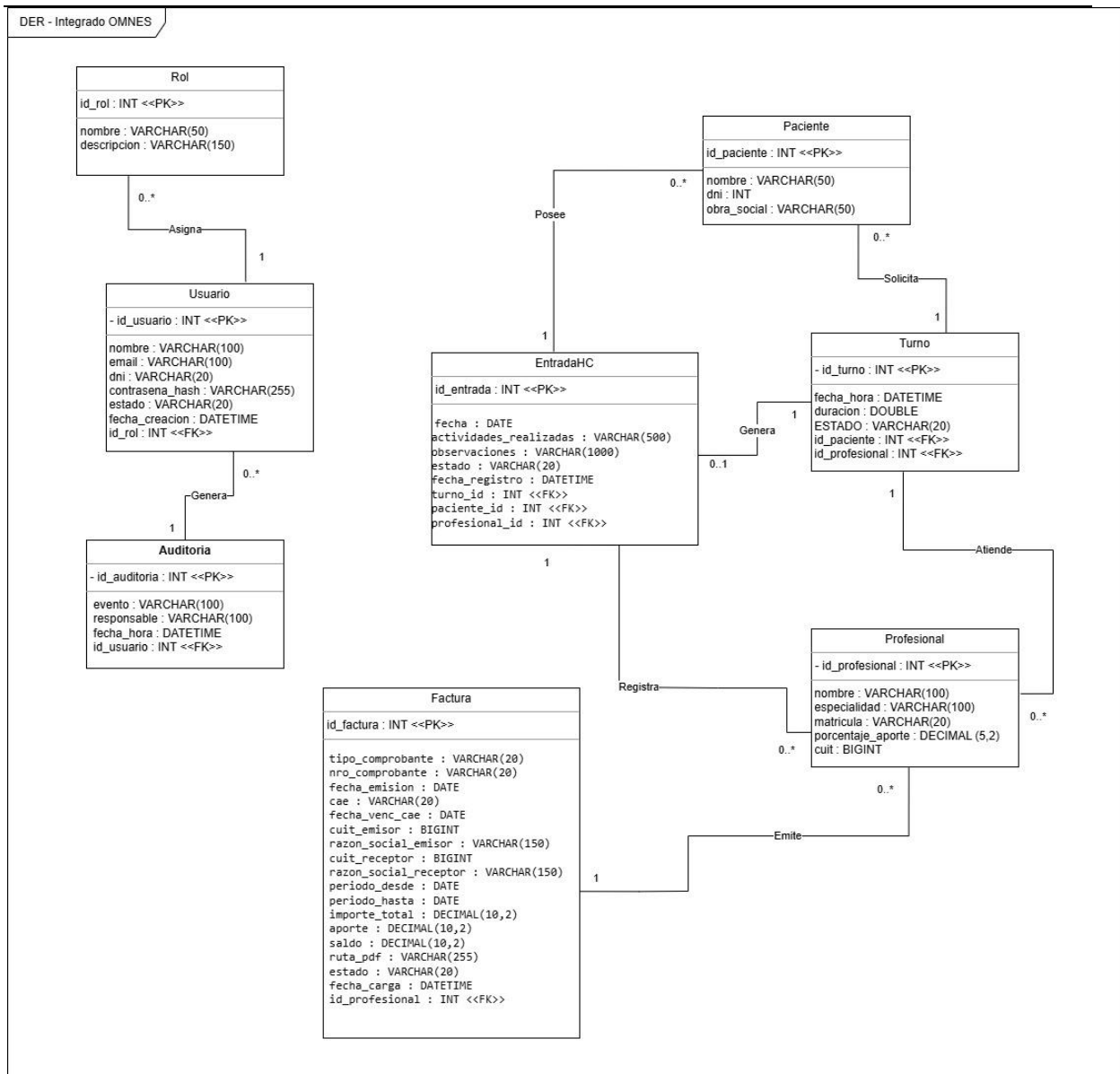


Figura 33 – Diagrama entidad-relación integrado

Normalización

El diseño de la base de datos fue realizado siguiendo los principios de normalización hasta Tercera Forma Normal (3FN), con el objetivo de minimizar redundancias y evitar anomalías de inserción, actualización y eliminación de datos.

Cada entidad representa un único concepto del dominio de negocio y las relaciones entre ellas se implementan mediante claves foráneas. Los atributos almacenan únicamente información dependiente de la clave primaria de cada tabla, eliminando dependencias parciales y transitivas.

La aplicación de estas reglas permite mantener la consistencia de la información y facilita futuras tareas de mantenimiento y evolución del sistema.

Creación de tablas MySQL

La implementación física de la base de datos se realizó utilizando MySQL 8.0. A partir del modelo relacional definido en la etapa de diseño, se procedió a la creación de las tablas necesarias para soportar las funcionalidades del Sistema Integral Fundación OMNES.

Las tablas fueron creadas respetando las claves primarias y claves foráneas definidas en el modelo de datos, garantizando la integridad referencial y la consistencia de la información almacenada.

The screenshot displays a MySQL IDE window titled 'SQL File 4*'. The editor shows SQL code for creating a table named 'Factura'. The code includes a 'REFERENCES' statement for 'Profesional(id_profesional)' and a 'CREATE TABLE' statement for 'Factura' with the following columns: 'id_factura' (INT AUTO_INCREMENT PRIMARY KEY), 'tipo_comprobante' (VARCHAR(20)), 'nro_comprobante' (VARCHAR(20)), 'fecha_emision' (DATE), 'cae' (VARCHAR(20)), and 'fecha_venc_cae' (DATE). Below the editor, the 'Output' window shows a table of execution results.

#	Time	Action	Message	Duration / Fetch
1	17:36:08	CREATE DATABASE omnes_db	1 row(s) affected	0.000 sec
2	17:36:08	USE omnes_db	0 row(s) affected	0.000 sec
3	17:46:42	CREATE TABLE Rol (id_rol INT AUTO_INCREM...	0 row(s) affected	0.031 sec
4	17:47:40	CREATE TABLE Profesional (id_profesional INT ...	0 row(s) affected	0.031 sec
5	17:48:11	CREATE TABLE Paciente (id_paciente INT AUT...	0 row(s) affected	0.047 sec
6	17:49:04	CREATE TABLE Usuario (id_usuario INT AUTO_...	0 row(s) affected	0.031 sec
7	17:49:58	CREATE TABLE Auditoria (id_auditoria INT AUT...	0 row(s) affected	0.032 sec
8	17:58:17	CREATE TABLE Turno (id_turno INT AUTO_INC...	0 row(s) affected	0.032 sec
9	17:59:08	CREATE TABLE EntradaHC (id_entrada INT AU...	0 row(s) affected	0.047 sec
10	18:08:09	CREATE TABLE Factura (id_factura INT AUTO_I...	0 row(s) affected	0.047 sec

Figura 34 – Ejecución de los scripts SQL para la creación de la base de datos del sistema.

Se muestra la ejecución satisfactoria de los scripts SQL utilizados para crear la base de datos omnes_db y las tablas correspondientes a los módulos de Usuarios, Turnos, Historia Clínica, Facturación y Auditoría.

Inserción de registros

Con el objetivo de verificar el correcto funcionamiento de la estructura creada, se incorporaron registros de prueba representativos de la operatoria real de la Fundación OMNES. Los datos cargados permiten validar las relaciones entre usuarios, pacientes, profesionales, turnos, historias clínicas y facturación.

La carga de información se realizó mediante sentencias INSERT INTO sobre cada una de las tablas definidas previamente.

Roles

The screenshot displays a database management interface. At the top, a SQL script is shown with lines 157 to 165. Line 157 contains an INSERT INTO statement for the 'Factura' table. Lines 159 to 165 are SELECT statements for various tables: Rol, Usuario, Profesional, Paciente, Turno, EntradaHC, and Factura.

Below the script, a 'Result Grid' shows the results of the queries. It has columns for 'id_rol', 'nombre', and 'descripcion'. The data is as follows:

id_rol	nombre	descripcion
1	Administradora	Acceso completo al sistema
2	Profesional	Acceso a pacientes y turnos
3	Contadora	Acceso a facturación
NULL	NULL	NULL

At the bottom, the 'Output' section shows the 'Action Output' table, which lists the execution of the SQL scripts. The table has columns for '#', 'Time', 'Action', 'Message', and 'Duration / Fetch'. The actions include creating tables, inserting data, and selecting data with limits.

#	Time	Action	Message	Duration / Fetch
9	17:59:08	CREATE TABLE EntradaHC (id_entrada INT ...	0 row(s) affected	0.047 sec
10	18:08:09	CREATE TABLE Factura (id_factura INT AUT...	0 row(s) affected	0.047 sec
11	18:33:02	INSERT INTO Rol (nombre, descripcion) VALUE...	3 row(s) affected Records: 3 Duplicates: 0 Wami...	0.015 sec
12	18:33:30	INSERT INTO Usuario (nombre,email,dni,contras...	2 row(s) affected Records: 2 Duplicates: 0 Wami...	0.015 sec
13	18:33:43	INSERT INTO Profesional (nombre,especialidad,...	1 row(s) affected	0.000 sec
14	18:33:56	INSERT INTO Paciente (nombre,dni,obra_social) ...	1 row(s) affected	0.016 sec
15	18:34:15	INSERT INTO Turno (fecha_hora,duracion,estad...	1 row(s) affected	0.015 sec
16	18:41:10	INSERT INTO EntradaHC (fecha,actividades_rea...	1 row(s) affected	0.016 sec
17	18:42:25	INSERT INTO Factura (tipo_comprobante,nro_co...	1 row(s) affected	0.000 sec
18	18:43:09	SELECT * FROM Rol LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
19	18:43:09	SELECT * FROM Usuario LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
20	18:43:09	SELECT * FROM Profesional LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
21	18:43:09	SELECT * FROM Paciente LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
22	18:43:09	SELECT * FROM Turno LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
23	18:43:09	SELECT * FROM EntradaHC LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
24	18:43:09	SELECT * FROM Factura LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

Figura 35 – Ejecución de los scripts SQL INSERT INTO.

Usuarios y profesionales

Result Grid							
Filter Rows:							
	id_usuario	nombre	email	dni	contrasena_hash	estado	fecha_creacion
▶	1	María Gómez	maria@omnes.org	30123456	HASH123	Activo	2026-06-07 18:33:30
▶	2	Juan Pérez	juan@omnes.org	28987654	HASH456	Activo	2026-06-07 18:33:30
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Rol 1 Usuario 2 x Profesional 3 Paciente 4 Turno 5 EntradaHC 6 Factura 7 Apply Revert

Result Grid					
Filter Rows:					
	id_profesional	nombre	especialidad	matricula	porcentaje_aporte
▶	1	María Gómez	Psicopedagogía	MP-1234	15.00
*	NULL	NULL	NULL	NULL	NULL

Rol 1 Usuario 2 Profesional 3 x Paciente 4 Turno 5 EntradaHC 6 Factura 7 Apply Revert

Paciente y Turno

Result Grid			
Filter Rows:			
	id_paciente	nombre	dni
▶	1	Juan Pérez	32145678
*	NULL	NULL	NULL

Rol 1 Usuario 2 Profesional 3 Paciente 4 x Turno 5 EntradaHC 6 Factura 7 Apply Revert

Result Grid					
Filter Rows:					
	id_turno	fecha_hora	duracion	estado	id_paciente
▶	1	2026-06-06 10:00:00	60	Asignado	1
*	NULL	NULL	NULL	NULL	NULL

Rol 1 Usuario 2 Profesional 3 Paciente 4 Turno 5 x EntradaHC 6 Factura 7 Apply Revert

Historia clínica y factura

Result Grid								
Filter Rows:								
	id_entrada	fecha	actividades_realizadas	observaciones	estado	fecha_registro	turno_id	paciente_id
▶	1	2026-06-06	Evaluación inicial	Paciente con buena predisposición	Atendido	2026-06-07 18:41:10	1	1
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Rol 1 Usuario 2 Profesional 3 Paciente 4 Turno 5 EntradaHC 6 x Factura 7 Apply Revert

Result Grid														
Filter Rows:														
	id_factura	tipo_comprobante	nro_comprobante	fecha_emision	cae	fecha_venc_cae	cuit_emisor	razon_social_emisor	cuit_receptor	razon_social_receptor	periodo_desde	periodo_hasta	importe_total	aporte
▶	1	Factura C	0001-00000001	2026-06-06	12345678901234	2026-07-06	27234567891	María Gómez	30712345678	Fundación OMNES	2026-06-01	2026-06-30	100000.00	15000.00
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Rol 1 Usuario 2 Profesional 3 Paciente 4 Turno 5 EntradaHC 6 Factura 7 x Apply Revert

Consultas SQL

Una vez creada la estructura de la base de datos y cargados los registros de prueba, se realizaron diversas consultas SQL con el objetivo de verificar el correcto funcionamiento de las relaciones entre las entidades del sistema.

Las consultas implementadas permitieron recuperar información vinculada a la asignación de turnos, el registro de historias clínicas y la gestión de facturación, validando el funcionamiento de las claves foráneas y las operaciones de combinación de tablas mediante sentencias JOIN.

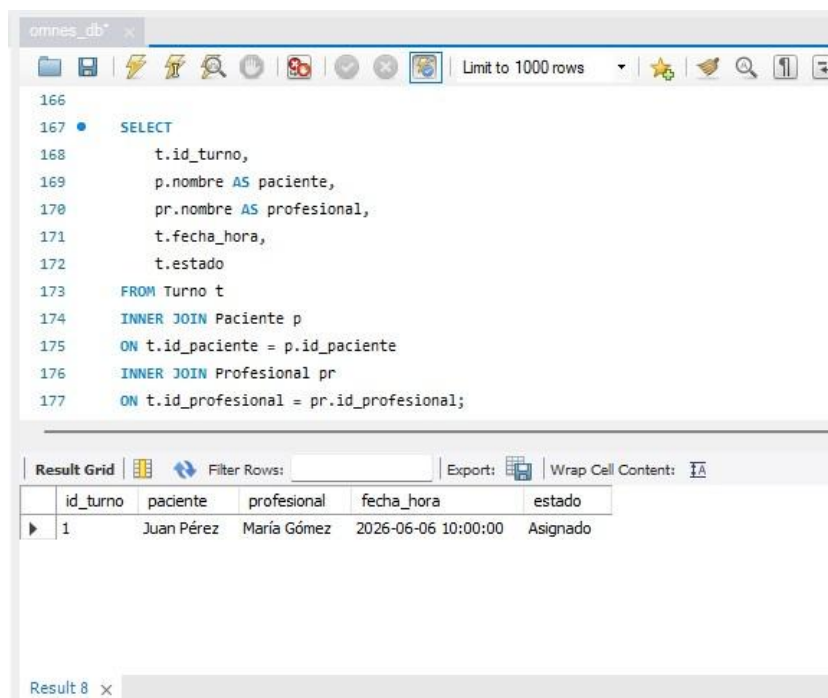
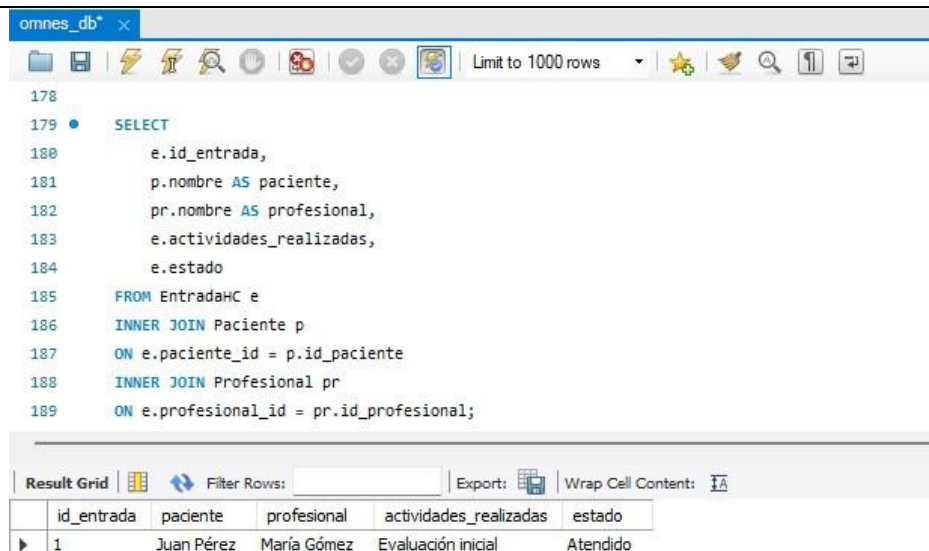


Figura 36 – Consulta de turnos asignados con información de pacientes y profesionales.

La consulta permite visualizar la información consolidada de los turnos registrados en el sistema, mostrando el paciente asociado, el profesional asignado, la fecha programada y el estado actual del turno.



The screenshot shows the 'omnes_db' database interface. The SQL query is as follows:

```

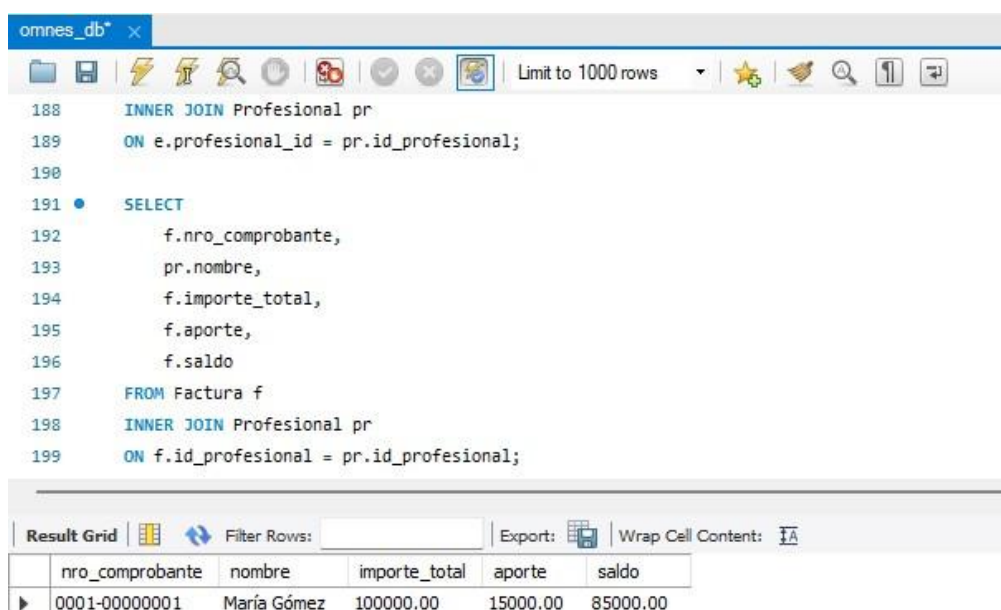
178
179 • SELECT
180     e.id_entrada,
181     p.nombre AS paciente,
182     pr.nombre AS profesional,
183     e.actividades_realizadas,
184     e.estado
185 FROM EntradaHC e
186 INNER JOIN Paciente p
187 ON e.paciente_id = p.id_paciente
188 INNER JOIN Profesional pr
189 ON e.profesional_id = pr.id_profesional;
  
```

The result grid displays the following data:

	id_entrada	paciente	profesional	actividades_realizadas	estado
▶	1	Juan Pérez	María Gómez	Evaluación inicial	Atendido

Figura 37 – Consulta de registros de historias clínicas.

La consulta recupera los registros de historias clínicas almacenados en la base de datos, vinculando cada entrada con el paciente y profesional correspondiente mediante operaciones JOIN sobre las tablas relacionadas.



The screenshot shows the 'omnes_db' database interface. The SQL query is as follows:

```

188 INNER JOIN Profesional pr
189 ON e.profesional_id = pr.id_profesional;
190
191 • SELECT
192     f.nro_comprobante,
193     pr.nombre,
194     f.importe_total,
195     f.aporte,
196     f.saldo
197 FROM Factura f
198 INNER JOIN Profesional pr
199 ON f.id_profesional = pr.id_profesional;
  
```

The result grid displays the following data:

	nro_comprobante	nombre	importe_total	aporte	saldo
▶	0001-00000001	María Gómez	100000.00	15000.00	85000.00

Figura 38 – Consulta de facturación por profesional.

La consulta permite visualizar las facturas registradas en el sistema junto con los importes facturados, aportes institucionales calculados y saldo correspondiente a cada profesional.

Eliminación de registros

Para completar la validación de las operaciones básicas sobre la base de datos, se realizó una prueba de eliminación de registros mediante sentencias DELETE. Este tipo de operación permite remover información almacenada cuando así lo requiere la lógica de negocio.

No obstante, debido a la necesidad de preservar la trazabilidad de la información clínica y administrativa, se recomienda que en futuras versiones del sistema se implemente una estrategia de baja lógica mediante el uso de estados, evitando la eliminación física de registros relevantes para la operación institucional.

Código de ejemplo

```
DELETE FROM Turno
WHERE id_turno = 1;
```

Definiciones de comunicación

Requerimientos de comunicación

El sistema requiere una comunicación permanente entre las estaciones cliente y el servidor central para permitir el acceso concurrente a la información institucional. Los mecanismos de comunicación implementados fueron definidos en el Diagrama de Despliegue presentado anteriormente.

Tecnologías y protocolos utilizados

Componente	Tecnología / Estándar
Aplicación cliente	JavaFX
Persistencia de datos	MySQL 8
Acceso a datos	JDBC
Protocolo de transporte	TCP/IP
Puerto utilizado	3306
Infraestructura de red	LAN Ethernet / Wi-Fi
Control de acceso	Usuarios y roles
Respaldo	mysqldump
Almacenamiento externo	NAS / Disco externo

Repositorio del proyecto

Como parte de las actividades de implementación y control de versiones, se creó un repositorio público en GitHub destinado al almacenamiento y seguimiento de los artefactos desarrollados durante el proyecto.

El repositorio permite centralizar la documentación técnica, los diagramas UML, los modelos de datos, los prototipos de interfaz, los scripts SQL y futuras versiones del sistema. Esta práctica facilita la trazabilidad de los cambios realizados durante el ciclo de vida del desarrollo y constituye una herramienta fundamental para la gestión colaborativa del proyecto.

Asimismo, el uso de GitHub permite mantener un historial de versiones de los distintos componentes del sistema, facilitando tareas de mantenimiento, evolución y recuperación de información ante modificaciones futuras.

Repositorio: <https://github.com/Davidemanuelgigena/sistema-gestion-omnes>

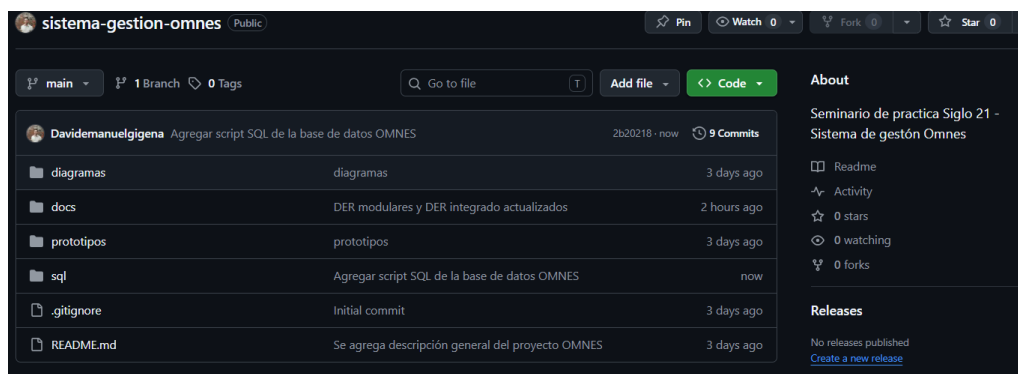


Figura 39 – Repositorio GitHub utilizado para la gestión de versiones del proyecto.

Desarrollo del sistema en Java

Introducción

En esta etapa se desarrolló una implementación funcional del módulo de gestión de turnos del Sistema Integral Fundación OMNES utilizando el lenguaje de programación Java. El objetivo principal fue aplicar los conceptos fundamentales de Programación Orientada a Objetos (POO) estudiados durante la asignatura, incorporando mecanismos de encapsulamiento, herencia, abstracción, polimorfismo y manejo de excepciones.

Para simplificar el alcance de la implementación y mantener coherencia con los modelos desarrollados en las etapas anteriores, se seleccionó el caso de uso "Asignar Turno", involucrando las entidades Paciente, Profesional y Turno.

La aplicación desarrollada permite registrar pacientes y profesionales, asignar turnos y consultar la información almacenada mediante un menú interactivo ejecutado desde consola. Durante el desarrollo se utilizaron constructores, objetos, estructuras condicionales y repetitivas, así como técnicas de validación y control de errores.

Diseño orientado a objetos

Diagrama de clases de implementación

Para la implementación se definió una estructura orientada a objetos basada en una clase abstracta denominada Persona, a partir de la cual heredan las clases Paciente y Profesional. Esta estructura permite reutilizar atributos y comportamientos comunes, reduciendo la duplicación de código y favoreciendo la mantenibilidad de la solución.

La clase Turno representa la asignación de un paciente a un profesional en una fecha y horario determinados. Finalmente, una clase principal contiene el menú de interacción con el usuario y coordina las operaciones del sistema.

El siguiente diagrama muestra las relaciones de herencia y asociación utilizadas durante la implementación.

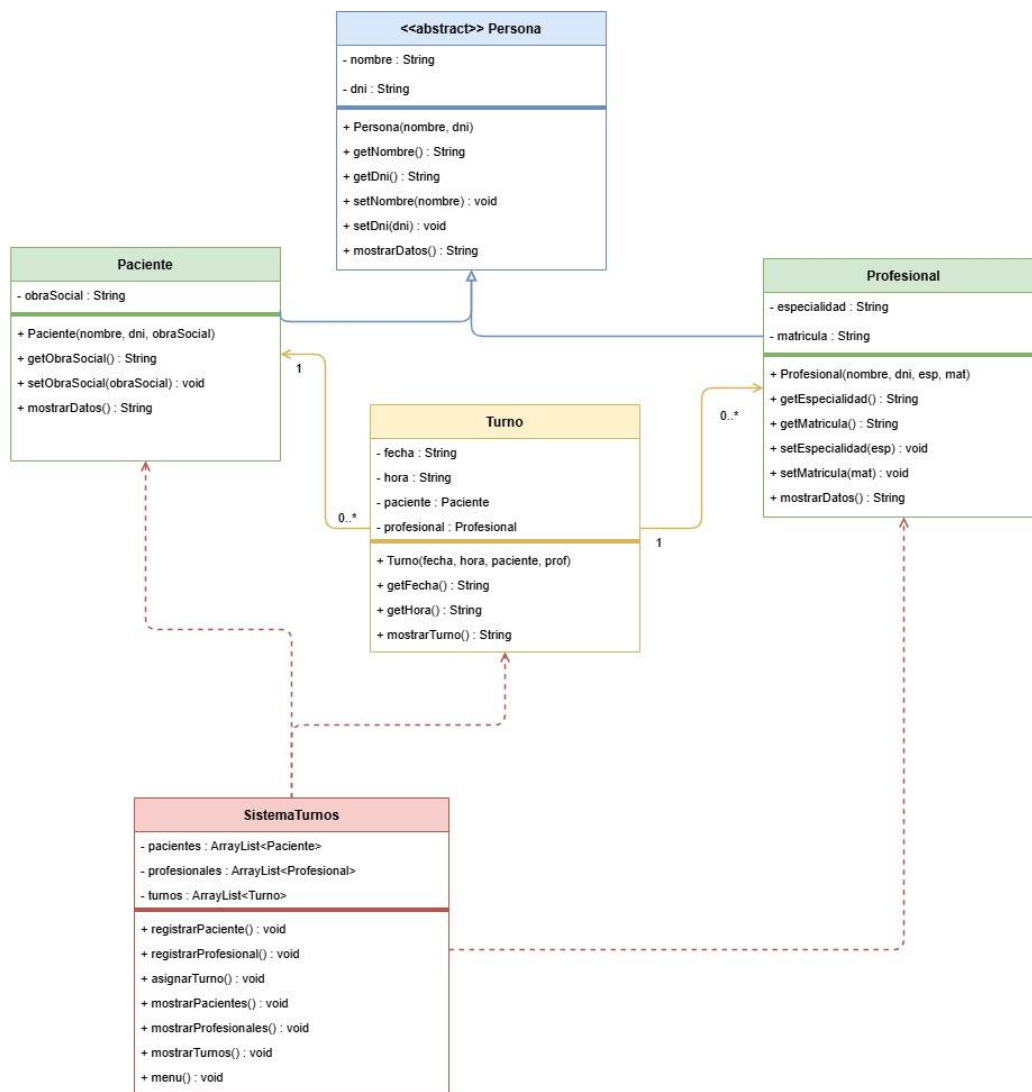


Figura 40 – Diagrama de clases de implementación correspondiente a la primera versión del sistema desarrollada por consola.

Aplicación de Programación Orientada a Objetos

Encapsulamiento

El encapsulamiento se implementó mediante la declaración de atributos privados dentro de las clases y el acceso controlado a través de métodos getters y setters. Esta práctica permite proteger la integridad de los datos y controlar las modificaciones realizadas sobre los objetos.

Herencia

La herencia se aplicó utilizando una clase abstracta denominada Persona, desde la cual derivan las clases Paciente y Profesional. De esta manera, ambas clases comparten atributos comunes como nombre y DNI, evitando duplicación de código.

Abstracción

La abstracción se implementó mediante la utilización de la clase abstracta Persona, que define características generales compartidas por los distintos tipos de personas registradas en el sistema, dejando que cada clase hija implemente sus comportamientos específicos.

Polimorfismo

El polimorfismo se aplicó mediante la redefinición de métodos heredados en las clases derivadas, permitiendo que cada objeto responda de forma particular a una misma operación según su tipo concreto.

Implementación

Estructura del proyecto

Para la implementación del módulo de gestión de turnos se desarrolló una aplicación Java utilizando Programación Orientada a Objetos. El proyecto fue organizado en clases independientes, respetando los principios de modularidad, reutilización de código y separación de responsabilidades.

La estructura del proyecto contempla una clase abstracta denominada Persona, dos clases derivadas (Paciente y Profesional), una clase Turno para representar las asignaciones realizadas y una clase principal denominada SistemaTurnos encargada de gestionar la interacción con el usuario mediante un menú de opciones.

Esta organización permitió aplicar los conceptos fundamentales de Programación Orientada a Objetos estudiados durante la asignatura, manteniendo una estructura simple, clara y fácilmente mantenible.

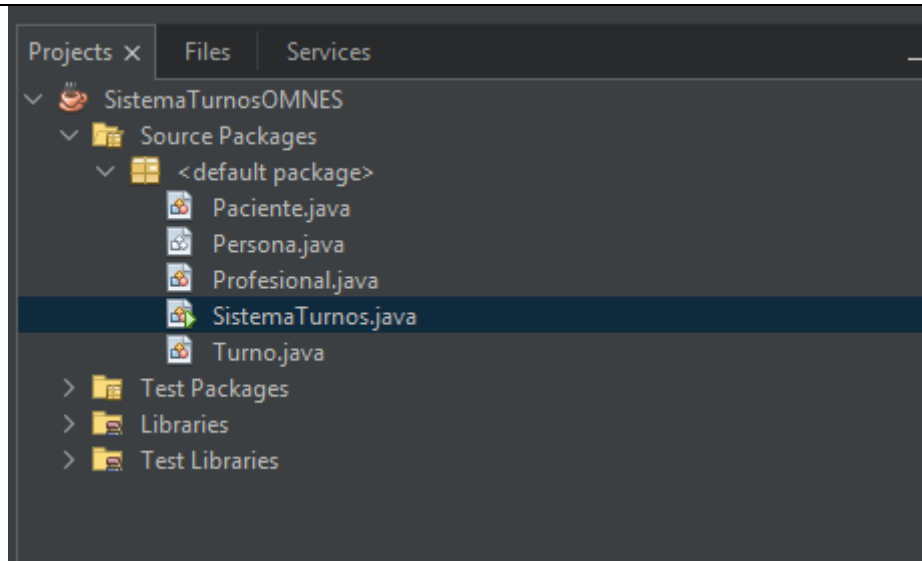


Figura 41 – Estructura del proyecto correspondiente a la primera implementación del sistema.

Clases desarrolladas

Clase abstracta Persona

La clase Persona constituye la superclase abstracta del sistema y representa la base común para las entidades Paciente y Profesional. Su finalidad es centralizar los atributos y comportamientos compartidos por los distintos tipos de personas gestionadas por la aplicación, evitando la duplicación de código y favoreciendo la reutilización mediante el mecanismo de herencia.

Dentro de esta clase se definieron los atributos nombre y dni, ambos declarados con modificador de acceso privado para preservar el encapsulamiento de la información. El acceso y modificación de dichos atributos se realiza mediante los métodos getter y setter, garantizando un acceso controlado a los datos del objeto.

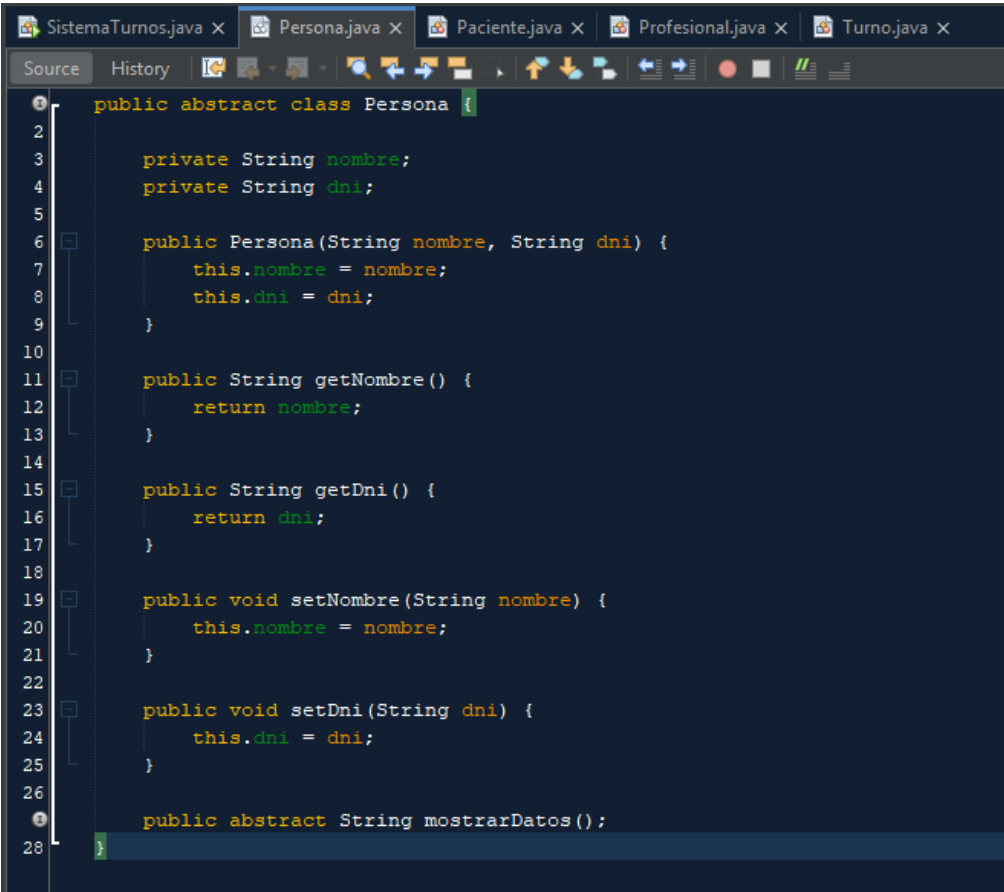
Asimismo, la clase incorpora un constructor que recibe como parámetros el nombre y el DNI de la persona, inicializando dichos atributos al momento de crear el objeto. Este constructor es reutilizado por las clases derivadas mediante la invocación al método `super()`, evitando la duplicación de código y favoreciendo la reutilización propia del mecanismo de herencia.

Finalmente, se declaró el método abstracto `mostrarDatos()`, el cual debe ser implementado por las clases derivadas. Esta decisión permite que cada tipo de persona defina su propia forma de presentar la información, aplicando el principio de polimorfismo de la Programación Orientada a Objetos.

La utilización de una clase abstracta permitió aplicar los principios de abstracción, encapsulamiento, herencia y polimorfismo, estableciendo una base sólida para la organización del modelo de dominio del prototipo.

Conceptos de Programación Orientada a Objetos aplicados

Concepto	Aplicación en la clase Persona
Encapsulamiento	Los atributos nombre y dni fueron declarados privados (private) y su acceso se realiza mediante métodos getter y setter.
Abstracción	La clase fue declarada como abstract, definiendo una estructura común para las entidades del dominio sin permitir su instanciación directa.
Herencia	La clase actúa como superclase de Paciente y Profesional, permitiendo reutilizar atributos y comportamientos comunes.
Polimorfismo	Se definió el método abstracto mostrarDatos(), cuya implementación es redefinida por las clases derivadas.
Constructor	El constructor inicializa los atributos comunes de toda persona registrada en el sistema.



```

1 public abstract class Persona {
2
3     private String nombre;
4     private String dni;
5
6     public Persona(String nombre, String dni) {
7         this.nombre = nombre;
8         this.dni = dni;
9     }
10
11     public String getNombre() {
12         return nombre;
13     }
14
15     public String getDni() {
16         return dni;
17     }
18
19     public void setNombre(String nombre) {
20         this.nombre = nombre;
21     }
22
23     public void setDni(String dni) {
24         this.dni = dni;
25     }
26
27     public abstract String mostrarDatos();
28 }

```

Figura 42 - Implementación de la clase abstracta Persona en Java.

Clase Paciente

La clase Paciente representa a las personas que solicitan atención dentro de la Fundación OMNES. Esta clase hereda de la superclase abstracta Persona, reutilizando los atributos comunes nombre y dni, e incorporando el atributo obraSocial, necesario para identificar la cobertura médica de cada paciente.

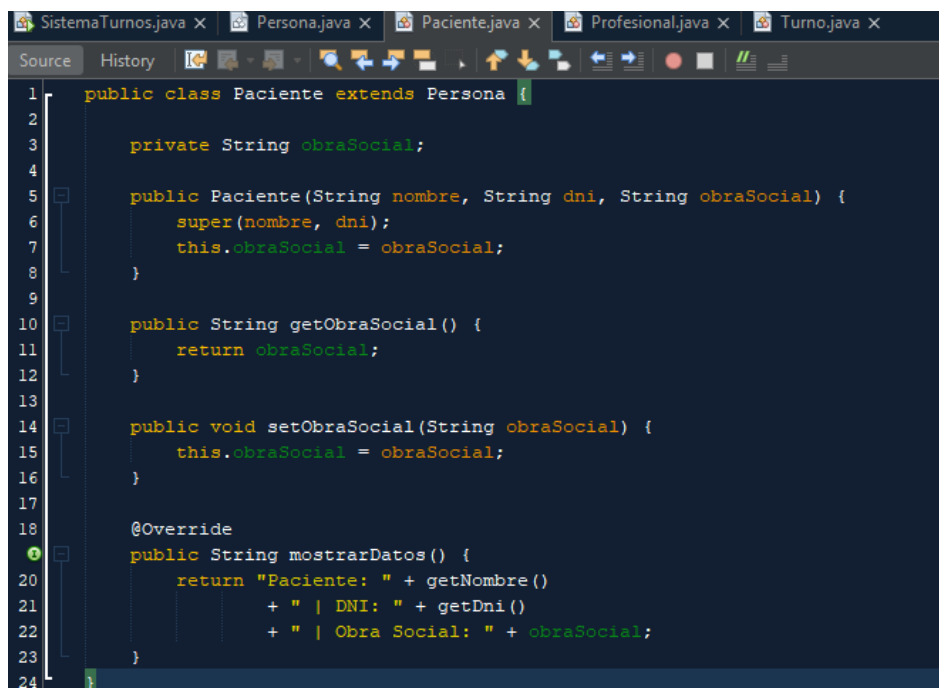
La implementación mediante herencia permitió reutilizar la estructura definida en la clase Persona, evitando la duplicación de atributos y métodos comunes. El constructor de la clase recibe como

parámetros el nombre, el DNI y la obra social, inicializando los atributos heredados mediante la invocación al constructor de la superclase utilizando el método `super()`.

Asimismo, la clase redefine el método `mostrarDatos()` mediante la anotación `@Override`, implementando una presentación específica de la información correspondiente a cada paciente. Esta sobrescritura constituye una aplicación del principio de polimorfismo, permitiendo que cada clase derivada proporcione su propia implementación del método definido en la clase abstracta.

Durante el proceso de asignación de turnos, los objetos de tipo `Paciente` son registrados previamente en el sistema y posteriormente asociados a un objeto `Turno`, estableciendo la relación entre el paciente, el profesional y la fecha de atención.

Concepto	Aplicación en la clase Paciente
Herencia	La clase <code>Paciente</code> extiende la clase abstracta <code>Persona</code> , reutilizando sus atributos y métodos comunes.
Encapsulamiento	El atributo <code>obraSocial</code> fue declarado privado (<code>private</code>) y su acceso se realiza mediante métodos <code>getter</code> y <code>setter</code> .
Constructor	El constructor inicializa la información específica del paciente y utiliza <code>super()</code> para inicializar los atributos heredados.
Polimorfismo	El método <code>mostrarDatos()</code> sobrescribe la implementación abstracta definida en <code>Persona</code> , adaptando la presentación de la información del paciente.
Declaración y creación de objetos	Los objetos <code>Paciente</code> son instanciados durante el registro de pacientes y posteriormente utilizados en la creación de los turnos del sistema.



```

1  public class Paciente extends Persona {
2
3      private String obraSocial;
4
5      public Paciente(String nombre, String dni, String obraSocial) {
6          super(nombre, dni);
7          this.obraSocial = obraSocial;
8      }
9
10     public String getObraSocial() {
11         return obraSocial;
12     }
13
14     public void setObraSocial(String obraSocial) {
15         this.obraSocial = obraSocial;
16     }
17
18     @Override
19     public String mostrarDatos() {
20         return "Paciente: " + getNombre()
21             + " | DNI: " + getDni()
22             + " | Obra Social: " + obraSocial;
23     }
24 }

```

Figura 43 - Implementación de la clase `Paciente` en Java

Clase Profesional

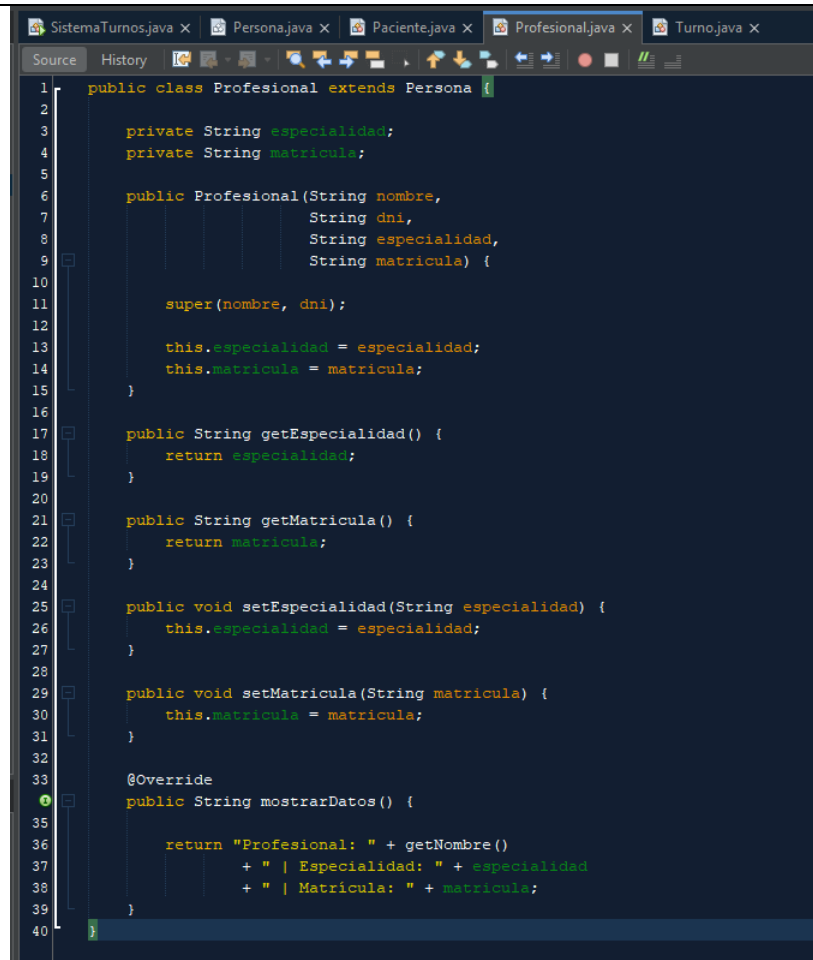
La clase Profesional representa a los especialistas que brindan atención dentro de la Fundación OMNES. Esta clase hereda de la superclase abstracta Persona, reutilizando los atributos comunes nombre y dni, e incorporando los atributos especialidad y matricula, necesarios para identificar la formación y habilitación profesional de cada integrante del sistema.

El constructor de la clase recibe como parámetros el nombre, el DNI, la especialidad y la matrícula profesional. Los atributos heredados son inicializados mediante la invocación al constructor de la superclase utilizando el método `super()`, mientras que los atributos propios de la clase son inicializados dentro del constructor correspondiente.

Asimismo, la clase redefine el método `mostrarDatos()` mediante la anotación `@Override`, proporcionando una implementación específica para visualizar la información de cada profesional registrado. Esta sobrescritura constituye una aplicación del principio de polimorfismo, permitiendo que cada clase derivada implemente su propia forma de representar la información.

Durante el proceso de asignación de turnos, los objetos de tipo Profesional son registrados previamente en el sistema y posteriormente seleccionados para asociarlos a un paciente, conformando así la información necesaria para la creación de un nuevo turno.

Concepto	Aplicación en la clase Profesional
Herencia	La clase Profesional extiende la clase abstracta Persona, reutilizando sus atributos y métodos comunes.
Encapsulamiento	Los atributos especialidad y matricula fueron declarados privados (<code>private</code>) y su acceso se realiza mediante métodos <code>getter</code> y <code>setter</code> .
Constructor	El constructor inicializa la información específica del profesional y utiliza <code>super()</code> para inicializar los atributos heredados.
Polimorfismo	El método <code>mostrarDatos()</code> sobrescribe la implementación abstracta definida en Persona, adaptando la presentación de la información del profesional.
Declaración y creación de objetos	Los objetos Profesional son instanciados durante el registro de profesionales y posteriormente utilizados para asignar turnos a los pacientes.



```

1  public class Profesional extends Persona {
2
3      private String especialidad;
4      private String matricula;
5
6      public Profesional(String nombre,
7                          String dni,
8                          String especialidad,
9                          String matricula) {
10
11          super(nombre, dni);
12
13          this.especialidad = especialidad;
14          this.matricula = matricula;
15      }
16
17      public String getEspecialidad() {
18          return especialidad;
19      }
20
21      public String getMatricula() {
22          return matricula;
23      }
24
25      public void setEspecialidad(String especialidad) {
26          this.especialidad = especialidad;
27      }
28
29      public void setMatricula(String matricula) {
30          this.matricula = matricula;
31      }
32
33      @Override
34      public String mostrarDatos() {
35
36          return "Profesional: " + getNombre()
37              + " | Especialidad: " + especialidad
38              + " | Matricula: " + matricula;
39      }
40  }

```

Figura 44 - Implementación de la clase Profesional en Java.

Clase Turno

La clase Turno constituye la entidad principal del prototipo desarrollado, ya que implementa el proceso central de la aplicación: la asignación de turnos entre pacientes y profesionales de la Fundación OMNES. Su finalidad es registrar la fecha y el horario de una consulta, asociando un paciente previamente registrado con un profesional disponible.

Para representar esta relación, la clase mantiene referencias a objetos de las clases Paciente y Profesional, estableciendo una asociación entre ambas entidades dentro del modelo de dominio. De esta manera, cada objeto Turno almacena la información necesaria para identificar quién recibirá la atención, quién la brindará y cuándo se llevará a cabo.

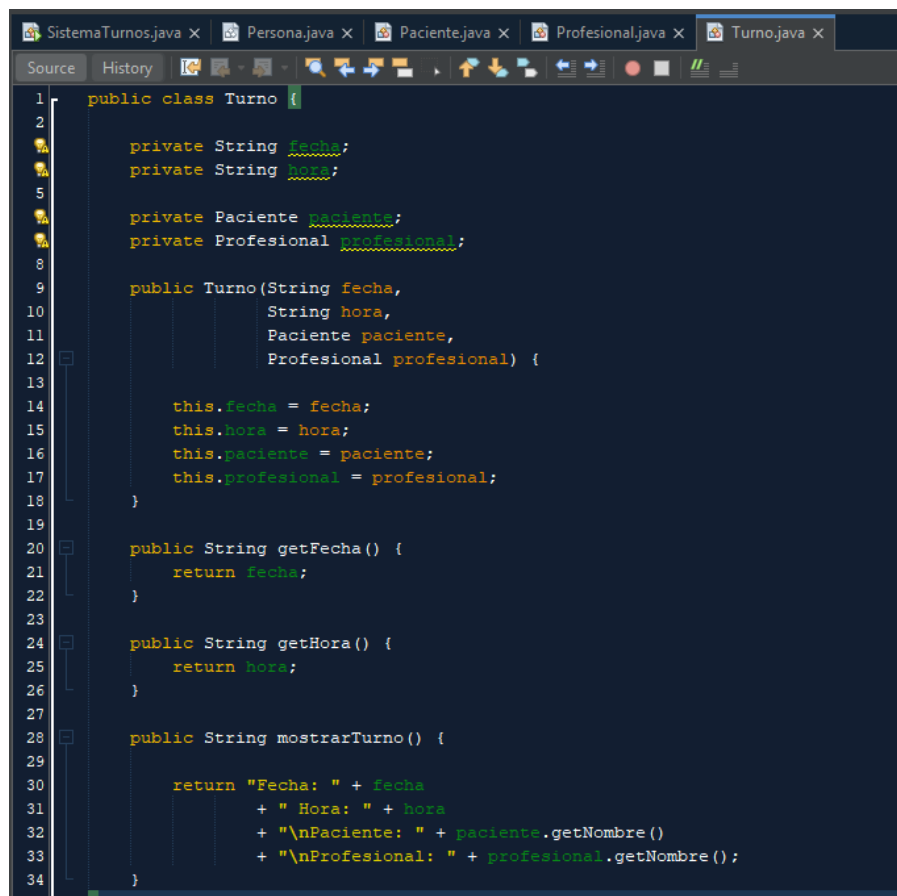
El constructor de la clase recibe como parámetros la fecha, el horario, el paciente y el profesional, inicializando todos los atributos necesarios para representar un turno completo. Esta implementación demuestra la utilización de la declaración y creación de objetos, así como la asociación entre diferentes clases mediante referencias a objetos.

Asimismo, la clase incorpora el método `mostrarTurno()`, encargado de presentar de forma clara la información almacenada en cada turno, accediendo a los datos de los objetos `Paciente` y `Profesional` mediante sus respectivos métodos de acceso.

Durante la ejecución del programa, cada nuevo turno es creado a partir de los pacientes y profesionales previamente registrados, siendo posteriormente incorporado a la colección de turnos administrada por la aplicación.

Conceptos de Programación Orientada a Objetos aplicados

Concepto	Aplicación en la clase Profesional
Asociación entre objetos	La clase mantiene referencias a objetos <code>Paciente</code> y <code>Profesional</code> , representando la relación existente entre ambas entidades.
Encapsulamiento	Los atributos <code>fecha</code> , <code>hora</code> , <code>paciente</code> y <code>profesional</code> fueron declarados privados (<code>private</code>), garantizando un acceso controlado mediante métodos públicos.
Constructor	El constructor inicializa la información correspondiente a la fecha, horario, paciente y profesional de cada turno registrado.
Declaración y creación de objetos	Cada objeto <code>Turno</code> se crea utilizando como parámetros objetos previamente instanciados de las clases <code>Paciente</code> y <code>Profesional</code> .
Reutilización de objetos	El método <code>mostrarTurno()</code> accede a la información de los objetos asociados utilizando sus métodos públicos, evitando duplicar información dentro de la clase.



```

1  public class Turno {
2
3      private String fecha;
4      private String hora;
5
6      private Paciente paciente;
7      private Profesional profesional;
8
9      public Turno(String fecha,
10                  String hora,
11                  Paciente paciente,
12                  Profesional profesional) {
13
14          this.fecha = fecha;
15          this.hora = hora;
16          this.paciente = paciente;
17          this.profesional = profesional;
18      }
19
20      public String getFecha() {
21          return fecha;
22      }
23
24      public String getHora() {
25          return hora;
26      }
27
28      public String mostrarTurno() {
29
30          return "Fecha: " + fecha
31                + " Hora: " + hora
32                + "\nPaciente: " + paciente.getNombre()
33                + "\nProfesional: " + profesional.getNombre();
34      }
35  }

```

Figura 45 - Implementación de la clase Turno en Java.

Clase SistemaTurnos

La clase **SistemaTurnos** constituye el núcleo funcional de la aplicación, ya que concentra la lógica principal del prototipo y coordina la interacción entre las distintas entidades del sistema. Desde esta clase se administran los procesos de registro de pacientes, registro de profesionales, asignación de turnos y consulta de la información almacenada.

Para gestionar la información durante la ejecución del programa se implementaron tres colecciones dinámicas mediante la clase **ArrayList**, destinadas al almacenamiento de pacientes, profesionales y turnos. La utilización de esta estructura permitió incorporar y recuperar objetos de forma dinámica, sin necesidad de definir un tamaño fijo para las colecciones, facilitando el crecimiento de la información durante la ejecución de la aplicación.

La clase también incorpora un objeto de tipo **Scanner**, utilizado para capturar la información ingresada por el usuario desde la consola y permitir la interacción con el menú principal del sistema.

El constructor de la clase inicializa las colecciones de objetos y el mecanismo de entrada de datos, garantizando que la aplicación disponga de todos los recursos necesarios antes de comenzar su ejecución.

Dentro de esta clase se implementaron los principales procesos funcionales del prototipo:

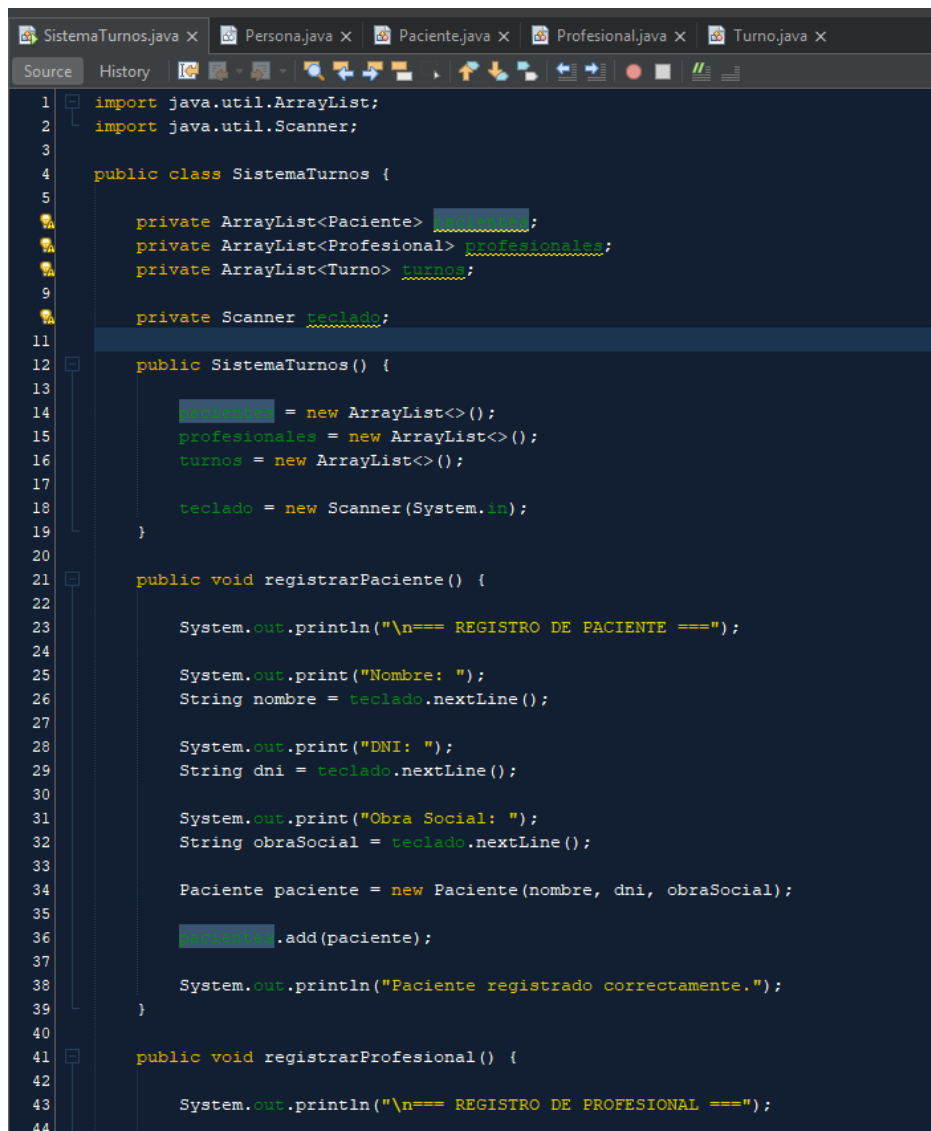
- **registrarPaciente()**, encargado de solicitar los datos del paciente, crear un nuevo objeto Paciente y almacenarlo en la colección correspondiente.
- **registrarProfesional()**, responsable de registrar la información de los profesionales disponibles para la atención.
- **asignarTurno()**, proceso central del sistema, mediante el cual se seleccionan un paciente y un profesional previamente registrados para generar un nuevo objeto Turno.
- **mostrarPacientes()**, **mostrarProfesionales()** y **mostrarTurnos()**, destinados a visualizar la información almacenada en las distintas colecciones.
- **menu()**, encargado de controlar el flujo general de ejecución de la aplicación mediante una estructura repetitiva y un menú de opciones.

Para garantizar la robustez del sistema se incorporó un bloque **try-catch** durante la lectura de las opciones del menú, capturando posibles errores producidos por el ingreso de datos inválidos y evitando la finalización inesperada de la aplicación. Esta implementación mejora la experiencia del usuario y constituye una correcta aplicación del manejo de excepciones en Java.

Finalmente, el método **main()** actúa como punto de entrada de la aplicación, creando una instancia de la clase **SistemaTurnos** e iniciando la ejecución del menú principal.

Conceptos de Programación Orientada a Objetos y Java aplicados

Concepto	Aplicación en la clase Profesional
ArrayList	Se utilizaron tres colecciones dinámicas para almacenar pacientes, profesionales y turnos durante la ejecución del sistema.
Declaración y creación de objetos	Se instancian objetos Paciente, Profesional y Turno, incorporándolos posteriormente a las colecciones correspondientes.
Constructor	El constructor inicializa las colecciones (ArrayList) y el objeto Scanner utilizado para la interacción con el usuario.
Condicionales	Se emplean estructuras if para validar la existencia de información antes de realizar determinadas operaciones.
Estructuras repetitivas	Se utilizan ciclos for para recorrer las colecciones y un ciclo do-while para mantener activo el menú principal hasta la finalización del programa.
Manejo de excepciones	Se implementa un bloque try-catch para controlar errores producidos durante el ingreso de datos desde la consola.
Modularidad	Cada proceso principal del sistema fue implementado en métodos independientes, favoreciendo la organización y reutilización del código



```

1  import java.util.ArrayList;
2  import java.util.Scanner;
3
4  public class SistemaTurnos {
5
6      private ArrayList<Paciente> pacientes;
7      private ArrayList<Profesional> profesionales;
8      private ArrayList<Turno> turnos;
9
10     private Scanner teclado;
11
12     public SistemaTurnos() {
13
14         pacientes = new ArrayList<>();
15         profesionales = new ArrayList<>();
16         turnos = new ArrayList<>();
17
18         teclado = new Scanner(System.in);
19     }
20
21     public void registrarPaciente() {
22
23         System.out.println("\n=== REGISTRO DE PACIENTE ===");
24
25         System.out.print("Nombre: ");
26         String nombre = teclado.nextLine();
27
28         System.out.print("DNI: ");
29         String dni = teclado.nextLine();
30
31         System.out.print("Obra Social: ");
32         String obraSocial = teclado.nextLine();
33
34         Paciente paciente = new Paciente(nombre, dni, obraSocial);
35
36         pacientes.add(paciente);
37
38         System.out.println("Paciente registrado correctamente.");
39     }
40
41     public void registrarProfesional() {
42
43         System.out.println("\n=== REGISTRO DE PROFESIONAL ===");
44

```

```

45     System.out.print("Nombre: ");
46     String nombre = teclado.nextLine();
47
48     System.out.print("DNI: ");
49     String dni = teclado.nextLine();
50
51     System.out.print("Especialidad: ");
52     String especialidad = teclado.nextLine();
53
54     System.out.print("Matrícula: ");
55     String matricula = teclado.nextLine();
56
57     Profesional profesional =
58         new Profesional(nombre, dni, especialidad, matricula);
59

```

```

60     profesionales.add(profesional);
61
62     System.out.println("Profesional registrado correctamente.");
63 }
64
65 public void mostrarPacientes() {
66
67     System.out.println("\n=== PACIENTES ===");
68
69     if (pacientes.isEmpty()) {
70         System.out.println("No hay pacientes registrados.");
71         return;
72     }
73
74     for (Paciente p : pacientes) {
75         System.out.println(p.mostrarDatos());
76     }
77 }
78
79 public void mostrarProfesionales() {
80
81     System.out.println("\n=== PROFESIONALES ===");
82
83     if (profesionales.isEmpty()) {
84         System.out.println("No hay profesionales registrados.");
85         return;
86     }
87
88     for (Profesional p : profesionales) {

```

```

89         System.out.println(p.mostrarDatos());
90     }
91 }
92
93 public void asignarTurno() {
94
95     if (pacientes.isEmpty() || profesionales.isEmpty()) {
96
97         System.out.println(
98             "Debe registrar al menos un paciente y un profesional.");
99         return;
100     }
101
102     System.out.println("\n=== ASIGNAR TURNO ===");
103
104     System.out.println("\nPacientes:");
105
106     for (int i = 0; i < pacientes.size(); i++) {
107         System.out.println(
108             (i + 1) + " - " + pacientes.get(i).getNombre());
109     }
110
111     System.out.print("Seleccione paciente: ");
112     int indicePaciente = Integer.parseInt(teclado.nextLine()) - 1;
113
114     System.out.println("\nProfesionales:");
115
116     for (int i = 0; i < profesionales.size(); i++) {
117         System.out.println(
118             (i + 1) + " - " + profesionales.get(i).getNombre());
119     }
120
121     System.out.print("Seleccione profesional: ");
122     int indiceProfesional = Integer.parseInt(teclado.nextLine()) - 1;
123

```

```

124     System.out.print("Fecha: ");
125     String fecha = teclado.nextLine();
126
127     System.out.print("Hora: ");
128     String hora = teclado.nextLine();
129
130     Turno turno = new Turno(
131         fecha,
132         hora,
133         pacientes.get(indicePaciente),
134         profesionales.get(indiceProfesional));
135
136     turnos.add(turno);
137
138     System.out.println("Turno asignado correctamente.");
139 }
140
141 public void mostrarTurnos() {
142
143     System.out.println("\n=== TURNOS ===");
144
145     if (turnos.isEmpty()) {
146
147         System.out.println("No hay turnos registrados.");
148         return;
149     }
150
151     for (Turno t : turnos) {
152
153         System.out.println("-----");
154         System.out.println(t.mostrarTurno());
155     }
156 }
157
158 public void menu() {
159
160     int opcion;
161

```

```

162 do {
163
164     System.out.println("\n=====");
165     System.out.println(" SISTEMA TURNOS OMNES ");
166     System.out.println("=====");
167     System.out.println("1 - Registrar paciente");
168     System.out.println("2 - Registrar profesional");
169     System.out.println("3 - Asignar turno");
170     System.out.println("4 - Mostrar pacientes");
171     System.out.println("5 - Mostrar profesionales");
172     System.out.println("6 - Mostrar turnos");
173     System.out.println("7 - Salir");
174
175     System.out.print("Opción: ");
176
177     try {
178
179         opcion = Integer.parseInt(teclado.nextLine());
180
181         switch (opcion) {
182
183             case 1:
184                 registrarPaciente();
185                 break;
186
187             case 2:
188                 registrarProfesional();

```

Figura 46 - Implementación de la clase SistemaTurnos en Java.

Manejo de excepciones

Con el objetivo de incrementar la robustez del prototipo y evitar interrupciones inesperadas durante la ejecución, se implementó el manejo de excepciones mediante la estructura **try-catch** dentro del menú principal de la aplicación.

Esta implementación permite controlar los errores que pueden producirse durante el ingreso de datos por parte del usuario, particularmente cuando se espera un valor numérico para seleccionar una opción del menú y se introduce un dato incompatible con el tipo solicitado. En estas situaciones, la excepción es capturada y tratada de manera controlada, evitando la finalización abrupta del programa.

Cuando se detecta una entrada inválida, el sistema informa el inconveniente mediante un mensaje descriptivo y restablece la variable de control del menú, permitiendo que la aplicación continúe su ejecución normal sin pérdida de la información previamente almacenada.

La utilización de este mecanismo constituye una buena práctica de programación, ya que mejora la confiabilidad del sistema, incrementa la tolerancia frente a errores de ingreso y proporciona una experiencia de uso más segura para el usuario.

Conceptos aplicados

Concepto	Aplicación en la clase Profesional
Manejo de excepciones	Se implementó un bloque try-catch para controlar errores producidos durante el ingreso de opciones del menú principal.
Validación de datos	Se verifica que la opción ingresada pueda convertirse correctamente a un valor numérico mediante Integer.parseInt().
Recuperación ante errores	Cuando ocurre una excepción, el sistema informa el error y continúa su ejecución sin finalizar el programa.

Robustez	La aplicación mantiene su funcionamiento normal aun cuando el usuario ingresa información incorrecta.
----------	---

```

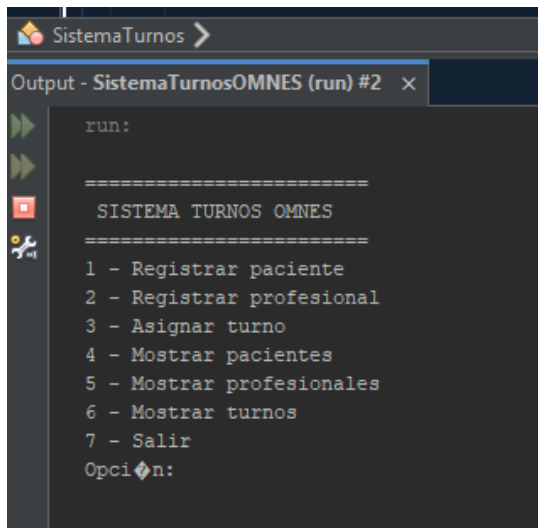
177     try {
178
179         opcion = Integer.parseInt(teclado.nextLine());
180
181         switch (opcion) {
182
183             case 1:
184                 registrarPaciente();
185                 break;
186
187             case 2:
188                 registrarProfesional();
189                 break;
190
191             case 3:
192                 asignarTurno();
193                 break;
194
195             case 4:
196                 mostrarPacientes();
197                 break;
198
199             case 5:
200                 mostrarProfesionales();
201                 break;
202
203             case 6:
204                 mostrarTurnos();
205                 break;
206
207             case 7:
208                 System.out.println("Fin del programa.");
209                 break;
210
211             default:
212                 System.out.println("Opción inválida.");
213         }
214     } catch (Exception e) {
215
216         System.out.println(
217             "Error: debe ingresar un valor válido.");
218
219         opcion = 0;
220     }
221
222     } while (opcion != 7);
223 }
224
225
226 public static void main(String[] args) {
227
228     SistemaTurnos sistema = new SistemaTurnos();
229
230     sistema.menu();
231 }
232

```

Figura 47 - Implementación del manejo de excepciones mediante la estructura try-catch.

Menú principal

El menú principal constituye el punto de interacción entre el usuario y el sistema. Su implementación se realizó mediante una estructura repetitiva **do-while** que mantiene la aplicación en ejecución hasta que el usuario selecciona la opción de salida. La navegación entre las distintas funcionalidades se resuelve utilizando una estructura **switch-case**, invocando el método correspondiente a cada operación disponible.



```
run:
=====
SISTEMA TURNOS OMNES
=====
1 - Registrar paciente
2 - Registrar profesional
3 - Asignar turno
4 - Mostrar pacientes
5 - Mostrar profesionales
6 - Mostrar turnos
7 - Salir
Opción:
```

Figura 48 – Menú principal en consola.

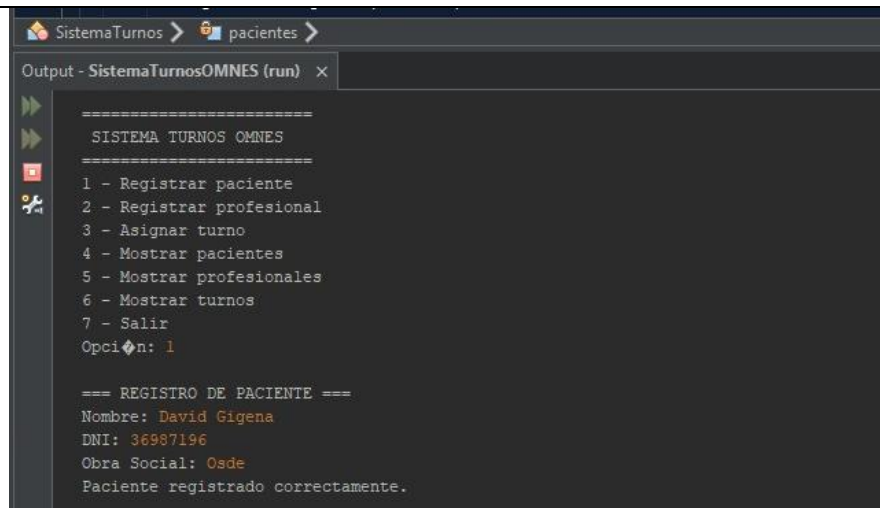
Ejecución y pruebas

Registro de pacientes

Con el objetivo de verificar el correcto funcionamiento del sistema, se realizaron pruebas sobre las funcionalidades implementadas. La primera prueba consistió en registrar un paciente mediante el menú principal de la aplicación.

Durante la ejecución se ingresaron los datos correspondientes al nombre, DNI y obra social del paciente. Una vez completada la carga, el sistema creó una nueva instancia de la clase Paciente y la almacenó dentro de la colección correspondiente, confirmando la operación mediante un mensaje informativo.

Esta prueba permitió validar el funcionamiento de los constructores, la creación de objetos y el almacenamiento de información en estructuras dinámicas de tipo ArrayList.



```

SistemaTurnos > pacientes >
Output - SistemaTurnosOMNES (run) x
=====
SISTEMA TURNOS OMNES
=====
1 - Registrar paciente
2 - Registrar profesional
3 - Asignar turno
4 - Mostrar pacientes
5 - Mostrar profesionales
6 - Mostrar turnos
7 - Salir
Opcion: 1

=== REGISTRO DE PACIENTE ===
Nombre: David Gigena
DNI: 36987196
Obra Social: Osde
Paciente registrado correctamente.
  
```

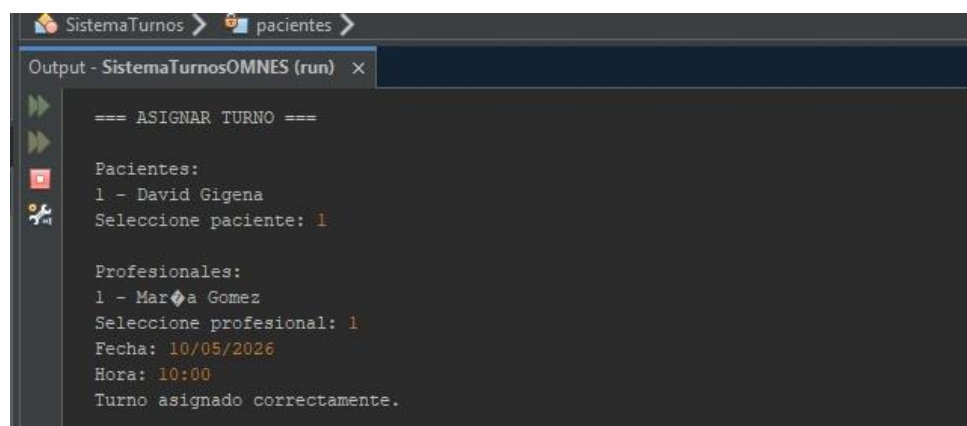
Figura 49 – Registro de pacientes.

Asignación de turnos

Posteriormente se realizó una prueba de asignación de turnos entre pacientes y profesionales previamente registrados.

El sistema presentó al usuario las listas disponibles de pacientes y profesionales, permitiendo seleccionar cada uno mediante su correspondiente identificador. Luego se ingresaron la fecha y horario del turno, generándose una nueva instancia de la clase Turno.

Esta funcionalidad permitió validar las relaciones existentes entre objetos y el uso de referencias a instancias previamente creadas dentro del sistema.



```

SistemaTurnos > pacientes >
Output - SistemaTurnosOMNES (run) x
=====
=== ASIGNAR TURNO ===
Pacientes:
1 - David Gigena
Seleccione paciente: 1

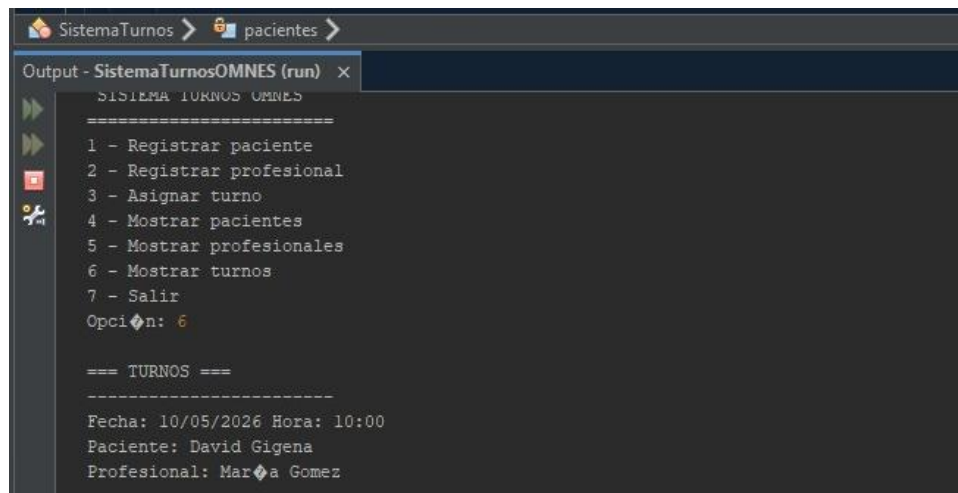
Profesionales:
1 - Mariana Gomez
Seleccione profesional: 1
Fecha: 10/05/2026
Hora: 10:00
Turno asignado correctamente.
  
```

Figura 50 – Asignación de turnos.

Consulta de turnos

Una vez registrados los datos de prueba, se verificó la funcionalidad de consulta de turnos. El sistema recuperó correctamente la información almacenada y mostró los datos asociados al paciente, profesional, fecha y horario del turno registrado.

Esta prueba permitió comprobar el correcto almacenamiento de los objetos creados y la utilización de los métodos implementados para la recuperación y visualización de información.



```

SistemaTurnos > pacientes >
Output - SistemaTurnosOMNES (run) x
=====
1 - Registrar paciente
2 - Registrar profesional
3 - Asignar turno
4 - Mostrar pacientes
5 - Mostrar profesionales
6 - Mostrar turnos
7 - Salir
Opción: 6

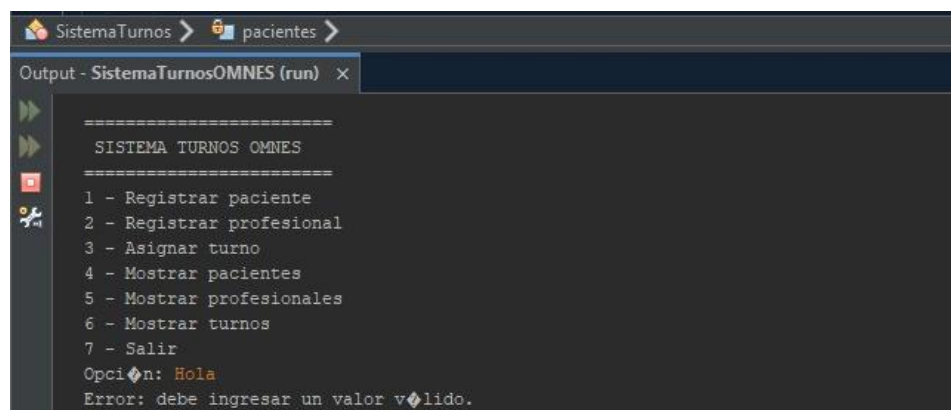
=== TURNOS ===
-----
Fecha: 10/05/2026 Hora: 10:00
Paciente: David Gigena
Profesional: María Gomez
  
```

Figura 51 – Consulta de turnos.

Manejo de excepciones

Finalmente se realizó una prueba de validación ingresando datos inválidos dentro del menú principal. El sistema detectó correctamente la excepción generada, informó el error al usuario mediante un mensaje descriptivo y continuó su ejecución sin interrupciones.

Esta prueba permitió verificar el correcto funcionamiento del mecanismo de manejo de excepciones implementado mediante la estructura try-catch, confirmando la robustez del prototipo frente a entradas incorrectas.



```

SistemaTurnos > pacientes >
Output - SistemaTurnosOMNES (run) x
=====
1 - Registrar paciente
2 - Registrar profesional
3 - Asignar turno
4 - Mostrar pacientes
5 - Mostrar profesionales
6 - Mostrar turnos
7 - Salir
Opción: Hola
Error: debe ingresar un valor válido.
  
```

Figura 52 – Manejo de excepciones

Repositorio del proyecto

Se actualiza y registra el repositorio con los archivos correspondientes la implementación en java del modulo de gestión de turnos.

Repositorio: <https://github.com/Davidmanuelgigena/sistema-gestion-omnes>



Figura 53 – Repositorio GitHub utilizado para la gestión de versiones del proyecto actualizado.

Implementación del prototipo en Java

Selección del patrón de diseño y justificación

Para el desarrollo del prototipo se seleccionó el patrón de arquitectura Modelo–Vista–Controlador (MVC) debido a que proporciona una adecuada separación entre la lógica de negocio, la interfaz de usuario y la gestión de los datos. Esta organización resulta especialmente conveniente para aplicaciones que evolucionan de forma incremental, como el Sistema Integral de gestión para la Fundación OMNES.

La utilización de MVC evita concentrar toda la funcionalidad de la aplicación en una única clase o módulo, práctica habitual en desarrollos monolíticos de pequeña escala que dificultan el mantenimiento, la reutilización del código y la incorporación de nuevas funcionalidades. Al separar las responsabilidades de cada componente, es posible modificar la interfaz, la lógica de negocio o la persistencia de datos de forma independiente, reduciendo el acoplamiento entre las distintas partes del sistema.

En la arquitectura implementada, el Modelo representa las entidades del dominio (Paciente, Profesional y Turno), la Vista constituye la interfaz gráfica desarrollada en JavaFX y el Controlador coordina la interacción entre ambos componentes, gestionando además las operaciones de persistencia mediante JDBC y MySQL.

Como principal desventaja, el patrón MVC requiere una mayor organización inicial y un número superior de clases respecto de una implementación monolítica. Sin embargo, considerando que el proyecto fue concebido para crecer mediante la incorporación de nuevos módulos, los beneficios obtenidos en términos de mantenibilidad, escalabilidad y reutilización del código justifican ampliamente su utilización.

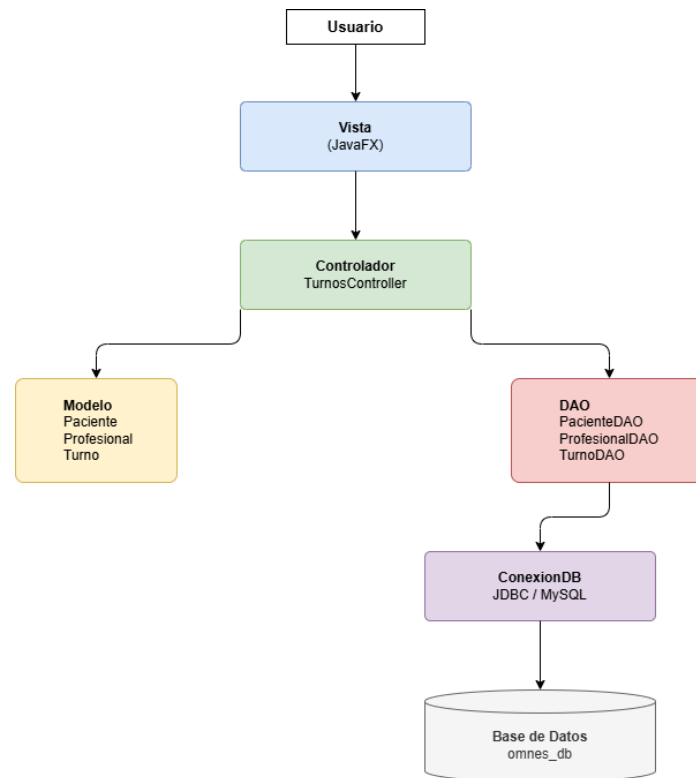


Figura 54 – Esquema general del patrón MVC implementado en el prototipo.

La arquitectura presentada constituye la base sobre la cual se organizó el desarrollo del prototipo. En el siguiente apartado se describe cómo fueron distribuidas las clases y componentes del sistema dentro de esta estructura, detallando la implementación de los modelos, la vista, el controlador y la capa de acceso a datos.

Selección del patrón de diseño y justificación

La arquitectura del prototipo fue organizada siguiendo la estructura definida por el patrón Modelo–Vista–Controlador (MVC), distribuyendo las responsabilidades de la aplicación en componentes independientes para facilitar su mantenimiento, escalabilidad y reutilización.

El Modelo (Model) está compuesto por las clases que representan las entidades principales del sistema: la clase abstracta Persona, de la cual heredan Paciente y Profesional, además de la clase Turno, encargada de representar la asignación de un paciente a un profesional en una fecha y horario determinados. Estas clases contienen la información y el comportamiento correspondiente al dominio del problema.

La Vista (View) se implementó mediante JavaFX, utilizando archivos FXML para la construcción de la interfaz gráfica y sus correspondientes controladores de eventos. Esta capa es responsable de presentar la información al usuario y capturar las acciones realizadas durante la utilización del sistema.

El Controlador (Controller) coordina la interacción entre la Vista y el Modelo, procesando las solicitudes del usuario, ejecutando las validaciones necesarias y gestionando las operaciones correspondientes sobre la información del sistema.

Para la persistencia de los datos se incorporó una capa de acceso a datos (DAO), responsable de establecer la comunicación con la base de datos MySQL mediante JDBC. Esta capa permite aislar la lógica de acceso a datos del resto de la aplicación, favoreciendo una mayor modularidad y facilitando futuras tareas de mantenimiento o ampliación del sistema.

La organización del proyecto en paquetes independientes permite mantener una estructura ordenada y coherente con la arquitectura MVC, facilitando el desarrollo incremental del prototipo y la incorporación de nuevos módulos en futuras etapas del proyecto.

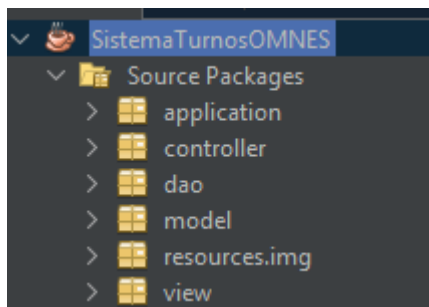


Figura 55 – Organización del proyecto siguiendo la arquitectura Modelo–Vista–Controlador (MVC).

Arquitectura MVC

Una vez seleccionado el patrón Modelo–Vista–Controlador (MVC), se definió la arquitectura del prototipo organizando cada componente según su responsabilidad dentro del sistema. Esta estructura permitió separar la lógica de negocio, la interfaz de usuario y el acceso a los datos, favoreciendo un desarrollo modular y facilitando la incorporación de nuevas funcionalidades.

En la implementación propuesta, el Modelo está constituido por las clases del dominio del sistema (Persona, Paciente, Profesional y Turno), responsables de representar las entidades principales del módulo de gestión de turnos. Estas clases encapsulan la información y el comportamiento propio de cada objeto, independientemente de la interfaz o de la base de datos.

La Vista corresponde a la interfaz gráfica desarrollada mediante JavaFX y Scene Builder, desde donde el usuario registra pacientes, profesionales y turnos, además de consultar la información almacenada. La interfaz actúa únicamente como medio de interacción, sin contener lógica de negocio ni acceso directo a la base de datos.

El Controlador administra la comunicación entre la Vista y el Modelo. Recibe las acciones realizadas por el usuario, valida la información ingresada, coordina las operaciones correspondientes y solicita el

acceso a los datos mediante las clases DAO. De esta manera, concentra el flujo de ejecución de la aplicación sin depender de la implementación de la interfaz gráfica ni de la base de datos.

Para la persistencia de la información se incorporó una capa de acceso a datos (DAO), encargada de establecer la comunicación con MySQL mediante JDBC. Esta capa encapsula las operaciones de consulta, inserción y recuperación de información, evitando que la lógica de negocio dependa directamente de instrucciones SQL.

La Figura 56 presenta la arquitectura MVC implementada en el prototipo desarrollado. Puede observarse el flujo completo de la información desde la interacción del usuario con la interfaz gráfica desarrollada en JavaFX, pasando por el controlador de la aplicación, las entidades del dominio y la capa de acceso a datos (DAO), hasta la persistencia de la información en MySQL mediante JDBC. Esta organización permitió desacoplar las responsabilidades de cada componente, facilitando el mantenimiento, la escalabilidad y la incorporación de nuevas funcionalidades durante el desarrollo del sistema.

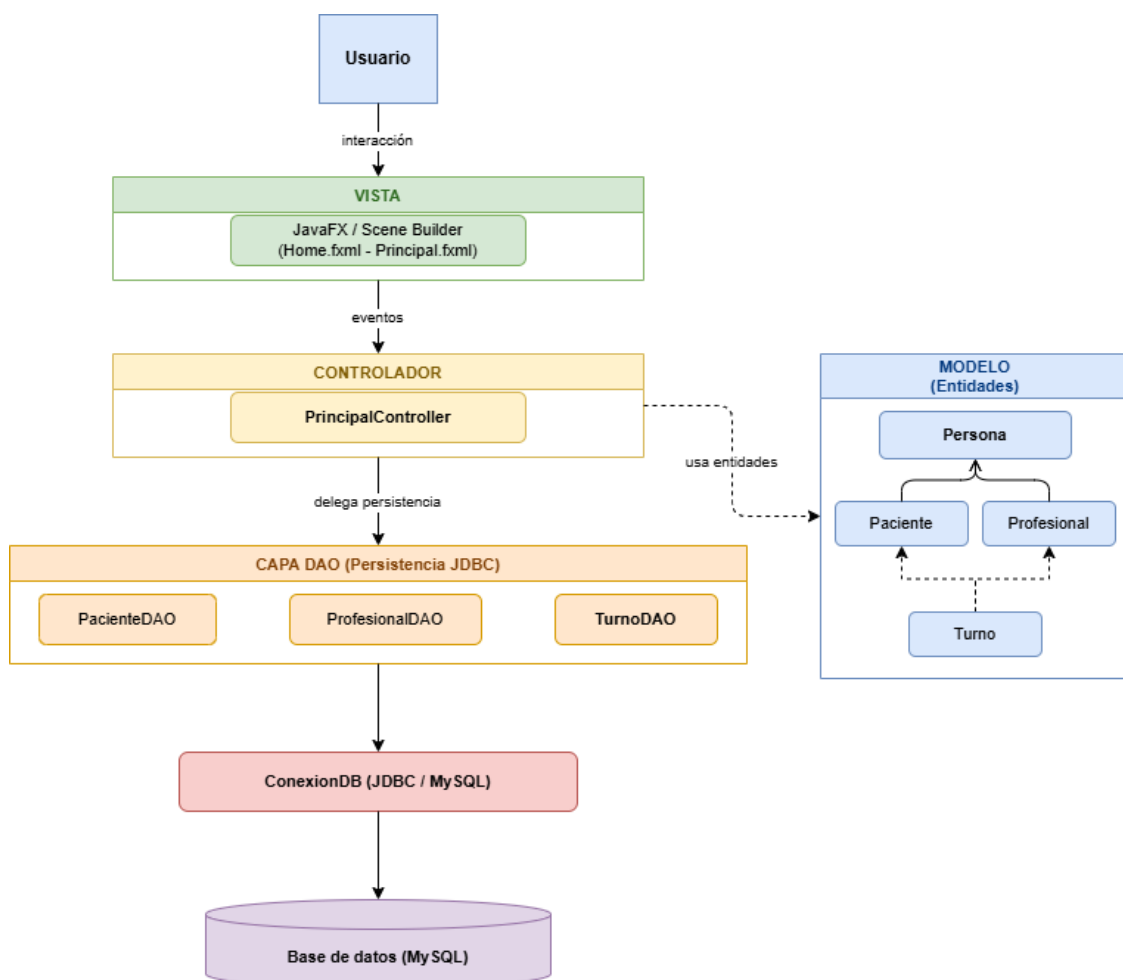


Figura 56 – Arquitectura Modelo–Vista–Controlador (MVC) implementada en el prototipo del sistema.

Desarrollo de las clases

Una vez definida la arquitectura Modelo–Vista–Controlador (MVC), se desarrollaron las clases necesarias para implementar el funcionamiento completo del sistema, incorporando una interfaz gráfica desarrollada con JavaFX y un mecanismo de persistencia de datos mediante MySQL y JDBC.

El proyecto fue organizado en paquetes de acuerdo con la responsabilidad de cada componente, respetando los principios de modularidad y separación de responsabilidades propuestos por el patrón MVC. Esta organización permitió mantener desacopladas la lógica de negocio, la presentación y el acceso a los datos, facilitando tanto el desarrollo como el mantenimiento futuro de la aplicación.

El paquete model reúne las entidades del dominio del sistema. En él se implementaron las clases Persona, Paciente, Profesional y Turno, responsables de representar la información utilizada por la aplicación. Estas entidades encapsulan los atributos y comportamientos propios de cada objeto, siendo utilizadas por el resto de las capas sin depender de la interfaz gráfica ni de la base de datos.

El paquete dao implementa la capa de persistencia mediante el patrón Data Access Object (DAO). Para ello se desarrollaron las clases ConexionDB, encargada de establecer la conexión con la base de datos MySQL mediante JDBC, y las clases PacienteDAO, ProfesionalDAO y TurnoDAO, responsables de ejecutar las operaciones de consulta, inserción, modificación y actualización de la información almacenada.

La lógica de interacción entre la interfaz gráfica y el modelo se implementó en el paquete controller. La clase HomeController administra la pantalla inicial del sistema, mientras que PrincipalController concentra la gestión integral de los turnos y coordina la comunicación entre la interfaz y la capa de persistencia. Asimismo, las clases PacientesController y ProfesionalesController administran las operaciones correspondientes a cada uno de estos módulos.

Finalmente, el paquete view contiene los archivos FXML desarrollados con SceneBuilder, responsables de definir la estructura visual de las distintas pantallas de la aplicación. Entre ellas se encuentran la pantalla de bienvenida, la pantalla principal de gestión de turnos y las ventanas destinadas a la administración de pacientes y profesionales.

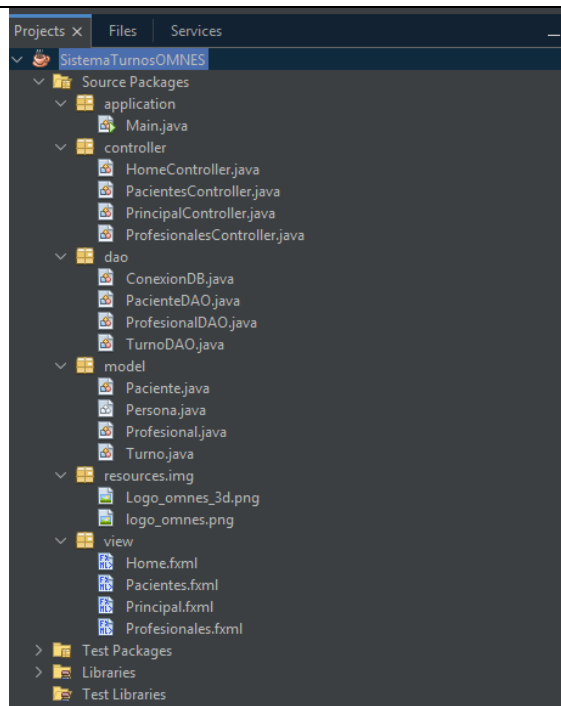


Figura 57 – Arquitectura Modelo–Vista–Controlador (MVC) implementada en el prototipo del sistema.

La Figura 57 presenta la organización del proyecto implementado, mostrando la distribución de las clases según la arquitectura MVC adoptada para el desarrollo del sistema.

Interfaz gráfica del sistema

Una vez implementadas las clases del modelo, la capa de persistencia y los controladores, se desarrolló la interfaz gráfica del sistema utilizando JavaFX y Scene Builder. El objetivo fue proporcionar un entorno visual intuitivo que permitiera administrar la información del sistema sin necesidad de interactuar directamente con la base de datos o mediante comandos por consola.

La aplicación fue diseñada siguiendo criterios de simplicidad y organización, distribuyendo las funcionalidades en distintas pantallas según la tarea que realiza el usuario. De esta manera, la navegación del sistema comienza con una pantalla de bienvenida y continúa hacia el módulo principal, desde donde es posible acceder a la gestión de pacientes, profesionales y turnos.



Sistema Integral Fundación OMNES

Módulo de Gestión de Turnos

Iniciar Sistema

© Fundación OMNES - Sistema Integral

Figura 58 – Pantalla de bienvenida del Sistema Integral Fundación OMNES.

La Figura 58 muestra la pantalla de bienvenida del sistema, la cual constituye el punto de acceso a la aplicación e incorpora la identidad visual de la Fundación OMNES.

Luego de iniciar la aplicación, el usuario accede a la pantalla principal del módulo de gestión de turnos. Desde esta interfaz es posible registrar nuevos turnos, modificar la información existente, cancelar turnos previamente registrados y consultar la agenda almacenada en la base de datos.

Sistema Integral Fundación OMNES

Gestión de Turnos

Gestionar Pacientes Gestionar Profesionales

Paciente:

Profesional:

Fecha:

Hora:

Duración (min):

Estado:

Guardar Modificar Cancelar Turno Limpiar

Turnos Registrados

Paciente	Profesional	Fecha	Hora	Duración	Estado
Juan Pérez	María Gómez	2026-06-06	08:30	60	Programado
Juan Pérez	María Gómez	2026-06-30	11:30	60	Cancelado

Estado: Listo Base de datos: MySQL Versión 1.0

Figura 59 – Pantalla principal del módulo de gestión de turnos del Sistema Integral Fundación OMNES.

Para administrar la información de las personas que participan en el proceso, el sistema incorpora una ventana destinada a la gestión de pacientes, desde la cual es posible registrar nuevos pacientes, modificar sus datos y consultar los registros existentes.

Gestión de Pacientes

Nombre:

DNI:

Obra Social:

Guardar Modificar Eliminar Limpiar

Pacientes registrados

Seleccione un paciente de la tabla para modificarlo o eliminarlo.

Nombre	DNI	Obra Social
Juan Pérez	32145678	APROSS

Figura 60 – Ventana de gestión de pacientes.

De manera similar, la aplicación dispone de un módulo para la administración de profesionales, permitiendo registrar especialistas, actualizar su información y mantener disponible el listado utilizado posteriormente durante la asignación de turnos.

Gestión de Profesionales

Nombre:

Especialidad:

Matrícula:

CUIT:

Porcentaje de Aporte:

Profesionales registrados

Seleccione un profesional de la tabla para modificarlo o eliminarlo.

Nombre	Especialidad	Matrícula	CUIT	Porcentaje de Apo...
María Gómez	Psicopedagogía	MP-1234	27234567891	15.0

Figura 61 – Ventana de gestión de profesionales.

Finalmente, la interfaz principal integra el módulo de gestión de turnos, donde la administrativa puede asociar pacientes y profesionales, seleccionar la fecha, el horario y la duración de cada atención, además de administrar el estado correspondiente del turno. La información registrada se actualiza automáticamente en la tabla de turnos y permanece almacenada en la base de datos.

Gestión de Turnos

Turno modificado correctamente.

Paciente: Julian Martinez

Profesional: David Emanuel Gigena - Psicomotricista

Fecha: 30/6/2026

Hora: 16:30

Duración (min): 60

Estado: Atendido

Guardar Modificar Cancelar Turno Limpiar

Turnos Registrados

Paciente	Profesional	Fecha	Hora	Duración	Estado
Juan Pérez	María Gómez	2026-06-06	08:30	60	Programado
Juan Pérez	María Gómez	2026-06-30	11:30	60	Cancelado
Julian Martinez	David Emanuel Gigena	2026-06-30	16:30	60	Programado

Estado: Listo Base de datos: MySQL Versión 1.0

Figura 62 – Ejemplo de gestión de turnos con información persistida en la base de datos.

La implementación de esta interfaz permitió reemplazar la versión desarrollada inicialmente por consola por una aplicación de escritorio más amigable para el usuario, manteniendo la separación de responsabilidades definida por la arquitectura MVC y facilitando futuras ampliaciones del sistema.

Persistencia de datos

Con el objetivo de garantizar el almacenamiento permanente de la información, el sistema incorpora una capa de persistencia implementada mediante JDBC y una base de datos MySQL. Esta solución permite conservar los registros generados por la aplicación, independientemente del cierre o reinicio del sistema.

La comunicación entre la aplicación y la base de datos se realiza a través de la clase ConexionDB, responsable de establecer la conexión con MySQL utilizando el controlador JDBC. Sobre esta conexión trabajan las clases PacienteDAO, ProfesionalDAO y TurnoDAO, las cuales implementan las operaciones necesarias para registrar, consultar y actualizar la información correspondiente a cada entidad del sistema.

Cada acción ejecutada desde la interfaz gráfica genera una operación sobre la base de datos. Por ejemplo, al registrar un nuevo paciente o profesional, la información es almacenada inmediatamente

en las tablas correspondientes. Del mismo modo, la creación, modificación y cancelación de turnos se reflejan automáticamente en la tabla Turno, manteniendo sincronizada la información entre la aplicación y la base de datos.

La Figura XX presenta el esquema de tablas implementado en MySQL, mientras que las Figuras XX, XX y XX muestran ejemplos de la información persistida para pacientes, profesionales y turnos luego de ser registrada desde la aplicación.

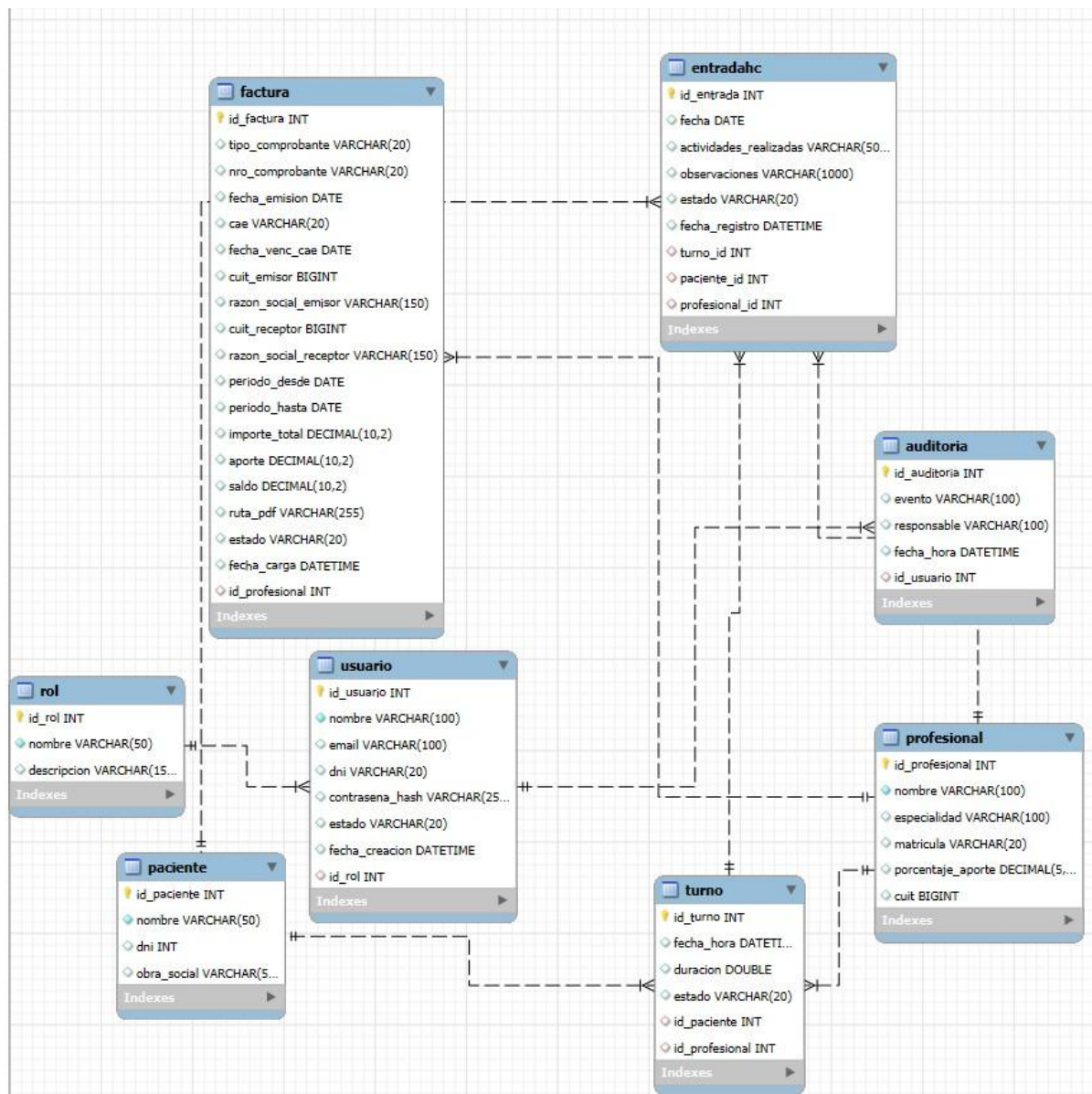


Figura 63 – Esquema de tablas implementado en MySQL.

202

203 • `SELECT * FROM Paciente;`

204

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

	id_paciente	nombre	dni	obra_social
▶	1	Juan Pérez	32145678	APROSS
	5	David gigena	36987196	Swiss medical
	6	Julian Martinez	12345678	Osde
	7	Leonardo Perez	23654123	Osde
*	NULL	NULL	NULL	NULL

Figura 64 – Registros almacenados en la tabla Paciente.

202

203 • `SELECT * FROM Profesional;`

204

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

	id_profesional	nombre	especialidad	matricula	porcentaje_aporte	cuit
▶	1	María Gómez	Psicopedagogía	MP-1234	15.00	27234567891
	4	David Emanuel Gigena	Psicomotricista	MP-5265	15.00	20369871960
*	NULL	NULL	NULL	NULL	NULL	NULL

Figura 65 – Registros almacenados en la tabla Profesional.

202

203 • `SELECT * FROM Turno;`

204

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

	id_turno	fecha_hora	duracion	estado	id_paciente	id_profesional
▶	1	2026-06-06 08:30:00	60	Programado	1	1
	7	2026-06-30 11:30:00	60	Cancelado	1	1
	8	2026-06-30 16:30:00	60	Cancelado	6	4
*	NULL	NULL	NULL	NULL	NULL	NULL

Figura 66 – Registros almacenados en la tabla Turno.

La utilización del patrón DAO permitió desacoplar la lógica de negocio del acceso a los datos, evitando que la interfaz gráfica interactúe directamente con la base de datos. Esta organización favorece la mantenibilidad del sistema y facilita futuras modificaciones sobre el mecanismo de persistencia sin afectar el resto de la aplicación.

Manejo de excepciones

Durante el desarrollo del prototipo se incorporó un mecanismo básico de manejo de excepciones con el objetivo de evitar interrupciones inesperadas en la ejecución de la aplicación y brindar una mejor experiencia al usuario.

Las operaciones relacionadas con la conexión a la base de datos y la ejecución de consultas SQL fueron encapsuladas mediante bloques try-catch, permitiendo capturar errores de conexión, consultas inválidas o problemas de persistencia sin provocar el cierre de la aplicación.

De igual manera, las acciones ejecutadas desde la interfaz gráfica validan la información ingresada antes de procesarla. Cuando el usuario omite datos obligatorios o se produce alguna situación excepcional durante la ejecución, el sistema informa el problema mediante mensajes de alerta utilizando los componentes de JavaFX, evitando comportamientos inesperados y facilitando la corrección por parte del usuario.

Este tratamiento de excepciones permitió incrementar la robustez del prototipo y mejorar la confiabilidad general del sistema durante su utilización.

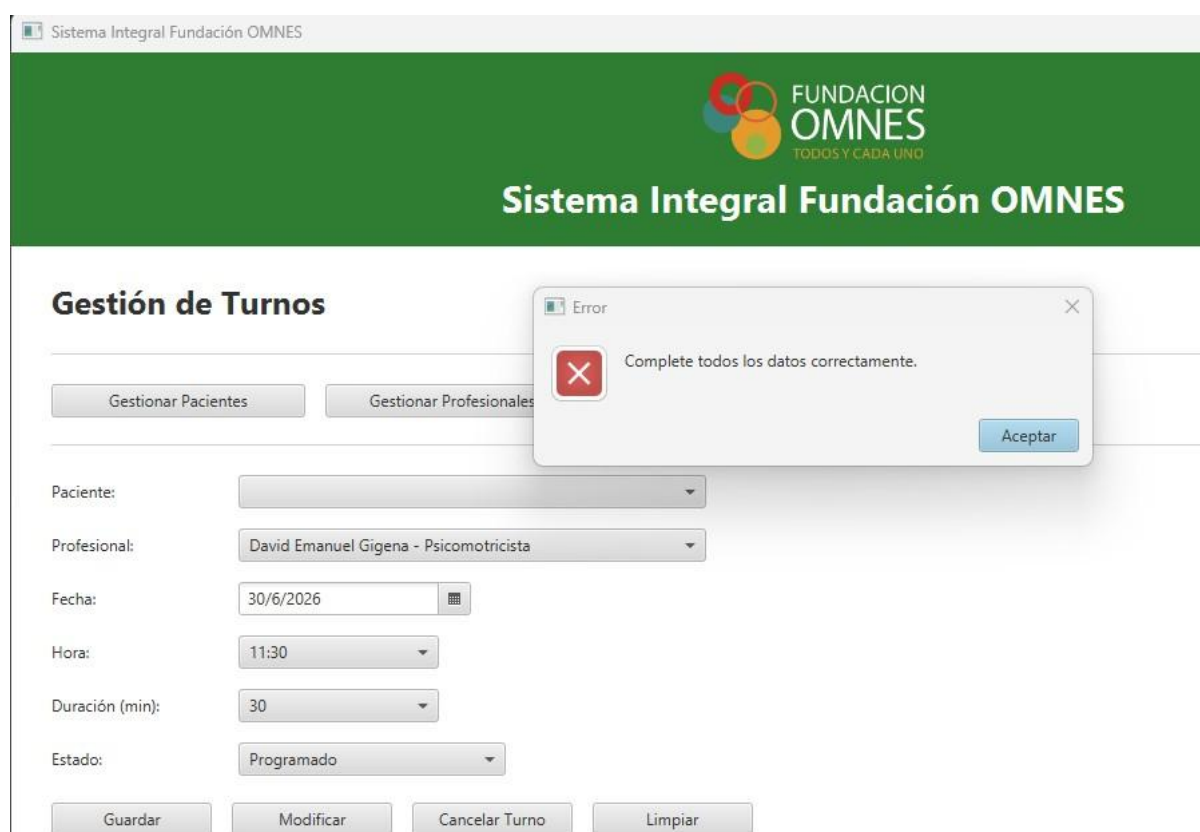


Figura 67 – Ejemplo de validación implementada mediante mensajes de alerta ante datos incompletos.

Uso de ArrayList

Las colecciones dinámicas implementadas mediante la clase ArrayList fueron utilizadas para administrar la información recuperada desde la base de datos. Cada clase DAO obtiene los registros correspondientes mediante consultas SQL y los almacena temporalmente en listas, permitiendo posteriormente transferir dicha información hacia los componentes gráficos de JavaFX.

Este mecanismo facilita el recorrido de los registros obtenidos y permite que los controles visuales, como las tablas (TableView) y listas desplegables (ComboBox), puedan cargarse dinámicamente con la información almacenada en MySQL.

La utilización de ArrayList permitió mantener una estructura flexible para el manejo de colecciones de objetos, evitando el uso de arreglos de tamaño fijo y simplificando la interacción entre la capa de persistencia y la interfaz gráfica.

Verificación y Validación (V&V)

Con el propósito de verificar el correcto funcionamiento del sistema desarrollado, se realizaron pruebas funcionales sobre cada uno de los módulos implementados.

En primer lugar, se verificó el funcionamiento de la conexión con la base de datos MySQL, comprobando la correcta persistencia de la información mediante las operaciones de alta, modificación y consulta realizadas desde la aplicación.

Posteriormente se evaluó el funcionamiento de los módulos de gestión de pacientes, profesionales y turnos, verificando la correcta interacción entre la interfaz gráfica, los controladores y la capa de acceso a datos.

Finalmente, se comprobó la actualización automática de la información visual luego de cada operación realizada, verificando además que los datos persistidos coincidieran con los registros almacenados en la base de datos.

Las pruebas efectuadas permitieron confirmar el cumplimiento de los requerimientos funcionales definidos para el prototipo y validar la correcta integración de los distintos componentes desarrollados.

Caso de prueba	Resultado esperado	Resultado obtenido
Registrar paciente	Paciente almacenado	Correcto
Registrar profesional	Profesional almacenado	Correcto
Registrar turno	Turno creado	Correcto
Modificar turno	Información actualizada	Correcto

Cancelar turno	Estado = Cancelado	Correcto
Consultar turnos	Datos visibles	Correcto
Persistencia	Datos guardados en MySQL	Correcto

Conclusión

El desarrollo del presente trabajo permitió aplicar de manera integrada los conocimientos adquiridos durante la carrera en un proyecto que abarcó todas las etapas fundamentales del proceso de desarrollo de software. A partir del relevamiento de necesidades de la Fundación OMNES se realizó el análisis del sistema existente, la propuesta de mejora mediante el modelo TO-BE, la definición de los requisitos funcionales, el modelado utilizando diagramas UML y el diseño de la arquitectura de la solución.

Como resultado de este proceso se desarrolló un prototipo funcional del módulo de Gestión de Turnos, implementado mediante el lenguaje Java, utilizando JavaFX para la interfaz gráfica, JDBC como mecanismo de acceso a datos y MySQL como sistema gestor de base de datos. La aplicación permite administrar pacientes, profesionales y turnos, incorporando persistencia de datos, validaciones de información y una estructura basada en la arquitectura Modelo–Vista–Controlador (MVC), favoreciendo la organización del código, su mantenimiento y futuras ampliaciones.

La utilización del patrón DAO permitió desacoplar la lógica de acceso a datos de la lógica de negocio, mientras que la organización del proyecto en capas contribuyó a obtener una solución más modular y escalable. Asimismo, la incorporación de una interfaz gráfica mejoró significativamente la experiencia de uso respecto de la primera versión desarrollada por consola, representando una evolución natural del prototipo inicialmente construido.

Si bien el sistema implementa únicamente uno de los módulos previstos dentro del proyecto integral para la Fundación OMNES, su diseño fue realizado considerando una arquitectura que permitirá incorporar, en futuras etapas, funcionalidades como historias clínicas, facturación, gestión de usuarios, auditoría, reportes y administración de tareas, manteniendo una estructura consistente y reutilizable.

Finalmente, el desarrollo de este trabajo permitió consolidar competencias relacionadas con el análisis, diseño e implementación de sistemas de información, integrando herramientas, metodologías y buenas prácticas de ingeniería de software. La experiencia obtenida constituye una base sólida para el desarrollo de proyectos de mayor complejidad y representa una aplicación práctica de los contenidos abordados a lo largo de la carrera.

Repositorio del proyecto final

Se actualiza y registra el repositorio con los archivos correspondientes la implementación en java del modulo de gestión de turnos.

Repositorio: <https://github.com/Davidemanuelgigena/sistema-gestion-omnes>



Figura 68 – Repositorio GitHub utilizado para la gestión de versiones del proyecto actualizado.

Referencias

- Benvenuto, M. (s. f.). *Entrevistas en el análisis de sistemas*. <https://shorturl.at/ayFHU>
- Casalini, L. (2018). *Validación de requerimientos*. <https://shorturl.at/ktHW3>
- Cedeño, R. (2017). *Sistemas de información: Conceptos y aplicaciones*. Editorial ULEAM.
- Díaz Ríos, J. (s. f.). *Características de los requerimientos de software*. <https://shorturl.at/bcgqO>
- Kendall, K. E., & Kendall, J. E. (2011). *Análisis y diseño de sistemas* (8ª ed.). Pearson.
- Saroka, D. (2002). *Gestión de la información y toma de decisiones en organizaciones*. Editorial UBA.
- Sommerville, I. (2005). *Ingeniería del software* (7ª ed.). Pearson Addison Wesley.
- Booch, G. (1996). *Análisis y diseño orientado a objetos, con aplicaciones*. Addison Wesley Iberoamericana S. A.
- Braude, E. (2003). *Ingeniería de software, una perspectiva orientada a objetos*. Alfaomega Grupo Editor S. A.
- Institute of Electrical and Electronics Engineers Standards Association. (2008). Standard for Software Quality Assurance Plans (ANSI/IEEE estándar n. ° 829).
- Institute of Electrical and Electronics Engineers Standards Association. (1990). Standard for Software and System Test (ANSI/IEEE estándar n. ° 610).
- Rumbaugh, J., Jacobson, I. y Booch, G. (2006). *El lenguaje unificado de modelado* (2.a ed.). Pearson-Addison Wesley.
- Oracle. (2024). *Java Platform, Standard Edition Documentation*. <https://docs.oracle.com/en/java/>
- GitHub. (2024). *GitHub Documentation*. <https://docs.github.com>

MySQL. (2024). *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/>

Deitel, P., y Deitel, H. (2016). *Cómo programar a Java*. Editorial Pearson.

Equipo Geek. (2019). *Clases abstractas e interfaces: ¿qué son?* <https://ifgeekthen.nttdata.com/es/clases-abstractas-e-interfaces#:~:text=Una%20clase%20abstracta%20puede%20contener,los%20m%C3%A9todos%20s%C3%AD%20debe%20serlo>.

Garro, A. (s.f.). *Java. Capítulo 18 interfaces*. <https://www.arkaitzgarro.com/java/capitulo-18.html#declaracion-de-una-interfaz>

Martin, J., y Odell, J. (1995). *Principios del Análisis y Diseño Orientado a Objetos*. Editorial Prentice-Hall.

Meza González, J.D. (2021). Modificadores de acceso public, protected, default y private en Java. Encapsulamiento en Java. *Programar ya*. <https://www.programarya.com/Cursos/Java/Modificadores-de-Acceso>

Pressman, R. (2010). *Ingeniería del Software: Un enfoque práctico*. Editorial McGraw-Hill.

Shaw, M., y Garlan, D. (2015). *Software Architecture: Perspectives on an Emerging Discipline*. Editorial Pearson India.

Oracle. (2024). *JDBC Basics*. <https://docs.oracle.com/javase/tutorial/jdbc/>

Oracle. (2024). *JavaFX FXML Tutorial*. <https://docs.oracle.com/javase/8/javafx/api/javafx/fxml/package-summary.html>